

High-End Timer (N2HET) Module

This chapter provides a general description of the High-End Timer (N2HET). The N2HET is a software-controlled timer with a dedicated specialized timer micromachine and a set of 30 instructions. The N2HET micromachine is connected to a port of up to 32 input/output (I/O) pins.

NOTE: This chapter describes a superset implementation of the N2HET module that includes features and functionality that require DMA. Since not all devices have DMA capability, consult your device-specific datasheet to determine the applicability of these features and functions to your device being used.

Topic	Page
23.1 Overview.....	1072
23.2 N2HET Functional Description.....	1076
23.3 Angle Functions	1108
23.4 N2HET Control Registers	1135
23.5 HWAG Registers.....	1162
23.6 Instruction Set.....	1178

23.1 Overview

The N2HET is a fifth-generation Texas Instruments (TI) advanced intelligent timer module. It provides an enhanced feature set compared to previous generations.

This timer module provides sophisticated timing functions for real-time applications such as engine management or motor control. The high resolution hardware channels allow greater accuracy for widely used timing functions such as period and pulse measurements, output compare, and PWMs.

The reduced instruction set, based mostly on very simple, but comprehensive instructions, improves the definition and development cycle time of an application and its derivatives. The N2HET breakpoint feature, combined with various stop capabilities, makes the N2HET software application easy to debug.

23.1.1 Features

- Programmable timer for input and output timing functions
- Reduced instruction set (30 instructions) for dedicated time and angle functions
- Up to maximum of 128 96-bit words of instruction RAM protected by parity. Check your datasheet for the actual number of words implemented.
- User defined configuration of 25-bit virtual counters for timer, event counters and angle counters
- 7-bit hardware counters for each pin allow up to 32-bit resolution in conjunction with the 25-bit virtual counters
- Up to 32 pins usable for input signal measurements or output signal generation
- Programmable suppression filter for each input pin with adjustable suppression window
- Low CPU overhead and interrupt load
- Efficient data transfer to or from the CPU memory with dedicated High-End-Timer Transfer Unit (HTU) or DMA
- Diagnostic capabilities with different loopback mechanisms and pin status readback functionality
- Hardware Angle Generator (HWAG)

23.1.2 Major Advantages

In addition to classic time functions such as input capture or multiple PWMs, higher-level time functions can be easily implemented in the timer program main loop. Higher-level time functions include angle driven wave forms, angle- and time-driven pulses, and input pulse width modulation (PWM) duty cycle measurement.

Because of these high-level functions, data exchanges with the CPU are limited to the fundamental parameters of the application (periods, pulse widths, angle values, etc.); and the real-time constraints for parameter communication are dramatically minimized; for example, few interrupts are required and asynchronous parameter updates are allowed.

The reduced instruction set and simple execution flow control make it simple and easy to develop and modify programs. Simple algorithms can embed the entire flow control inside the N2HET program itself. More complex algorithms can take advantage of the CPU access to the N2HET RAM. With this, the CPU program can make calculations and can modify the timer program flow by changing the data and control fields of the N2HET RAM. CPU access to the N2HET RAM also improves the debug and development of timer programs. The CPU program can stop the N2HET and view the contents of the program, control, and data fields that reside in the N2HET RAM.

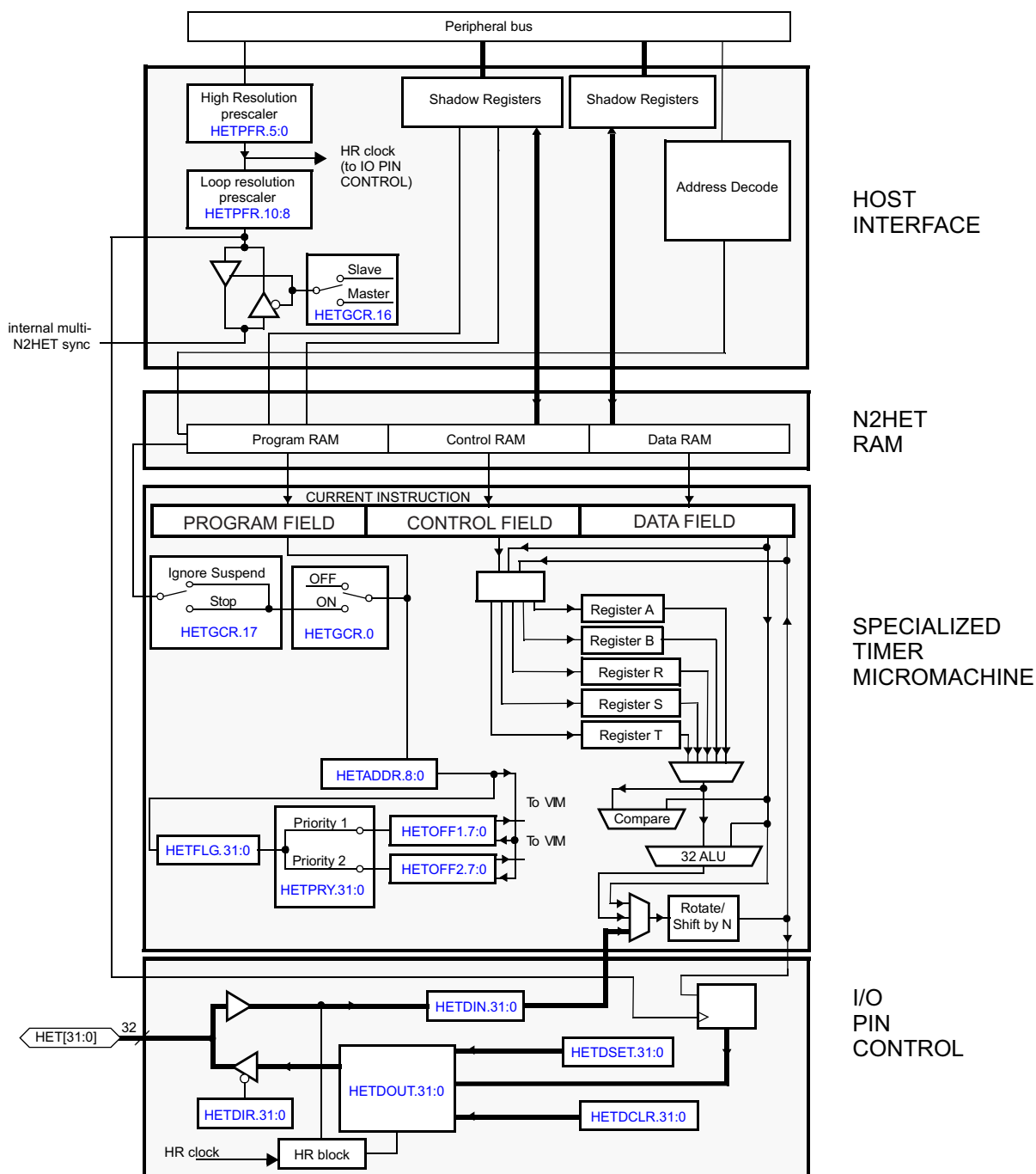
Finally, the modular structure provides maximum flexibility to address a wide range of applications. The timer resolution can be selected from two cascaded prescalers to adjust the loop resolution and HR clocks. The 32 I/O pins can provide any combination of input, period or pulse capture, and output compare, including high resolution for each channel.

23.1.3 Block Diagram

The N2HET module (see [Figure 23-1](#)) comprises four separate components:

- Host interface
- N2HET RAM
- Specialized timer micromachine
- I/O control (the N2HET is attached to an I/O port of up to 32 pins)

Figure 23-1. N2HET Block Diagram



23.1.4 Timer Module Structure and Execution

The timer consists of a specialized micromachine that operates a reduced instruction set. Two 25-bit registers and three 32-bit registers are available to manipulate information such as time, event counts, and angle values. System performance is improved by a wide instruction format (96 bits) that allows the N2HET to fetch the instructional operation code and data in one system cycle, thus increasing the speed at which data can be processed. The typical operations performed in the ALU are additions (count), compares, and magnitude compares (higher or same).

Each instruction is made up of a 32-bit program field, a 32-bit control field and a 32-bit data field. The N2HET execution unit fetches the complete 96-bit instruction in one cycle and executes it. All instructions include a 9-bit field for specifying the address of the next instruction to be executed. Some instructions also include a 9-bit conditional address, which is used as the next address whenever a particular condition is true. This makes controlling the flow of an N2HET program inexpensive; in many cases a separate branch instruction is not required.

The interface to the host CPU is based on both communication memory and control registers. The communication memory includes timer instructions (program and data). This memory is typically initialized by the CPU or DMA after reset before the timer starts execution. Once the timer program is loaded into the memory, the CPU starts the timer execution, and typically data parameters are then read or written into the timer memory. The control registers include bits for selecting timer clock, configuring I/O pins, and controlling the timer module.

The programmer implements timer functions by combining instructions in specific sequences. For instance, a single count (CNT) instruction implements a timer. A simple PWM generator can be implemented with a two instruction sequence: CNT and compare (ECMP or MCMP). A complex time function may include many instructions in the sequence. The total timer program is a set of instructions executed sequentially, one after the other. Reaching the end, the program must roll to the first instruction so that it behaves as a loop. The time for a loop to execute is referred to as a *loop resolution clock cycle* or *loop resolution period (LRP)*. When the N2HET rolls over to the first instruction, the timer waits for the loop resolution clock to restart the execution of the loop to ensure that only one loop is executed for each loop resolution clock.

The longest path through an N2HET program must be completed within the loop resolution clock (LRP). Otherwise, the program will execute unpredictably because some instructions will not be executed each time through the loop. This effect creates a strong link between the accuracy of the timer functions and the number of functions (the number of instructions) the timer can perform. High resolution (HR) hardware timer extensions are available for each of the N2HET pins to help overcome this limitation.

The high resolution hardware timers operate from the *high resolution clock*, which may be configured for frequency multiples between 2 and 128 times the loop resolution clock frequency. This extending the resolution of timer events and measurements well beyond what is possible with only loop resolution instructions.

Most of the commonly used N2HET instructions can operate either at loop resolution or high resolution; with the restriction that for each pin at most one high resolution instruction can be executed per loop resolution period.

Certain instructions (MOV32, ADM32, ...) can modify the data fields of other instructions. This feature enables the N2HET program to implement double buffering on capture and compare functions. For example, an ECMP compare instruction can be followed by a MOV32 instruction that is conditionally executed when the ECMP instruction matches. The host CPU can update the next compare value by writing asynchronously to the data field of the MOV32 instruction instead of writing directly to the data field of the ECMP instruction. The copy from the buffer (MOV32 data field) to the compare register (ECMP data field) will occur when the MOV32 instruction is actually executed which occurs after the ECMP instruction matches its current compare value. This is the same behavior as one would expect from a double buffered hardware compare register.

Other instructions (MOV64, RADM64) can modify both the control and data fields of other instructions. This allows the N2HET to implement toggle functionality. For example, an ECMP instruction can be followed by a pair of MOV64 instructions. The MOV64 instruction updates the data field of the ECMP instruction to implement the double buffering behavior. But it also updates the control field of the ECMP instruction which allows it to change things like pin action and the conditional address. If one MOV64 instruction configures the ECMP pin action to SET while the second changes it to CLEAR, and the two MOV64 instructions update the conditional address to point to each other, then a single ECMP instruction can be used to toggle a pin each time the compare match occurs.

23.1.5 Performance

Most instructions execute in one cycle, but a few take two or three cycles.

The N2HET can generate many complex output waveforms without CPU interrupts. Where special algorithms are needed following a specific event (for example, missing teeth or a short/long input signal), a minimal number of interrupts to the CPU are needed freeing the CPU for other tasks.

23.1.6 N2HET Compared to NHET

N2HET enhancements from NHET include:

- Eight new instructions: ADD, ADC, SUB, SBB, AND, OR, XOR, RCNT
- Full set of ALU flags Carry (C), Negative (N), Zero (Z), Overflow (V)
- Branch instruction (BR) extended to support signed and unsigned arithmetic comparison conditions
- Two additional 32-bit temporary working registers R, S.
- New HETAND register for AND-Sharing of High Resolution structure between pairs of pins
- Improved high resolution PCNT instruction

23.1.7 NHET and N2HET Compared to HET

Compared to the HET module, the N2HET contains all of the enhancements described in [Section 23.1.6](#) plus the following additional enhancements:

- New Interrupt Enable Set and Clear registers
- Capability to generate requests to the DMA module or the HET Transfer Unit (HTU) including new Request Enable Set and Clear registers
- N2HET RAM parity error detection
- Suppression filters for each of the 32 I/O channel and control register to configure the limiting frequency and counter clock
- Enhanced edge detection hardware that does not rely on the previous bit field in the control word of the N2HET instruction.
- The next, conditional and remote addresses are extended from 8 to 9 bits
- The loop resolution data fields are extended from 20 to 25 bits
- The high resolution data fields are extended from 5 to 7 bits
- Instructions with an adequate condition are able to specify the number of the request line, which triggers either the HET Transfer Unit (HTU) or the DMA module
- The CNT instruction provides a bit, which allows to configure either an equal comparison or a greater or equal comparison when comparing the selected register value with the Max-value
- The MOV32 instruction provides a new bit. If set to one the MOV32 will only perform the move, when the Z-flag is set. If set to zero the MOV32 will perform the move whenever it is executed (independent on the state of the Z-flag)
- There is a new instruction WCAPE, which is a combination of a time stamp and an edge counter
- New Open Drain, Pull Disable, and Pull Select registers

23.1.8 Instructions Features

The N2HET has the following instructions features:

- N2HET uses a RISC-based specialized timer micromachine to carry out a set of 30 instructions
- Instructions are implemented in a Very Long Instruction Word (VLIW) format (96-bits wide)
- The N2HET program execution is self-driven by external or internal events, branching to special routines based on input edges or output compares
- Instructions point to the next instruction executed, eliminating the need for a program counter
- Several instructions can change the program flow based on internal or external conditions

23.1.9 Program Usage

The N2HET instructions/program can be assembled with the N2HET assembler. The assembler generates a C-structure which can be included into the main application program. The application has to copy the content of the structure into the N2HET RAM, set up necessary registers and start the N2HET program execution. In addition to the C-structure, the assembler generates also a header file which makes it easy for the main application to access the different instructions and change for example the duty cycle of a PWM or read out the captured value of a specific signal edge.

23.2 N2HET Functional Description

The N2HET contains RAM into which N2HET code is loaded. The N2HET code is run by the specialized timer micromachine. The host interface and I/O control provide an interface to the CPU and external pins respectively.

23.2.1 Specialized Timer Micromachine

The N2HET has its own instruction set, detailed in [Section 23.6.1](#). The timer micromachine reads each instruction from the N2HET RAM. The program and control fields contain the instructions for how the specialized timer micromachine executes the command. For most instructions, the data field stores the information that needs to be manipulated.

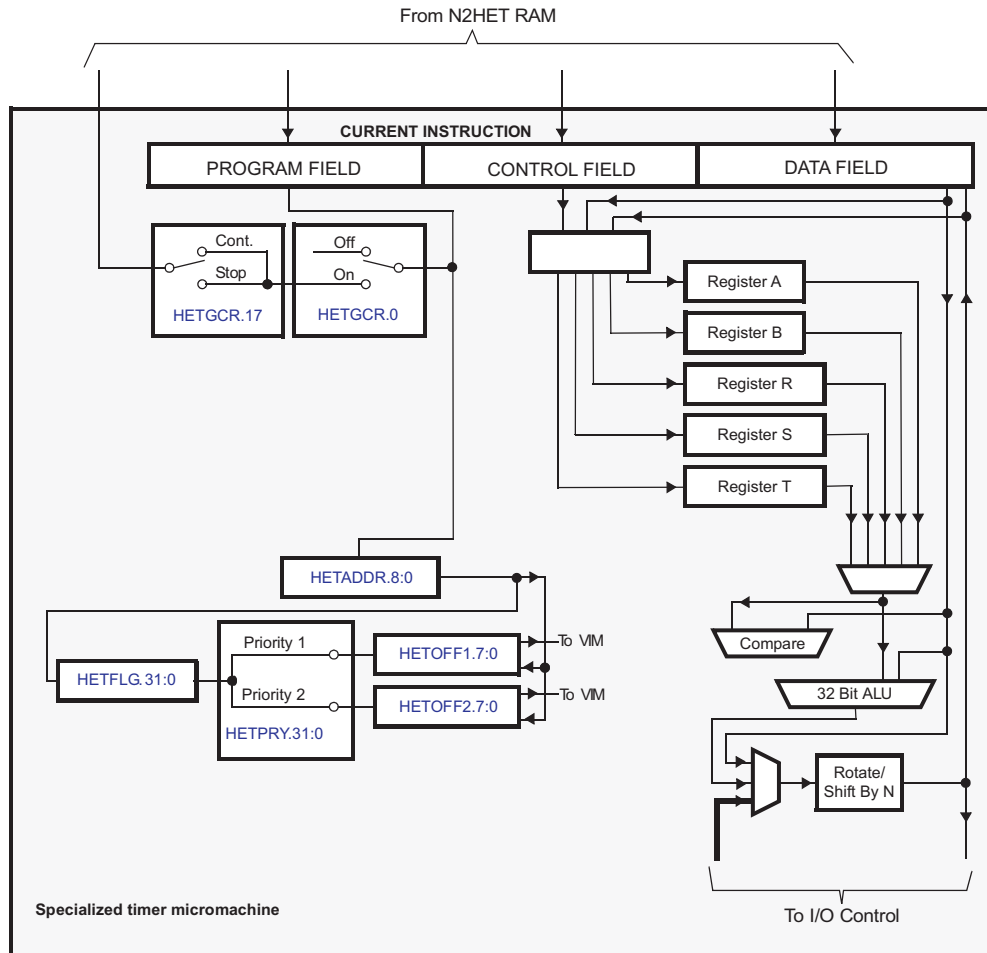
The specialized timer micromachine executes the instructions stored in the N2HET RAM sequentially. The N2HET program execution is self-driven by external or internal events. This means that input edges or output compares may force the program to branch to special routines using a conditional address.

[Figure 23-2](#) shows some of the major operations that the N2HET can carry out, namely compares, captures, angle functions, additions, and shifts. The N2HET contains five registers (A, B, R, S, and T) used to hold compare or counter values and are used by the N2HET instructions. Data may be taken from the registers or the data field for manipulation; likewise, the data may be returned to the registers or the data field.

23.2.1.1 Time Slots and Resolution Loop

Each instruction requires a specific number of cycles or time slots to execute. The resolution specified in the prescaler bitfields determines the timer accuracy. All input captures, event counts, and output compares are executed once in each resolution loop. HR captures and compares are possible (up to N2HET clock accuracy) on the HR I/O pins. For more information about the HR I/O structure, see [Section 23.2.5](#).

Figure 23-2. Specialized Timer Micromachine



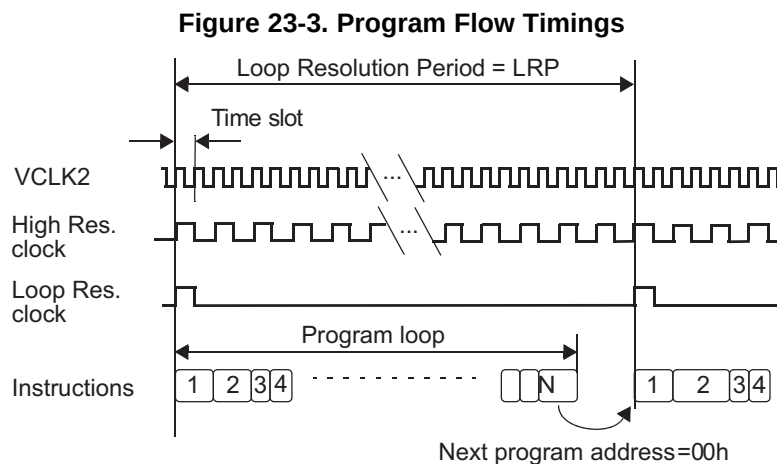
23.2.1.2 Program Loop Time

The program loop time is the sum of all cycles used for instruction execution. This time may vary from one loop to another if the N2HET program includes conditionally executed instructions.

The timer program restarts on every resolution loop. The start address is fixed at N2HET RAM address 00h. The longest path through a program must fit within one loop resolution period to guarantee complete accuracy.

The last instruction of a program must branch back to the fixed start address (next program address = 00h). When an N2HET program branches back to address 00h before the end of a loop resolution period, the N2HET detects this and pauses instruction execution until the beginning of the next loop resolution period.

The timing diagram in [Figure 23-3](#) illustrates the program flow execution.



23.2.1.3 Instruction Execution Sequence

The execution of a N2HET program begins with the first occurrence of the loop resolution clock, after the N2HET is turned on. At the first and subsequent occurrences of the loop resolution, the instruction at location address 00h is prefetched. The program execution begins at the occurrence of the loop resolution clock and continues executing the instructions until the program branches to 00h location. The instruction is prefetched at location 00h and execution flag is reset. The N2HET pauses instruction execution until the occurrence of the loop resolution clock and resumes normal execution.

N2HET programs must be written so that they complete execution and return to address 00h before the occurrence of the next loop resolution clock. If the N2HET program exceeds this execution time limit, then a program overflow condition occurs as described in [Section 23.2.1.4](#).

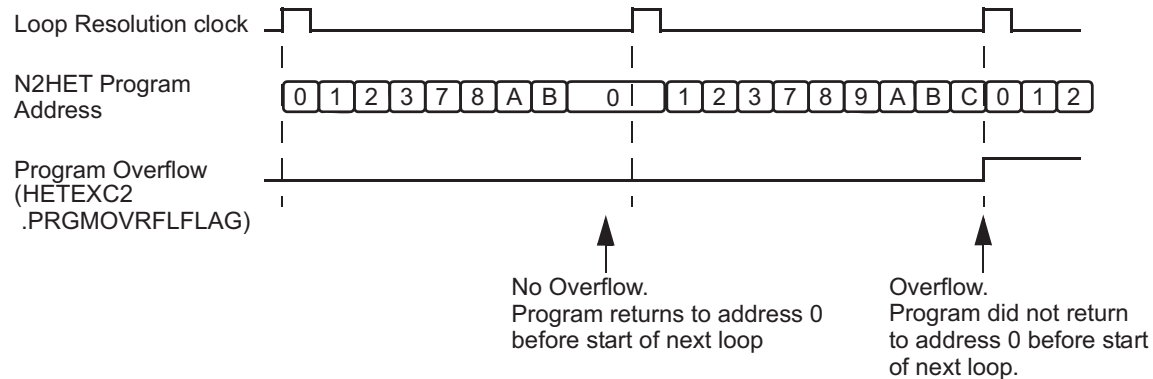
23.2.1.4 Program Overflow Condition

If the number of time slots used in a program loop exceeds the number available time slots in one loop resolution, the timer sets the program overflow interrupt flag located in the HETEXC2 register. To maintain synchronization of the I/Os, this condition should never be allowed to occur in a normal operation. The HETEXC2.PRGMOVRFLFLAG flag provides a mechanism for checking that the condition does not occur during the debug and validation phases.

As [Figure 23-4](#) illustrates, when a program overflow occurs, the currently executing N2HET program sequence is interrupted and restarted at N2HET address 0 for the beginning of the next loop resolution clock period. Also, HETEXC2.PRGMOVRFLFLAG is set.

If the instruction that caused the overflow (instruction at address 0xC in [Figure 23-4](#)) has any pin actions selected, these pin actions will not be performed. However other actions of the instruction including register and RAM updates will still be performed.

Figure 23-4. Use of the Overflow Interrupt Flag (HETEXC2)



23.2.1.5 Architectural Restrictions on N2HET Programs

Certain architectural restrictions apply to N2HET programs:

1. The size of an N2HET program must be greater than one instruction.
2. An extra wait state is incurred by any instruction that modifies a field in the next instruction to be executed.
3. Only one instruction (using high resolution) is allowed per high resolution pin.
4. Consecutive break points are not supported. Instructions with break points must have at least a distance of two instructions (for example, at addresses 1, 3, 5, 7, and so on, assuming the program executes linearly)

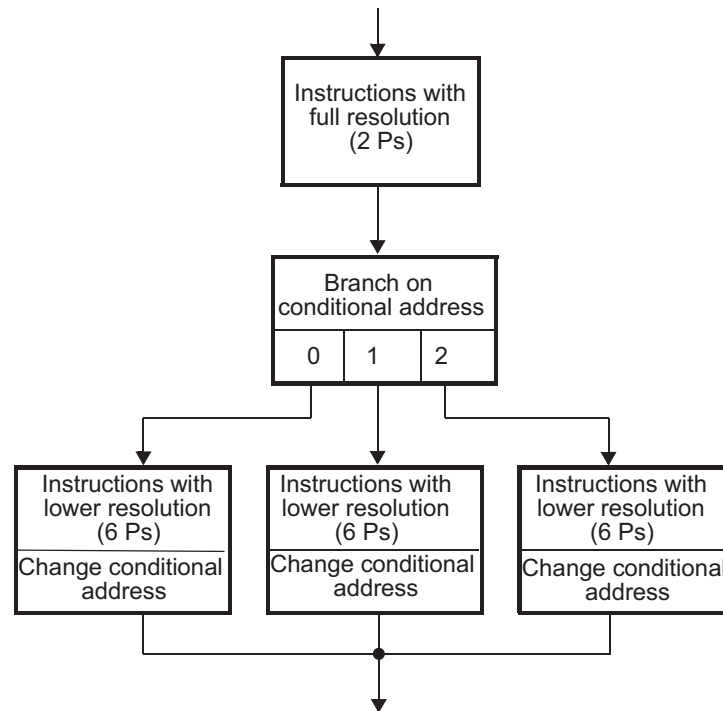
NOTE: While it would be unusual to code an N2HET program that is only one instruction long, it is trivial to modify such a program to meet the requirement of restriction 1. Simply add a second instruction to the program, which may be a simple branch to zero.

To enforce restriction 3, the high resolution pin structures respond only to the first instruction that is executed matching their pin number with `hr_lr=HIGH`, regardless of whether or not the `en_pin_action` field is ON. Subsequent instructions are ignored by the high resolution pin structure for the remainder of the loop resolution period.

23.2.1.6 Multi-Resolution Scheme

The N2HET has the capability to virtually extend the counter width by executing instructions only once every N loop resolution periods. This decreases the timer resolution, but extends the counter range which may be useful when generating or measuring slow signals. Figure 23-5 illustrates how a multi-resolution scheme may be implemented in an N2HET program. An unconditional Branch instruction and an index sequence, using a MOV64 instruction in each low resolution loop, is required to control this particular program flow.

NOTE: HR instructions must be placed in the main (full resolution) loop to ensure proper operation.

Figure 23-5. Multi-Resolution Operation Flow Example


23.2.1.7 Debug Capability

The N2HET supports breakpoints to allow you to more easily debug your N2HET program. [Figure 23-6](#) provides an illustration of the breakpoint mechanism.

The steps to enable an N2HET breakpoint are:

1. Make sure the device nTRST pin is high, since N2HET breakpoints are disabled whenever this pin is low. (Normally this is handled automatically when a JTAG debugger is attached).
2. Attach a JTAG debugger and connect to the device that has been already programmed with the N2HET code that needs to be debugged. (downloading to on-chip flash is outside the scope of this section).
3. Execute the CPU program at least until the point where the N2HET program RAM has been initialized by the CPU.
4. Open a memory window in the N2HET registers.
5. Make sure HETEXC2.DEBUGSTATUSFLAG bit is cleared.
6. Open a memory window on the N2HET RAM
7. Set bit 22 in the program field of the instruction(s) on which you wish to break. Note that this instruction will be executed **before** the N2HET is halted - slightly different from how CPU breakpoints behave.
8. Make sure the CPU and N2HET are running, if they are halted then restart the CPU through the JTAG emulator (N2HET will start when the CPU starts).
9. Both the CPU and N2HET will halt when breakpoint is reached.

When the N2HET is halted, its state machines are frozen but all of the N2HET control registers can be accessed through the JTAG emulator interface.

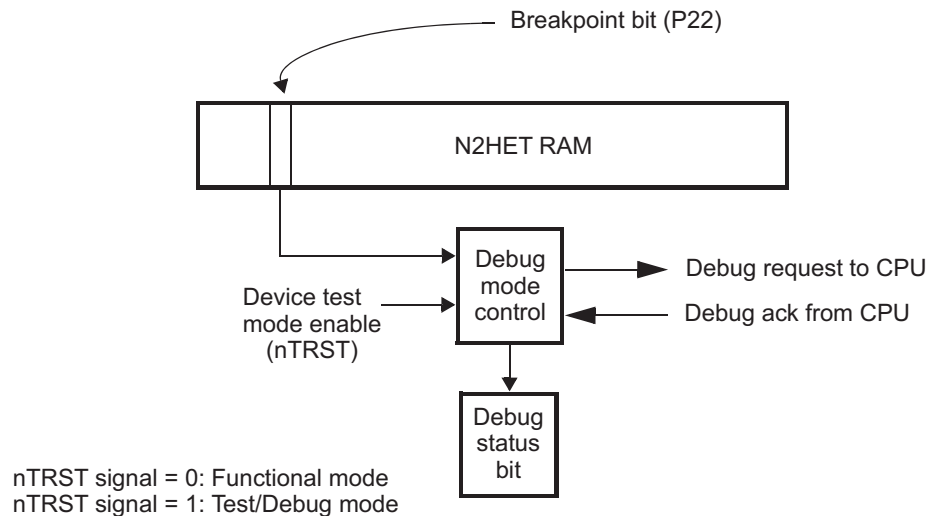
The current N2HET instruction address can be inspected by reading the HETADDR register; this should be pointing to the instruction that caused the breakpoint.

The N2HET internal working registers (A,B,R,S,T) are not directly visible through the JTAG emulator interface. If the content of these registers needs to be inspected, it is best to add an instruction like MOV32 which copies the register value to the N2HET RAM. This RAM location can be inspected when the N2HET halts.

To restart execution of both the CPU and the N2HET from the halted state:

1. Clear HETEXC2.DEBUGSTATUSFLAG bit.
2. Clear bit 22 in the program field of the instruction on which the breakpoint was reached.
3. Restart the CPU through the normal JTAG emulator procedure ('Run' or 'Go'). The N2HET will automatically start executing when it sees that the CPU has exited the debug state.

Figure 23-6. Debug Control Configuration



NOTE: Consecutive break points are not supported. Instructions with break points must have at least a distance of two instructions (for example, at N2HET addresses 1, 3, 5, 7, and so on)

23.2.2 N2HET RAM Organization

The N2HET RAM is organized into two sections. The first contains the N2HET program itself. The second contains parity protection bits for the N2HET program.

Each N2HET instruction is 96-bits wide but aligned to a 128-bit boundary. Instructions consist of three 32-bit fields: Program, Control, and Data. Instructions are separated by a fourth unimplemented address to force alignment to 128-bit boundaries.

The integrity of the N2HET program can be protected by Parity. Parity protection is enabled through the N2HET Parity Control Register (HETPCR).

Table 23-1 shows the base addresses for N2HET RAM and N2HET Parity RAM.

Table 23-1. N2HET RAM Base Addresses

N2HET1 Base Address	N2HET2 Base Address	Memory
0xFF46_0000	0xFF44_0000	N2HET Instruction RAM (Program/Control/Data)
0xFF46_2000	0xFF44_2000	N2HET Parity RAM

23.2.2.1 N2HET RAM Banking

Because the CPU must make updates to the N2HET RAM while the N2HET is executing, for example to update the duty cycle value of a PWM, it is important to understand how the N2HET RAM organization facilitates simultaneous accesses by both the HOST CPU and the N2HET.

The N2HET RAM is implemented as 4 banks of 96-bit wide two port RAM. This means that there a total of 8 ports available; four read and four write. Normally the N2HET will use up to two of these ports at a time. One read port is used to allow the N2HET to prefetch the next N2HET instruction while a write port may be used to update the data or control fields that have changed as a result of executing the current instruction.

N2HET accesses to its own internal RAM are given priority over accesses from an external host (CPU or DMA), this makes N2HET program execution deterministic which is a critical requirement for a timer.

Most N2HET instructions execute in a single cycle. Cases where a wait state impacts the N2HET program execution time are:

- The current N2HET instruction writes data back to the next N2HET in the execution sequence.
- The external host reads from an N2HET instruction where the automatic read-clear option is set, while the N2HET is executing from/on the same address (See [Section 23.2.4.3](#)).

Except for the case of automatic read-clear, the external host is stalled when the host and N2HET have a bank conflict. However this will typically only result in a stall of one cycle, due to the N2HET bank ordering which is organized on the N2HET Address least significant bit boundaries (See [Table 23-2](#)).

Assuming most of the N2HET program executes linearly through the N2HET Address space; if a bank conflict does exist it is usually resolved in the next cycle as the N2HET program moves to the next bank. N2HET programmers should avoid writing a program that accesses the same bank of N2HET RAM on every cycle, as this could lock the external host out of the N2HET memory completely.

[Table 23-2](#) describes the N2HET memory map, as viewed by the N2HET as well as from the memory space of the host CPU and DMA.

Table 23-2. N2HET RAM Bank Structure

N2HET Address	Host CPU or DMA Address Space				N2HET RAM Bank
	Program Field Address	Control Field Address	Data Field Address	Reserved Address	
000h	XX0000h	XX0004h	XX0008h	XX000Ch	A
001h	XX0010h	XX0014h	XX0018h	XX001Ch	B
002h	XX0020h	XX0024h	XX0028h	XX002Ch	C
003h	XX0030h	XX0034h	XX0038h	XX003Ch	D
004h	XX0040h	XX0044h	XX0048h	XX004Ch	A
:	:	:	:	:	:
03Fh	XX03F0h	XX03F4h	XX03F8h	XX03FCh	D
040h	XX0400h	XX0404h	XX0408h	XX040Ch	A
:	:	:	:	:	:
1FFh	XX1FF0h	XX1FF4h	XX1FF8h	XX1FFCh	D

NOTE: The external host interface supports any access size for reads, but only 32-bit writes to the N2HET RAM are supported. Reserved addresses should not be accessed, the result of doing so is indeterminate.

23.2.2.2 Parity Checking

The N2HET module can detect parity errors in N2HET RAM. As described in [Section 23.2.2](#) the N2HET allows 32-bit writes only. Therefore N2HET RAM parity checking is implemented using one parity bit per 32-bit field in N2HET RAM.

Even or odd parity selection for N2HET parity detection can be configured in the system module. Parity calculation and checking can be enabled/disabled by a 4-bit key in HETPCR.

During a read access to the N2HET RAM, the parity is calculated based on the data read from the RAM and compared with the good parity value stored in the parity bits. The parity check is performed when the N2HET execution unit makes a read access to N2HET RAM, but also when a different master (for example, CPU, HTU, DMA) performs the read access. If any 32-bit-word fails the parity check then an error is signaled to the ESM module. The N2HET address, which generated the error is detected and is captured in HETPAR for host system debugging. The address is frozen from being updated until it is read by the bus master.

The N2HET execution unit reads the instructions, which are 96-bit wide. They contain the program-, control- and data-field whereby each is 32-bit wide. So when fetching N2HET instructions parity checking is performed on three words in parallel.

If a parity error is detected in two or more words in the same cycle then only one address (word at the lower address) is captured. The captured N2HET address is always aligned to a 32-bit word boundary.

During debug, parity checking is still performed on accesses originating from the on-chip host CPU and DMA. However, parity errors that are detected during an access initiated by the debugger itself are ignored.

23.2.2.3 Parity Error Detection Actions

Detection of a N2HET parity error causes the following actions:

1. An error is signaled to the ESM module.
2. The Parity Address Register (HETPAR) is loaded with the address of the faulty N2HET field.
3. N2HET execution immediately stops. (The instruction that triggered the parity error is not executed.)
4. The Turn-On/Off-Bit in the N2HET Global Configuration Register (HETGCR) is automatically cleared.
5. All N2HET internal flags are cleared.
6. All N2HET pins selected by N2HET Parity Pin Register (HETPPR) enter a predefined safe state.
7. Register HETDOUT is also updated to reflect changes in pin state due to HETPPR.

The safe state for N2HET pins selected through the HETPPR register depends on how the pin is configured in the HETDIR, HETPDR, and HETPSL registers. [Table 23-3](#) explains how the safe state is determined.

Table 23-3. Pin Safe State Upon Parity Error Detection

Safe State	HETDIR	HETPDR	HETPSL
Drive Low	1	0	0
Drive High	1	0	1
High Impedance	1	1	x

23.2.2.4 Testing Parity Detection Logic

To test the parity detection logic, the parity RAM has to be made accessible to the CPU in order to allow a diagnostic program to insert parity errors. The control register bit HETPCR.TEST must be set in order to make the parity RAM accessible. Once HETPCR.TEST is set, the parity bits are accessible as described in [Table 23-4](#).

Each 32-bit N2HET field has its own parity bit in the N2HET Parity RAM as shown in [Table 23-4](#). There are no parity bits for the reserved fields, since there is no physical N2HET RAM for these fields.

Table 23-4. N2HET Parity Bit Mapping

Address N2HET1	Address N2HET2	Bits	
		[31:1]	[0]
0xFF46_2000	0xFF44_2000	Reads 0, Writes have no effect	Instruction 0 Program Field Parity Bit
0xFF46_2004	0xFF44_2004	Reads 0, Writes have no effect	Instruction 0 Control Field Parity Bit
0xFF46_2008	0xFF44_2008	Reads 0, Writes have no effect	Instruction 0 Data Field Parity Bit
0xFF46_200C	0xFF44_200C	Reads 0, Writes have no effect	Read 0
0xFF46_2010	0xFF44_2010	Reads 0, Writes have no effect	Instruction 1 Program Field Parity Bit
....		...	...

23.2.2.5 Initialization of Parity RAM

After device power up, the N2HET RAM contents including the parity bits cannot be guaranteed. In order to avoid false parity failures due to the random state in which RAM powers up, the RAM has to be initialized.

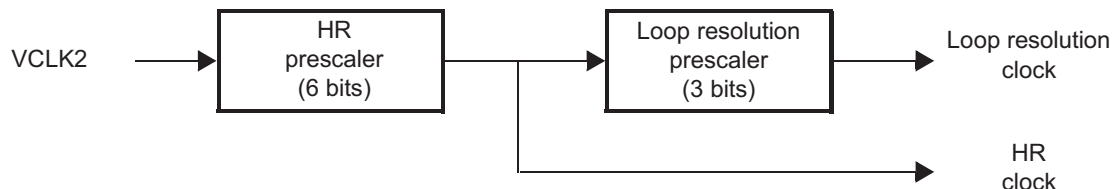
Before initializing the N2HET RAM, enable the N2HET parity logic by writing to HETPCR. Then the N2HET Instruction RAM should be initialized. With parity enabled, the N2HET parity RAM will be initialized automatically by N2HET at the same time that the N2HET instruction RAM is initialized by the CPU. Note that loading the N2HET program with parity enabled is also effective.

Another possibility to initialize the N2HET memory and its parity bits is, to use the system module to start the automatic initialization of all RAMs on the microcontroller. The RAMs will be initialized to '0'. Depending on the even/odd parity selection, the parity bit will be calculated accordingly.

23.2.3 Time Base

All N2HET timings are derived from VCLK2 (see [Figure 23-7](#)). Internally N2HET instructions execute at the VCLK2 rate; but the timer loop clock and the high-resolution hardware timer clock can be scaled down from VCLK2. Two prescalers are available to adjust the timer loop resolution clock for the program loop, and the high resolution (HR) clock for the HR I/O counters.

- **Time Slots:** The number of cycles available for instruction execution per loop. Time Slots is the number of VCLK2 cycles in a Loop Resolution Clock.
- **High Resolution Clock:** The high resolution clock is the smallest time increment with which a pin can change it's state or can be measured in the case of input signals. A 6-bit prescaler dividing VCLK2 by a user-defined HR prescale divide rate (hr) stored in the 6-bit HR prescale factor code (HETPFR). See [Table 23-5](#).
- **Loop Resolution Clock:** The loop resolution clock defines the timebase for executing all instructions in a N2HET program. Since instructions can be conditionally executed, the longest path through the N2HET program must fit into one loop resolution clock period (LRP). A 3-bit prescaler dividing the HR clock by a user-defined loop-resolution prescale divide rate (lr) stored in the 3-bit loop-resolution prescale factor code (HETPFR). See [Table 23-5](#).

Figure 23-7. Prescaler Configuration


The following abbreviations and relations are used in this document:

1. hr: high resolution prescale factor (1, 2, 3, 4,..., 63, 64)
2. lr: loop resolution prescale factor (1, 2, 4, 8, 16, 32, 64,128)
3. ts: Time slots (cycles) available for instruction execution per loop. $ts = hr \times lr$
4. HRP = high resolution clock period $HRP = hr \times T_{VCLK2}$ (ns)
5. LRP = loop resolution clock period $LRP = lr \times HRP$ (ns)

The loop resolution period (LRP) must be selected to be larger than the number of Time slots (VCLK2 cycles) required to complete the worst-case execution path through the N2HET program. Otherwise a program overflow condition may occur (see [Section 23.2.1.4](#)). Because of the relationship of time slots to the hr and lr prescalers as described in item 3 above, increasing either hr or lr increases the number of time slots available for program execution. However, lr would typically be increased first, since increasing hr results in a decrease in timer resolution since it reduces the clock to the High Resolution IO structures.

The divide rates hr and lr can be defined in the HETPFR register. [Table 23-5](#) lists the bit field encodings for the prescale options.

Table 23-5. Prescale Factor Register Encoding

LRPFC - Loop Resolution		HRPFC - High Resolution	
HETPFR[10:8]	Prescale Factor lr	HETPFR[5:0]	Prescale Factor hr
000	/1	000000	/1
001	/2	000001	/2
010	/4	000010	/3
011	/8	000011	/4
100	/16	:	:
101	/32	111101	/62
110	/64	111110	/63
111	/128	111111	/64

23.2.3.1 Determining Loop Resolution

As an example, consider an application that requires high resolution of HRP = 62.5 ns, and loop resolution of LRP = 8 μs, and needs at least 250 time slots for the N2HET application program.

Assuming VCLK2 = 32 MHz, the following shows which divide-by rates and which value in the Prescale Factor Register (HETPFR) is required for the above requirements:

$$hr = 2 \rightarrow HRP = \frac{hr}{VCLK2} = \frac{2}{32MHz} = 62.5ns$$

$$lr = 128 \rightarrow lr \times HRP = 128 \times 62.5ns = 8 \mu s$$

$$ts = hr \times lr = 2 \times 128 = 256$$

$$hr = 2, lr = 128 \rightarrow HETPFR[31:0] = 0x00000701 \quad (33)$$

In the example above, if the loop resolution period needs to decrease from 8 μs to 4 μs, then only 128 time slots will be available for program execution. The program may need to be restructured as suggested in [Section 23.2.1.6](#).

23.2.3.2 The 7-Bit HR Data Field

The instruction execution examples of ECMP ([Section 23.2.5.9](#)), MCMP ([Section 23.2.5.10](#)), PCNT ([Section 23.2.5.12](#)), PWCNT ([Section 23.2.5.11](#)), and WCAP ([Section 23.2.5.13](#)) show that the 7-bit HR data field can generate or measure high resolution delays (HR delay) relative to the start of an LRP within one N2HET loop LRP. The last section showed that:

$$LRP = lr \times HRP$$

There are lr high resolution clock periods (HRP) within the N2HET loop resolution clock period (LRP). If $lr = 128$ then the HR delay can range from 0 to 127 HRP clocks within LRP and all 7 bits of the HR data field are needed. Instead of being limited to measuring and triggering events based on the loop resolution clock period (LRP) the HR extension allows measurements and events to be described in terms fractions of an LRP (down to 1/128 of an LRP). The only limitation is that a maximum of one HR delay can be specified per pin during each loop resolution period.

[Table 23-6](#) shows which bits of the HR data field are not used by the high resolution IO structures if lr is less than 128. In this case the non-relevant bits (LSBs) of the HR data fields will be one of the following:

- Written as 0 for HR capture (for PCNT, WCAP)
- Or interpreted as 0 for HR compare (for ECMP, MCMP, PWCNT)

Table 23-6. Interpretation of the 7-Bit HR Data Field

Loop Resolution Prescale divide rate (lr)	Bits of the HR data field ⁽¹⁾							HRP Cycles delay range
	D[6]	D[5]	D[4]	D[3]	D[2]	D[1]	D[0]	
1	X X X X X X X							0
2	1/2	X X X X X X						0 to 1
4	1/2	1/4	X X X X X					0 to 3
8	1/2	1/4	1/8	X X X X				0 to 7
16	1/2	1/4	1/8	1/16	X X X			0 to 15
32	1/2	1/4	1/8	1/16	1/32	X X		0 to 31
64	1/2	1/4	1/8	1/16	1/32	1/64	X	0 to 63
128	1/2	1/4	1/8	1/16	1/32	1/64	1/128	0 to 127

⁽¹⁾ X = Non-relevant bit (treated as '0')

23.2.3.2.1 Example:

Prescale Factor Register (HETPFR) = 0x0300

—> $l_r = 8$ —> $LRP = 8 \times HRP$

Assumption: HR data field = 0x50 = 1010000b

$l_r = 8$ —> Bits D[3:0] are ignored —> HR delay = 101b = 5 HRPs

or by using the calculation with weight factors:

HR Delay

$= l_r \cdot (D[6] \cdot 1/2 + D[5] \cdot 1/4 + D[4] \cdot 1/8 + D[3] \cdot 1/16 + D[2] \cdot 1/32 + D[1] \cdot 1/64 + D[0] \cdot 1/128)$

$= 8 \cdot (1 \cdot 1/2 + 0 \cdot 1/4 + 1 \cdot 1/8 + 0 \cdot 1/16 + 0 \cdot 1/32 + 0 \cdot 1/64 + 0 \cdot 1/128)$

= 5 HRPs

23.2.4 Host Interface

The host interface controls all communications between timer-RAM and masters accessing the N2HET RAM. It includes following components:

23.2.4.1 Host Accesses to N2HET RAM

The host interface supports the following types of accesses to N2HET RAM:

- Read accesses of 8, 16, or 32 bits
- Read accesses of 64-bits that follow the shadow register sequence described in [Section 23.2.4.2](#).
- Write accesses of 32 bits

Writes of 8 or 16 bits to N2HET RAM by an external host are not supported.

23.2.4.2 64-bit Read Access

The consecutive read of a control field CF(n) and a data field DF(n) of the same instruction (n) performed by the same master (for example, CPU, DMA, or any other master) is always done as a simultaneous 64-bit read access. This means that at the same time CF(n) is read, DF(n) is loaded in a shadow register. So the second access will read DF(n) from the shadow register instead of the N2HET RAM.

In general a 64-bit read access of one master could be interrupted by a 64-bit read access of another master. A total of three shadow registers are available. Therefore up to three masters can perform 64-bit reads in an interleaved manner (Master1 CF, Master2 CF, Master3 CF, Master1 DF, Master2 DF, Master3 DF).

If all three shadow registers are activated and a 4th master performs a CF or DF read it will result in an address error and the RAM access will not happen. Other access types by a fourth master (reads from the PF field or writes to any of the fields) will occur because these access types do not require an available shadow register resource to complete.

23.2.4.3 Automatic Read Clear Feature

The N2HET provides a feature allowing to automatically clear the data field immediately after the data field is read by the external host CPU (or DMA). This feature is implemented via the *control bit*, which is located in the control field (bit C26). This is a static bit that can be used by any instruction, and specified in the N2HET program by adding the option (control = ON) to the N2HET instruction. The automatic read clear feature works for both 32 and 64 bit reads that follow the sequence described in [Section 23.2.4.2](#).

When the host CPU reads the data field of that instruction, the current data value is returned to the host CPU but the field is cleared automatically as a side effect of the read. In case the master reads data from an instruction currently executing, any new capture result is stored and this takes priority over the automatic read clear feature, so that the new capture result is not lost.

As an example of where the automatic read clear feature is useful, consider the PCNT instruction. If this instruction is configured for automatic read clear, then when the host CPU reads the PCNT data field it will be cleared automatically. The host CPU can then poll the PCNT data field again, and as long as the field returns a value of zero the host CPU program knows a new capture event has not occurred. If the data field were not cleared, it would be impossible for the host CPU to determine whether the data field holds data from the previous capture event, or if it happens to be data from a new capture event with the same value.

23.2.4.4 Emulation Mode

Emulation mode, used by the software debugger, is specified in the global configuration register. When the host CPU debugger hits a breakpoint, the CPU sends a suspend signal to the modules. Two modes of operation are provided: suspend and ignore suspend.

- Suspend

When a suspend is issued, the timer operation stops at the end of the current timer instruction. However, the CPU accesses to the timer RAM or control registers are freely executed.

- Ignore suspend

The timer RAM ignores the suspend signal and operates real time as normal.

23.2.4.5 Power-Down

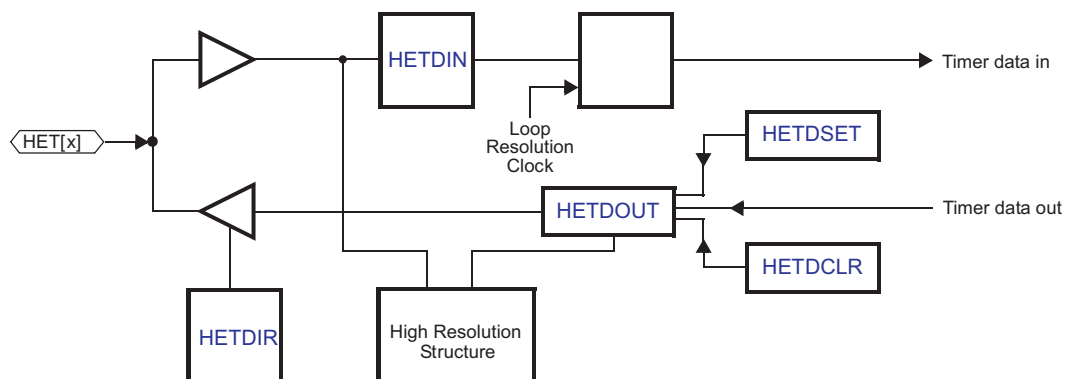
After setting the turn-off bit in the Global Configuration Register (HETGCR), it is required to delay until the end of the timer program loop before putting the N2HET in power-down mode. This can be done by waiting until the N2HET Current Address (HETADDR) becomes zero, before disabling the N2HET clock source in the device's Global Clock Module (GCM).

23.2.5 I/O Control

The N2HET has up to 32 pins. Refer to device specific data sheets for information concerning the number of N2HETIO available. All of the N2HET pins available are programmable as either inputs or outputs.

These 32 I/Os have an identical structure connected to pins HET[31] to HET[0]. See [Figure 23-8](#) for an illustration of the I/O control. In addition all 32 I/Os have a special HR structure based on the HR clock. This structure allows any N2HET instruction to use any of these I/Os with an accuracy of either loop resolution or high resolution accuracy.

Figure 23-8. I/O Control



Pins N2HET [31] to N2HET [0] can be used by the CPU as general-purpose inputs or outputs using the N2HET Data Input Register (HETDIN) for reading and N2HET Data Output Register (HETDOUT), N2HET Data Set Register (HETDSET) or N2HET Data Clear Register (HETDCLR) for writing, depending on the type of action to perform. The N2HET pins used as general-purpose inputs are sampled on each VCLK2 period.

23.2.5.1 Using General-Purpose I/O Data Set and Clear Registers

The N2HET Data Clear Register (HETDCLR) and N2HET Data Set Register (HETDSET) can be used to minimize the number of accesses to the peripheral to modify the output register and output pins. When the application needs to set or to reset some N2HET pins without changing the value of the others pins, the first possibility is to read N2HET Data Output Register (HETDOUT), modify the content (AND, OR, and so on), and write the result into N2HET Data Output Register (HETDOUT). However, this read-modify-write sequence could be interrupted by a different function modifying the same register which will result in a data coherency problem.

Using the N2HET Data Set Register (HETDSET) or N2HET Data Clear Register (HETDCLR), the application program must write the mask value (same mask value for the first option) to the register to set or reset the desired pins. Any bits written as 0 to HETDSET and HETDCLR are left unchanged, which avoids the possible coherency problem of the read-modify-write approach.

Coding Example (C program): Set pins using the 2 methods.

```
unsigned int MASK;                                /* Variable that content the bit mask */
volatile unsigned int *HETDOUT,*HETDSET;          /* Pointer to HET registers */
...
*HETDOUT = *HETDOUT | MASK;                      /* Read-modify-write of HETDOUT */
*HETDSET = MASK;                                  /* Set the pin without reading HETDOUT */
```

23.2.5.2 Loop Resolution Structure

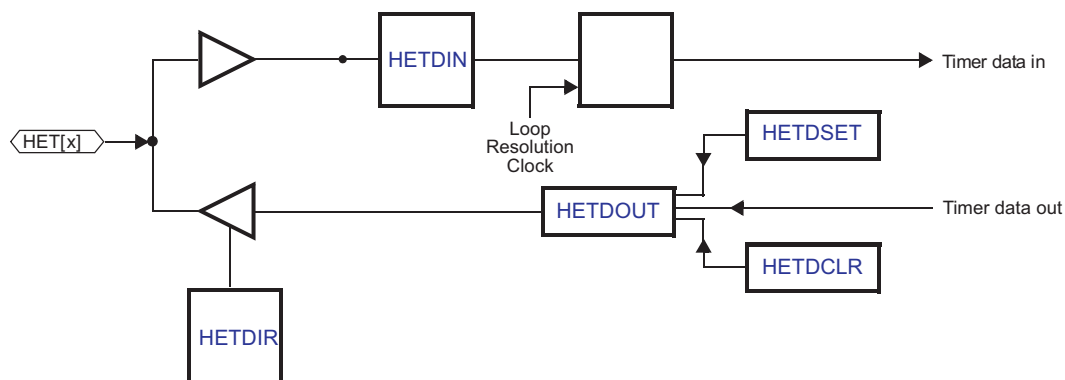
The N2HET uses the pins N2HET [31:0] as input and/or output by the way of the instruction set. Actually, each pin could monitor the N2HET program or could be monitored by the N2HET program. By using the I/O register of the N2HET, the CPU is able to interact with the N2HET program flow.

When an action (set or reset) is taken on a pin by the N2HET program, the N2HET will modify the pin at the rising edge of the next loop resolution clock.

When an event occurs on a N2HET I/O pin, it is taken into account at the next rising edge of the loop resolution clock.

The structure of each pin is shown in [Figure 23-9](#).

Figure 23-9. N2HET Loop Resolution Structure for Each Bit



The example in [Figure 23-10](#) shows a simple PWM generation with loop resolution accuracy. The corresponding program is:

HETPFR[31:0] register = 0x201 --> lr=4 and hr=2 --> ts = 8

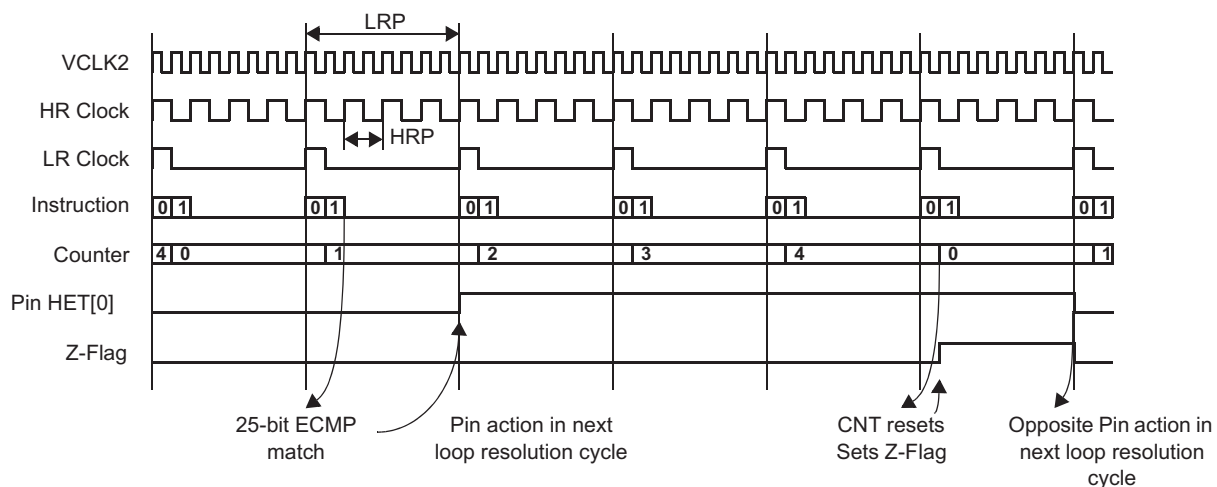
N2HET Program:

```
L00 CNT { next= L01, reg=A, irq=OFF, max = 4 }
L01 ECMP { next= L00, cond_addr= L00, hr_lr=LOW, en_pin_action=ON, pin=0,
          action=PULSEHI, reg=A, irq=OFF, data= 1, hr_data = 0x0 }
```

; 25 bit compare value is 1 and the 7-bit HR compare value is 0

The CNT and ECMP instructions are executed once each loop resolution cycle. When the CNT instruction is executed, the specified register (A) and the CNT instruction data field are both incremented by one. Next the ECMP is executed and the data field of the ECMP is compared with the specified register (A). If both values match, then the pin action (PULSEHI in this case) will be performed in the next loop resolution cycle. The CNT continues incrementing each loop resolution cycle. When the data field overflows (max + 1), then the Z-flag is set by the CNT instruction. In the next loop resolution cycle, the Z-flag is evaluated and the opposite pin action is performed if it is set. The Z-flag will only be active for one loop resolution cycle.

Figure 23-10. Loop Resolution Instruction Execution Example



23.2.5.3 High Resolution Structure

All 32 I/Os provide the HR structure based on the HR clock. The HR clock frequency is programmed through the Prescale Factor Register (HETPFR). In addition to the standard I/O structure, all pins have HR hardware so that these pins can be used as HR input captures (using the HR instructions PCNT or WCAP) or HR output compares (using the HR instructions ECMP, MCMP, or PWCNT).

All five HR instructions (PCNT, WCAP, ECMP, MCMP, and PWCNT) have a dedicated `hr_lr` bit (high resolution/low resolution; program field bit 8) allowing operation either in HR mode or in standard resolution mode by ignoring the HR field. By default, the `hr_lr` bit value is 0 which implies HR operation mode. However, setting this bit to one allows the use of several HR instructions on a single HR pin. Only one instruction is allowed to operate in HR mode (bit cleared to 0), but the other instructions can be used in standard resolution mode (bit set to 1).

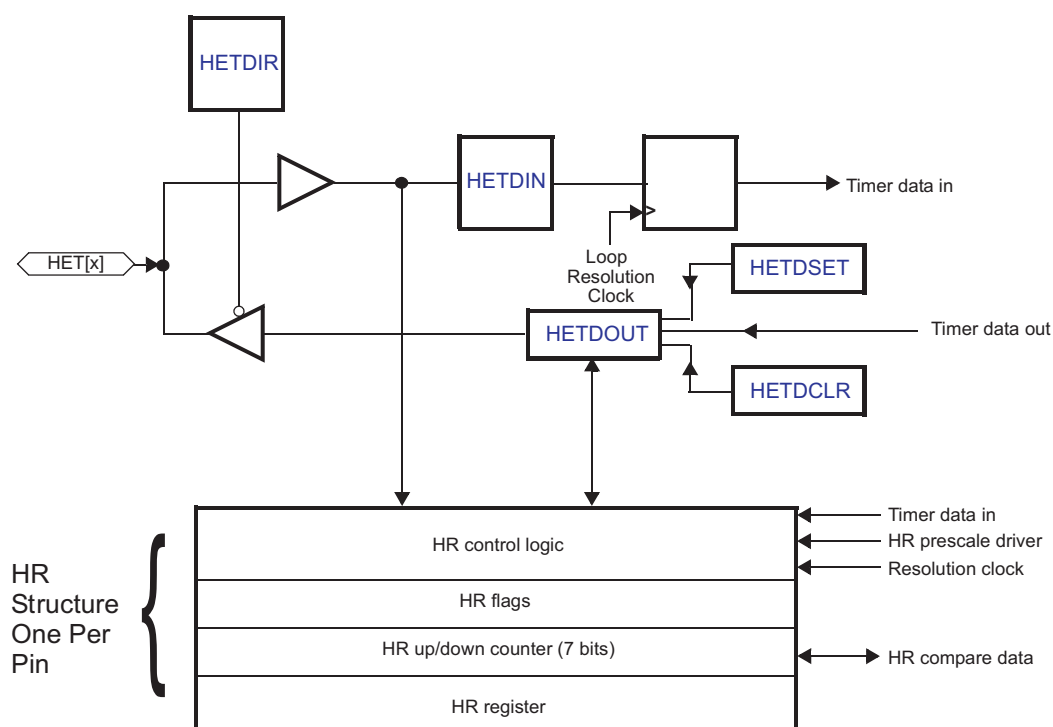
23.2.5.4 HR Block Diagram

Each time an HR instruction is executed on a given pin, the HR structure for that pin is programmed and synchronized to the next loop-resolution cycle (which HR function to perform and on which edges it should take an action) with the information given by the instruction. The HR structure for each pin decodes the pin select field of the instruction and programs its HR structure if it matches.

NOTE: For each N2HET pin, only one instruction specifying a high resolution operation (hr_lr = HIGH) is allowed to execute per loop resolution period. This includes any instructions where (hr_lr = HIGH) but (en_pin_action = OFF).

The first high resolution instruction that executes and specifies a particular pin locks out subsequent high resolution instructions from operating on the same pin until the end of the current loop resolution period.

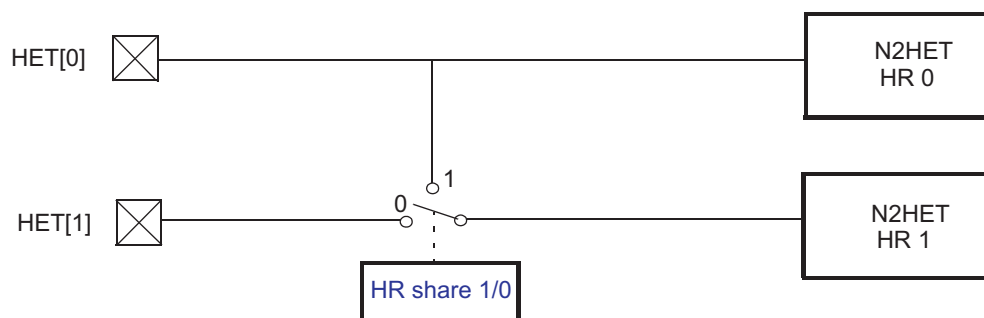
Figure 23-11. HR I/O Architecture



23.2.5.5 HR Structures Sharing (Input)

The HR Share Control Register (HETHRSH) allows two HR structures to share the same pin **for input capture only**. If these bits are set, the HR structures N and N+1 are connected to pin N. In this structure, pin N+1 remains available for general-purpose input/output. See [Figure 23-12](#).

Figure 23-12. Example of HR Structure Sharing for N2HET Pins 0/1



The following program gives an example how the HR share feature (HET[0] HR structure and HET[1] HR structure shared) can be used for the PCNT instruction:

```
L00 PCNT { next=L01, type=rise2fall, pin=0 }
L01 PCNT { next=L00, type=fall2rise, pin=1 }
```

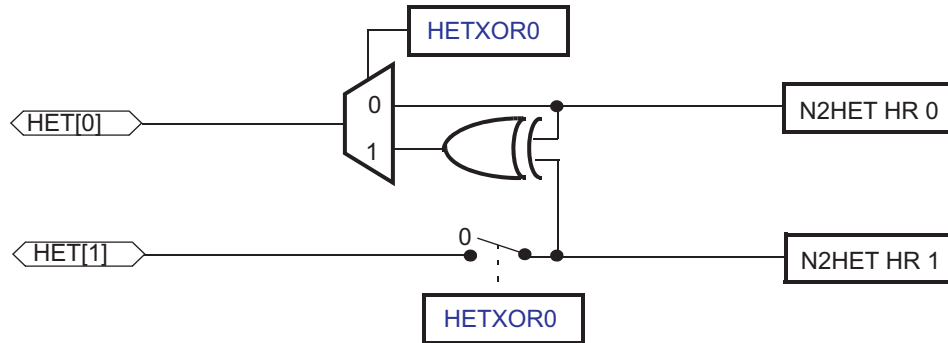
The HET[1] HR structure is also connected to the HET[0] pin. The L00_PCNT data field is able to capture a high pulse and the L01_PCNT captures a low pulse on the **same** pin (N2HET [0] pin).

23.2.5.6 AND / XOR-shared HR Structure (Output)

Usually the N2HET design allows only one HR structure to generate HR edges on a pin configured as output pin. The HETXOR register allows a logical XOR of the output signals of two consecutive HR structures N (even) and N+1 (odd). See [Figure 23-13](#). In this way, it is possible to generate pulses smaller than the loop resolution clock since both edges can be generated by two independent HR structures. This is especially required for symmetrical PWM. See [Figure 23-14](#).

The hardware provides a XOR gate that is connected to the outputs of the HR structure of two consecutive pins. In this structure, pin N+1 remains available for general-purpose input/output.

Figure 23-13. XOR-shared HR I/O



The following N2HET program gives an example for **one** channel of the symmetrical PWM. The generated timing is given in [Figure 23-14](#).

```
MAXC .equ 22
A_ .equ 0 ; HR structure HR0
B_ .equ 1 ; HR structure HR1

CN CNT { next=EA, reg=A, max=MAXC }

EA ECMP { next=EB, cond_addr=MA, hr_lr=HIGH, en_pin_action=ON, pin=A_,
          action=PULSELO, reg=A, data=17, hr_data=115 }

MA MOV32 { next=EB, remote=EA, type=IMTOREG&REM, reg=NONE, data=17, hr_data=19 }

EB ECMP { next=CN, cond_addr=MB, hr_lr=HIGH, en_pin_action=ON, pin=B_,
          action=PULSELO, reg=A, data=5, hr_data=13 }

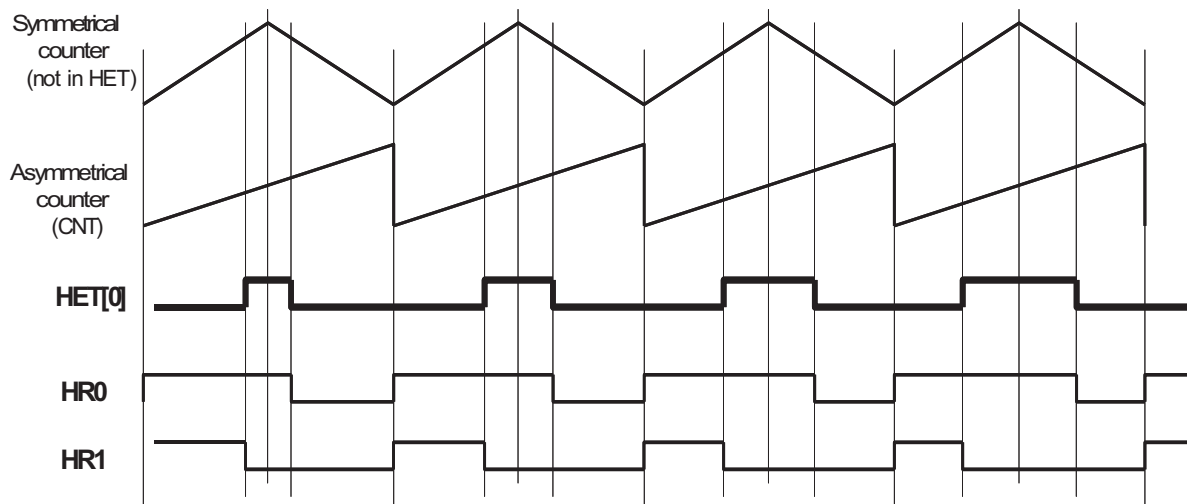
MB MOV32 { next=CN, remote=EB, type=IMTOREG&REM, reg=NONE, data=5, hr_data=13 }
```

N2HET Settings and output signal calculation for this example program:

- Pin HET[0] and HET[1] are XOR-shared.
- HETPFR[31:0] register = 0x700: lr=128, hr=1, time slots ts = 128
- PWM period (determined by CNT_max field) = (22+1) · LRP = 2944 HRP
- Length of high pulse of (HET[0] XOR HET[1]) =
 $LH = (17 \cdot LRP + 115 \cdot HRP) - (5 \cdot LRP + 13 \cdot HRP)$
 With lr=128 there is LRP = 128 · HRP, so
 $LH = (2291 - 653) \cdot HRP = 1638 \text{ HRP}$
- Duty cycle = DC = LH / PWM_period = 1638 HRP / (2944 · HRP) = 55.6 %

Figure 23-14 graphically shows the implementation of the XOR-shared feature. The first 2 waveforms (symmetrical counter and CNT) show a symmetric counter and asymmetric counter. The symmetric counter is shown only to highlight the axis of symmetry and is not implemented in the N2HET. The asymmetric counter, which is implemented with a CNT instruction, needs to be set to the period of the symmetric counter. The next two waveforms (HR [0] and HR [1]) show the output of the HR structures, which are the inputs for the XOR gate to create the PWM output on pin HET[0]. Notice that the pulses of signal HET[0] are centered about the axis of symmetry.

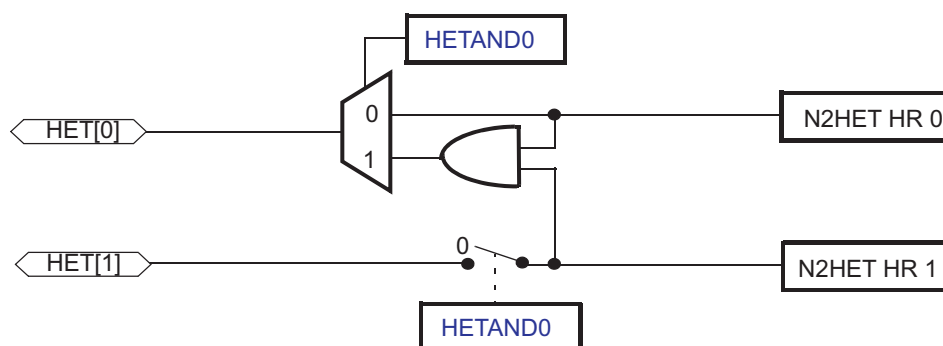
Figure 23-14. Symmetrical PWM with XOR-sharing Output



As an alternative, HR structures may be shared using a logical AND function to combine the effects of the pin structures. The HETAND allows sharing two consecutive HR structures N (even) and N+1 (odd). See Figure 23-15. In this structure, pin N+1 remains available for general-purpose input/output.

NOTE: Setting both the HETAND bit and HETXOR bits at the same time for a given pair of N2HET pins is not supported, must be avoided by the application program.

Figure 23-15. AND-shared HR I/O



23.2.5.7 Loop Back Mode

The loop back feature can be used by the application to monitor an N2HET output signal. For example, if a PWM is generated by HR structure 0, then a PCNT instruction assigned to HR structure 1 can measure back the pulse length or periods of the PWM output signal.

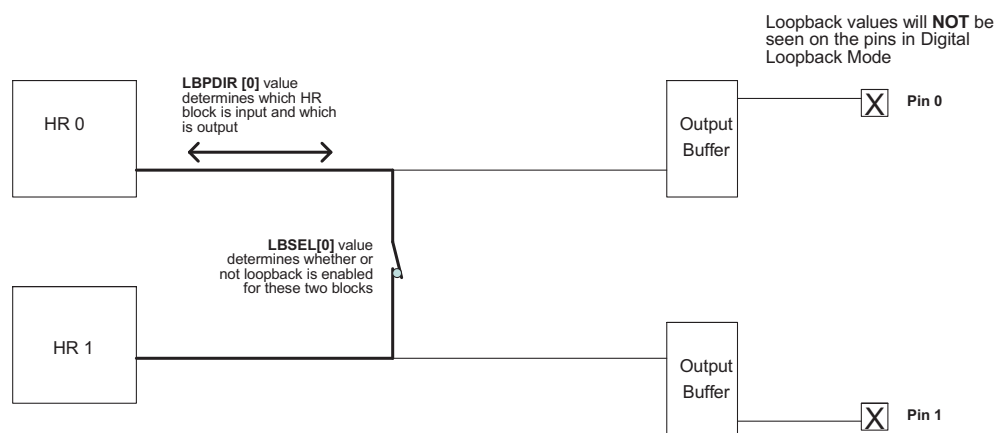
Loopback mode is activated between two high resolution structures by setting LBPSEL[x] to 1 in the HETLBPSEL register for the corresponding structure pair. The **direction** of the loopback between the two structures in the structure pair is determined by the value of LBPDIR[x] in the HETLBPDIR Register.

For example, if bit LBPSEL[0] is set to 1, then HR structures 0 and 1 will be internally connected in loop back mode. If bit LBPDIR[0] is set to 0, then structure 0 will be the input and structure 1 will be the output.

Digital Loopback

Digital loopback mode is enabled by setting LBPTYPE[x] to 0 in the HETLBPSEL register for the corresponding structure pairs. In digital loopback mode, the structure pairs are connected directly and the output buffers are bypassed. Therefore, the loopback values will NOT be seen on the corresponding pins. Figure 23-16 shows an example of digital loopback between structures HR0 and HR1. LBSEL[0] has been set to 1 to enable loopback between the two structures. LBTYPE[0] has been set to 0 to select digital mode for the loopback pair. The LBPDIR[0] value will determine the direction of the loopback by selecting which of the HR blocks is output, and which is input. The bold lines show the digital loopback path.

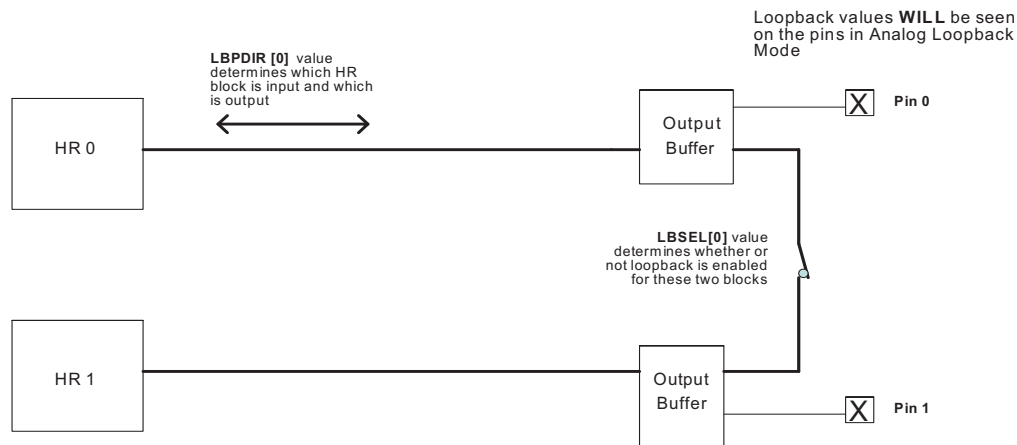
Figure 23-16. HR0 to HR1 Digital Loopback Logic: LBTYPE[0] = 0



Analog Loopback

Analog loopback mode is enabled by setting LBPTYPE[x] to 1 in the HETLBPSEL register for the corresponding structure pairs. In analog loopback mode, the structure pairs are connected outside of the output buffers. Therefore, the loopback values **WILL** be seen on the corresponding pins. [Figure 23-17](#) shows an example of analog loopback between structures HR0 and HR1. LBSEL[0] has been set to 1 to enable loopback between the two structures. LBTYPE[0] has been set to 1 to select analog mode for the loopback pair. The LPBDIR[0] value will determine the direction of the loopback by selecting which of the HR blocks is output, and which is input. The bold lines show the analog loopback path.

Figure 23-17. HR0 to HR1 Analog Loop Back Logic: LBTYPE[0] = 1



Note:

- The loop back direction can be selected independent of the HETDIR register setting.
- The pin that is not driven by the N2HET output pin actions can still be used as normal GPIO pin.

23.2.5.8 Edge Detection Input Timing

There are several timing requirements for input signals in order to be captured correctly by N2HET. Figure 23-18 illustrates these requirements, with min and max values described in Table 23-7 (Loop Resolution) and Table 23-8 (High Resolution).

Figure 23-18. N2HET Input Edge Detection

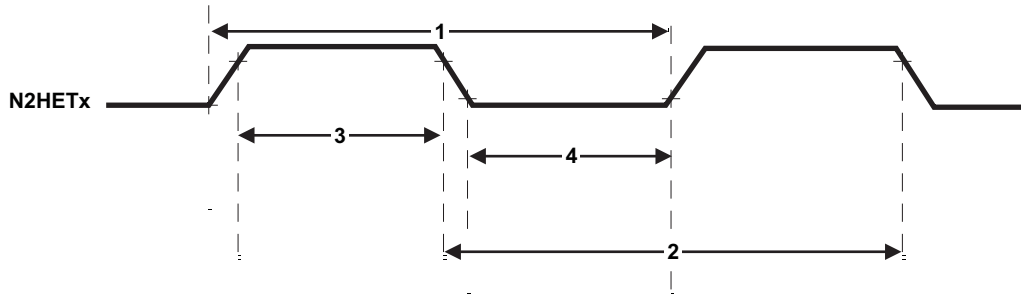


Table 23-7. Edge Detection Input Timing for Loop Resolution Instructions

Parameter #	Description	min	max
1	Input Signal Period, rising edge to rising edge	> 2 (hr) (lr) t _{c(VCLK2)}	< 2 ²⁵ (hr) (lr) t _{c(VCLK2)}
2	Input Signal Period, falling edge to falling edge		
3	Input Signal, high phase	> (hr) (lr) t _{c(VCLK2)}	
4	Input Signal, high phase		

Table 23-8. Edge Detection Input Timing for High Resolution Instructions

Parameter #	Description	min	max
1	Input Signal Period, rising edge to rising edge	> (hr) (lr) t _{c(VCLK2)}	< 2 ²⁵ (hr) (lr) t _{c(VCLK2)}
2	Input Signal Period, falling edge to falling edge		
3	Input Signal, high phase	> 2 (hr) t _{c(VCLK2)}	
4	Input Signal, high phase		

These are the N2HET architectural limitations. Actual limitations will be slightly different due to on chip routing and IO buffer delays, usually by several nanoseconds. Be sure to consult the device datasheet for actual timings that apply to that device. Also, certain devices place additional restrictions on which pins support the high resolution timings of Table 23-8, if present these additional limitations will also be called out in the device datasheet.

Note that the max limit in Table 23-7 and Table 23-8 is based on the counter range of a single N2HET instruction. The max value could be extended by employing an additional N2HET instruction to keep track of counter overflows of the input counter / capture instruction.

23.2.5.9 PWM Generation Example 1 (in HR Mode)

The following example shows how an ECMP instruction works in high resolution mode. The example assumes a VCLK2 of 32 MHz and the following values for the prescale divide rates (hr and lr), number of time slots (ts), high and loop resolution period (HRP and LRP):

$$\text{hr} = 2, \text{lr} = 4, \text{ts} = \text{hr} \times \text{lr} = 8$$

$$\text{HRP} = \text{hr} / \text{VCLK2} = 2 / 32 \text{ MHz} = 62.5 \text{ ns}$$

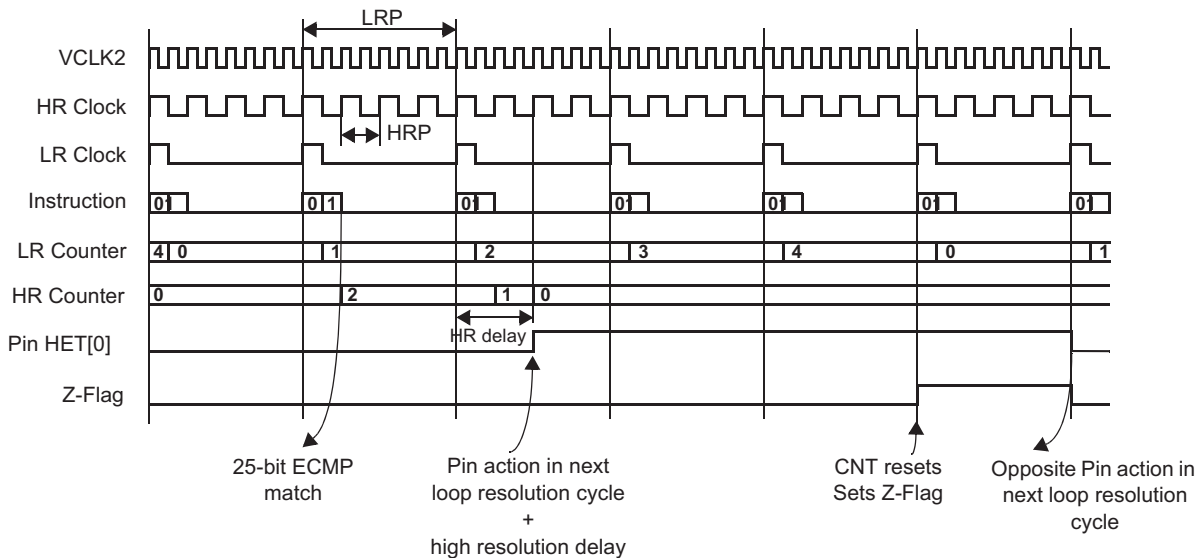
$$\text{LRP} = (\text{hr} \times \text{lr}) / \text{VCLK2} = 8 / 32 \text{ MHz} = 250 \text{ ns}$$

With $\text{ts} = 8$, there are eight time slots available for the program execution, which in this case will consist of one CNT and one ECMP instruction as shown below. The data field of the ECMP instruction is the 32-bit compare value, whereby the lower 7 bits represent the high resolution compare field.

When the 25-bit (loop resolution) compare matches, the HR compare value will be loaded from the 7 lower bits of the instruction data field to the HR counter. At the next loop resolution clock, the HR counter will count down at the HR clock frequency and perform the pin action when it reaches zero.

In the example illustrated by Figure 23-19, the 25-bit compare value is 1 and the 7-bit HR compare value is 2. According to Section 23.2.3.2, depending on the loop resolution divide rate (lr), only certain bits of the 7-bit HR compare value are valid. In this example only the upper 2 bits (D[6:5]) are taken into account. The example program below has a setting of `hr_data = 100000b`. Shifting this value right by 5 bits, results in 10b which equals the two HR clock cycles delay mentioned above.

Figure 23-19. ECMP Execution Timings



HETPFR[31:0] register = 0x201 --> lr=4 and hr=2 --> ts = 8

N2HET Program:

```
L00  CNT  { next= L01, reg=A, irq=OFF, max = 4 }
L01  ECMP { next= L00, cond_addr= L00, hr_lr=HIGH, en_pin_action=ON, pin=0,
          action=PULSEHI, reg=A, irq=OFF, data= 1, hr_data = 0x40 }
```

; 25 bit compare value is 1 and the 7-bit HR compare value is 2
; (Because of lr=4 the D[4:0] of the 7-bit HR field are ignored)

NOTE: ECMP Opposite Actions

ECMP opposite pin actions are always synchronized to the loop resolution clock.

Changing the duty cycle of a PWM generated by an ECMP instruction, can lead to a missing pulse if the data field of the instruction is updated directly. This can happen when it is changed from a high value to a lower value while the CNT instruction has already passed the new updated lower value. To avoid this a synchronous duty cycle update can be performed with the use of an additional instruction (MOV32). This instruction is only executed when the compare of the ECMP matches. For this the `cond_addr` of the ECMP needs to point to the MOV32. On execution of the MOV32, it moves its data field into the data field of the ECMP. The update of the duty cycle has to be made to the MOV32 data field instead of the ECMP data field.

23.2.5.10 PWM Generation Example 2 (in HR Mode)

The MCMP instruction can also be used in HR mode. In this case operation is exactly the same as for the ECMP instruction except that the 25-bit low resolution is now the result of a magnitude compare (greater or equal) rather than an equality compare. When the 25-bit (loop resolution) magnitude compare matches, the HR compare value will be loaded from the 7 lower bits of the instruction data field to the HR counter. At the next loop resolution clock, the HR counter will count down at the HR clock frequency and perform the pin action when it reaches zero.

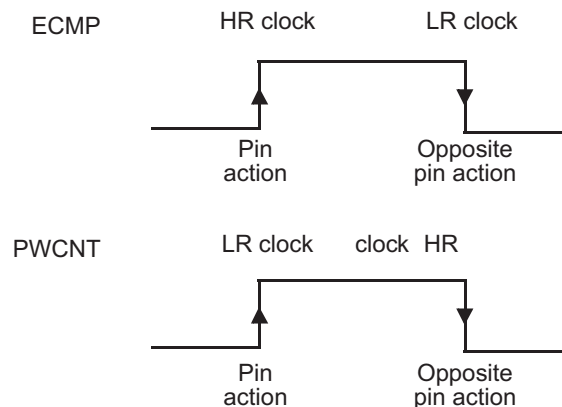
The MCMP instruction avoids the missing pulse problem of the ECMP instruction (see previous example), however the duty cycle of the signal might not be exact for one PWM period. The benefit of the MCMP is that it avoids adding another instruction to do the duty cycle update synchronously.

23.2.5.11 Pulse Generation Example (in HR Mode)

The PWCNT instruction may also be used in HR mode to generate pulse outputs with HR width. It generates a single pulse when the data field of the instruction is non-zero. It remains at the opposite pin action when the data field is zero.

The PWCNT instruction operates conversely to the ECMP instruction. See [Figure 23-20](#). For PWCNT, the opposite pin action is synchronous with the HR clock and for ECMP the pin action is synchronous with the HR clock. The PWCNT pin action is synchronous with the loop resolution clock.

Figure 23-20. High/Low Resolution Modes for ECMP and PWCNT



23.2.5.12 Pulse Measurement Example (in HR Mode)

The PCNT instruction captures HR measurement of the high/low pulse time or periods of the input. As shown in [Figure 23-21](#), at marker (1) the input goes HIGH and the HR counter immediately begins to count. The counter increments and rolls over until the falling edge on the input pin, where it captures the counter value into the HR capture register (marker (2)). The PCNT instruction begins counting when the synchronized input signal goes HIGH and captures both the 25-bit data field and the HR capture register into RAM when the synchronized input falls (marker (3)).

NOTE: The HR capture value written into RAM is shifted appropriately depending on the loop resolution prescale divide rate (lr). (See also [Section 23.2.3.2](#)).

Figure 23-21 shows what happens when the capture edge arrives *after* the HR counter overflows. This causes the incremented value to be captured by the PCNT instruction.

Figure 23-21. PCNT Instruction Timing (With Capture Edge After HR Counter Overflow)

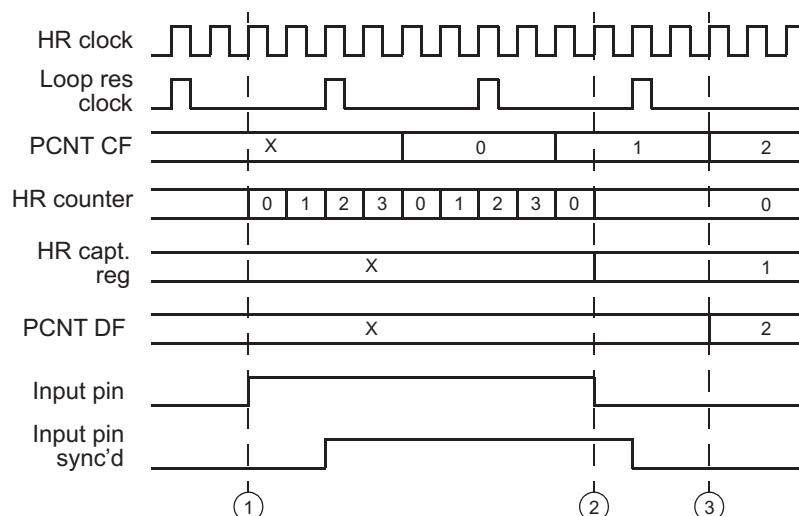
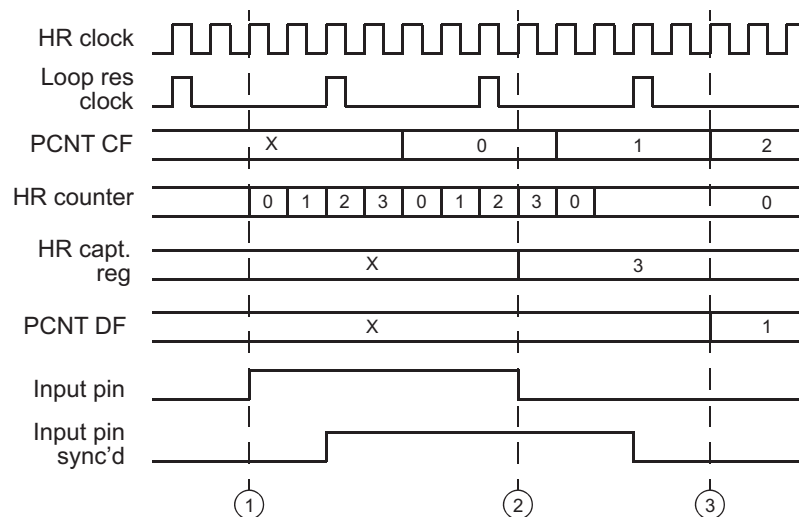


Figure 23-22 shows what happens when the capture edge arrives *before* the HR counter overflows. This causes the non-incremented value to be captured by the PCNT instruction.

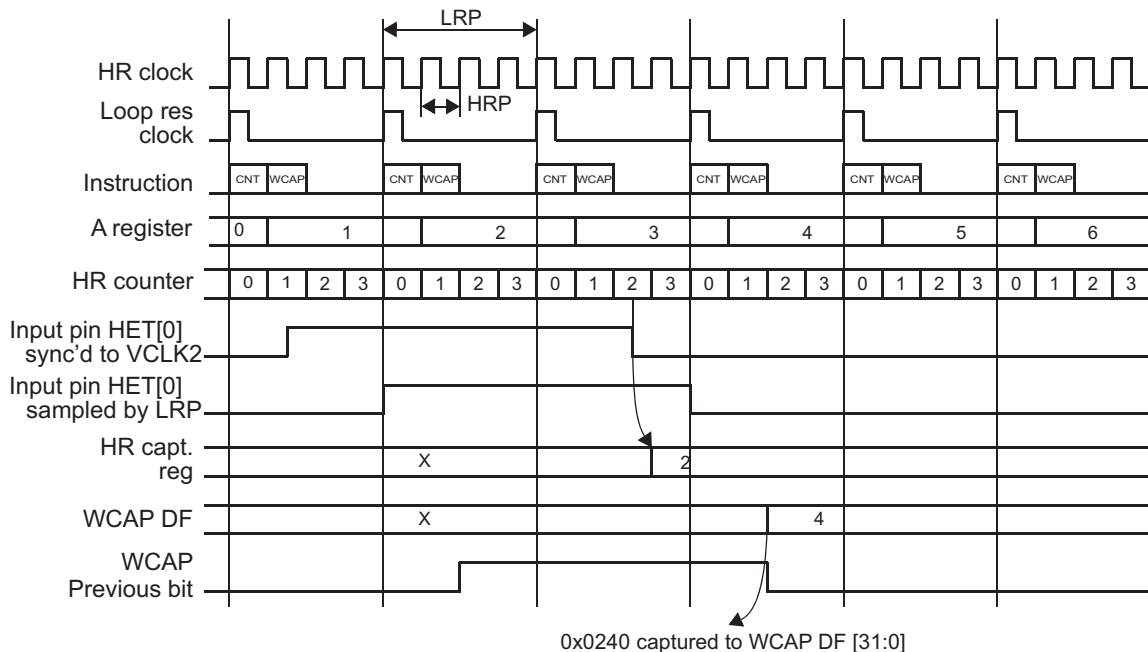
Figure 23-22. PCNT Instruction Timing (With Capture Edge Before HR Counter Overflow)



23.2.5.13 WCAP Execution Example (in HR Mode)

The HR capability is enabled for WCAP, if its `hr_lr` bit is zero. In this case the HR counter is always enabled and is synchronized with the resolution loop. When the specified edge is detected, the current value of the HR counter is captured in the HR capture register and written into the RAM after the next WCAP execution. The WCAP instruction effectively time stamps the free running timer saved in a register (for example, register A shown in Figure 23-23).

Figure 23-23. WCAP Instruction Timing



```
HETPFR_register = 0x0200 --> lr = 4, hr = 1, ts = 4
```

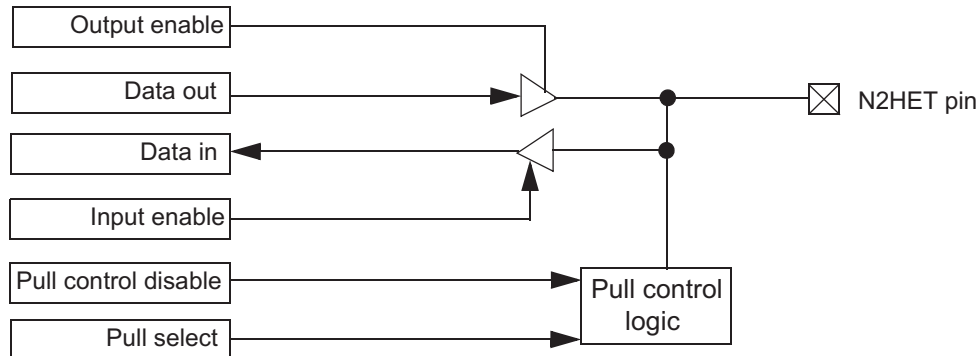
N2HET Program:

```
L00 CNT {reg=A, max=01ffffffh}
L01 WCAP {next=L00, cond_addr=L00, hr_lr=high, reg=A, event= FALL, pin=0,
         data=0}
```

In the example, the WCAP is configured to capture the counter when a **falling** edge occurs. The WCAP data field (WCAP_DF) is updated in the loop succeeding the loop in which the edge occurred. The WCAP instruction evaluates an edge by comparing its Previous bit with the sync'd input signal. In Figure 23-23, the current value of the counter (4) is captured to WCAP_DF[31:7] and the value of the HR capture register (2) is transferred to the valid bits (according the `lr` prescaler) of WCAP_DF[6:0]. Therefore, in the example 0x0240 is captured in WCAP_DF[31:0].

23.2.5.14 I/O Pull Control Feature

Figure 23-24. I/O Block Diagram Including Pull Control Logic



The following apply if the device is under reset:

- Pull control: The reset pull control on the pins is enabled and a pulldown is configured.
- Input buffer: The input buffer is enabled.
- Output buffer: The output buffer is disabled.

The following apply if the device is out of reset:

- Pull control: The pull control is enabled by clearing the corresponding bit in the N2HET Pull Disable Register (HETPULDIS). In this case, if the corresponding bit in the N2HET Pull Select Register (HETPSL) is set, the pin will have a pull-up; if the bit in the N2HET Pull Select Register (HETPSL) is cleared, the pin will have a pull-down. If the bit in the N2HET Pull Disable Register (HETPULDIS) is set, there is no pull-up or pull-down on the pin.
- Input buffer: The input buffer is disabled only if the pin direction is set to input AND the pull control is disabled AND pull down is selected as the pull bias. In all other cases, the input buffer is enabled.

NOTE: The pull-disable logic depends on the pin direction. If the pin is configured as output, then the pulls are disabled automatically. If the pin is configured as input, the pulls are enabled or disabled depending on the pull disable register bit.

- Output buffer: A pin can be driven as an output pin if the corresponding bit in the N2HET Direction Register (HETDIR) is set AND the open-drain feature (N2HET Open Drain Register (HETPDR)) is not enabled. See [Section 23.2.5.15](#) for more details.

The behavior of the input buffer, output buffer, and the pull control is summarized in [Table 23-9](#). When an input buffer is disabled, it appears as a logic low to on-chip logic.

Table 23-9. Input Buffer, Output Buffer, and Pull Control Behavior

Device under Reset?	Pin Direction (DIR) ⁽¹⁾	Pull Disable (PULDIS) ⁽¹⁾	Pull Select (PULSEL) ⁽¹⁾	Pull Control	Output Buffer	Input Buffer
Yes	X	X	X	Enabled	Disabled	Enabled
No	0	0	0	Pull down	Disabled	Enabled
No	0	0	1	Pull up	Disabled	Enabled
No	0	1	0	Disabled	Disabled	Disabled
No	0	1	1	Disabled	Disabled	Enabled
No	1	X	X	Disabled	Enabled	Enabled

⁽¹⁾ X = Don't care

23.2.5.15 Open-Drain Feature

The following apply if the open-drain feature is enabled on a pin, that is, the corresponding bit in the N2HET Open Drain Register (HETPDR) is set:

- Output buffer is enabled if a low signal is being driven internally to the pin.
- The output buffer is disabled if a high signal is being driven internally to the pin.

23.2.5.16 N2HET Pin Disable Feature

This feature is provided for the safe operation of systems such as power converters and motor drives. It can be used to inform the monitoring software of motor drive abnormalities such as over-voltage, over-current, and excessive temperature rise.

Table 23-10 shows the conditions for the output buffer to be enabled/disabled.

Figure 23-25. N2HET Pin Disable Feature Diagram

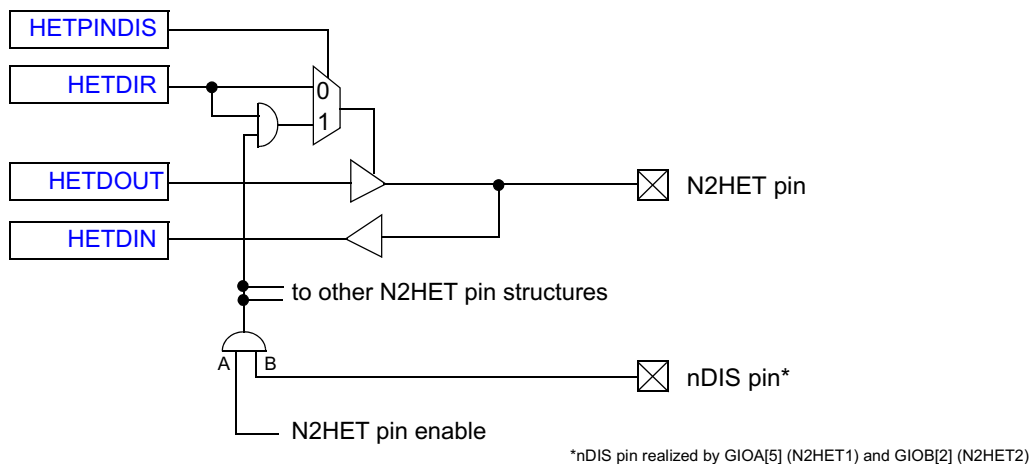


Table 23-10. N2HET Pin Disable Feature

HETPINDIS.x	nDIS Pin (Input)	HET_PIN_ENA (HETGCR.24)	HETDIR.x	Output Buffer
0	X	X	0	Disabled
0	X	X	1	Enabled
1	0	X	0	Disabled
1	0	X	1	Disabled
1	1	X	0	Disabled
1	1	0	1	Disabled
1	1	1	1	Enabled

An interrupt capable device I/O pin can share the same pin as the N2HET nDIS signal. Normally GIOA[5] serves as nDIS for N2HET1 and GIOB[2] as nDIS for N2HET2. Check the device datasheet for the actual implementation. Sharing a pin with a GIO pin that is Interrupt capable allows the N2HET nDIS input to also generate an interrupt to the CPU. An active low level on nDIS is intended to signal an abnormal situation as described above. All N2HET pins, which are selected with the N2HET Pin Disable Register (HETPINDIS), will be put in the high-impedance state by hardware immediately after the nDIS signal is pulled low. At this time a CPU interrupt is issued, if it is enabled in the GIO pin logic.

The bit HET_PIN_ENA is automatically cleared in the failure condition and this state remains as long as the software explicitly sets the bit again. The steps to do this are:

- Software detects, by reading the HETDIN register of the GIO pin, that the level on nDIS is inactive (high).
- Software sets bit HET_PIN_ENA to deactivate the high impedance state of the pins.

23.2.6 Suppression Filters

Each N2HET pin is equipped with a suppression filter. If the pin is configured as an input it enables to filter out pulses shorter than a programmable duration. Each filter consists of a 10-bit down counter, which starts counting at a programmable preloaded value and is decremented using the VCLK2 clock.

- The counter starts counting when the filter input signal has the opposite state of the filter output signal. The output signal is preset to the same input signal state after reset, in order to ensure proper operation after device reset.
- Once the counter reaches zero without detecting an opposite pin state on the filter input signal, the output signal is set to the opposite state.
- When the counter detects an opposite pin action on the filter input signal before reaching zero, the counter is loaded with its preload value and the opposite pin action on the filter output signal does not take place. The counter resumes at the preload value until it detects an opposite pin action on the input signal again.
- Therefore the filter output signal is delayed compared to the filter input signal. The amount of delay depends on the counter clock frequency (VCLK2) and the programmed preload value.
- The accuracy of the output signal is +/- the counter clock frequency.

Table 23-11 gives examples for a 100 MHz VCLK2 frequency.

Figure 23-26. Suppression Filter Counter Operation

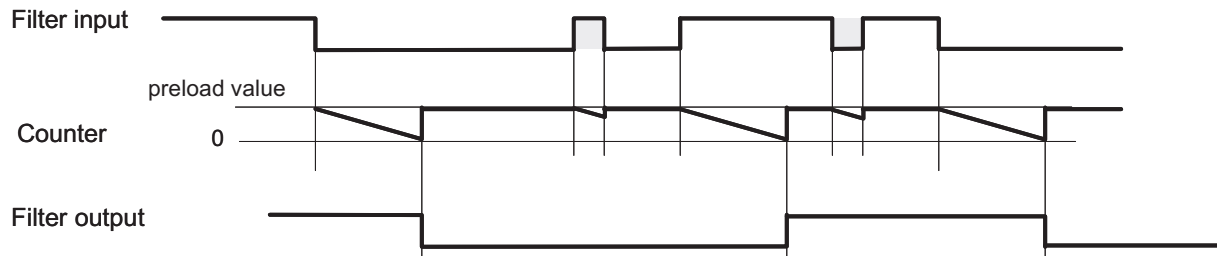


Table 23-11. Pulse Length Examples for Suppression Filter

Divider CCDIV	VCLK2	Possible values for the suppressed pulse length / frequency resulting from the programmable 10 bit preload value (0,1,...,1023)	
1	100.0 MHz	10 ns, 20 ns, ..., 10.22 μ s, 10.23 μ s	50 MHz, 25 MHz, ..., 48.924 kHz, 48.876 kHz
2	50.0 MHz	20 ns, 40 ns, ..., 20.44 μ s, 20.48 μ s	25 MHz, 12.5 MHz, ..., 24.462 kHz, 24.414 kHz
3	33.3 MHz	30 ns, 60 ns, ..., 30.66 μ s, 30.69 μ s	16.7 MHz, 8.3 MHz, ..., 16.308 kHz, 16.292 kHz

23.2.7 Interrupts and Exceptions

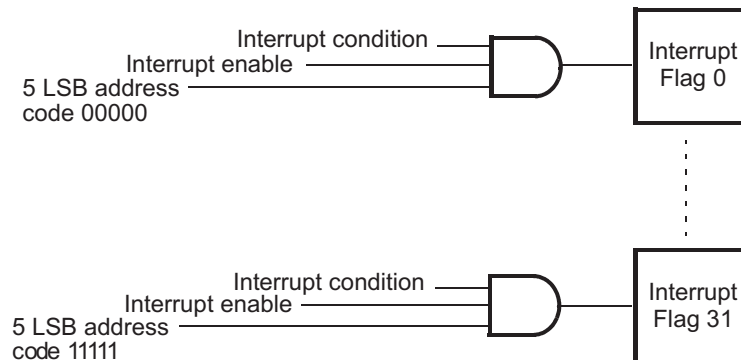
N2HET interrupts can be generated by any instruction that has an interrupt enable bit in its instruction format. When the interrupt condition in an instruction is true and the interrupt enable bit of that instruction is set, an interrupt flag is then set in the N2HET Interrupt Flag Register (HETFLG). The address code for this flag is determined by the five LSBs of the current timer program address. The flag in the N2HET Interrupt Flag Register (HETFLG) is set even if the corresponding bit in the N2HET Interrupt Enable Set Register (HETINTENAS) is 0. To generate an interrupt, the corresponding bit in the N2HET Interrupt Enable Set Register (HETINTENAS) must be 1. In the N2HET interrupt service routine, the main CPU must first determine which source inside the N2HET created the interrupt request. This operation is accelerated by the N2HET Offset Index Priority Level 1 Register (HETOFF1) or N2HET Offset Index Priority Level 2 Register (HETOFF2) that automatically provides the number of the highest priority source within each priority level. Reading the offset register will automatically clear the corresponding N2HET interrupt flag that created the request. However, if the offset registers are not used by the N2HET interrupt service routine, the flag should be cleared explicitly by the CPU once the interrupt has been serviced.

Table 23-12. Interrupt Sources and Corresponding Offset Values in Registers HETOFFx

Source No.	Offset Value
no interrupt	0
Instruction 0, 32, 64...	1
Instruction 1, 33, 65...	2
:	:
Instruction 31, 63, 95...	32
Program Overflow	33
APCNT underflow:	34
APCNT overflow	35

The instructions capable of generating interrupts are listed in [Table 23-75](#).

Figure 23-27. Interrupt Functionality on Instruction Level



Each interrupt source is associated with a priority level (level 1 or level 2). When multiple interrupts with the same priority level occur during the same loop resolution the lowest flag bit is serviced first.

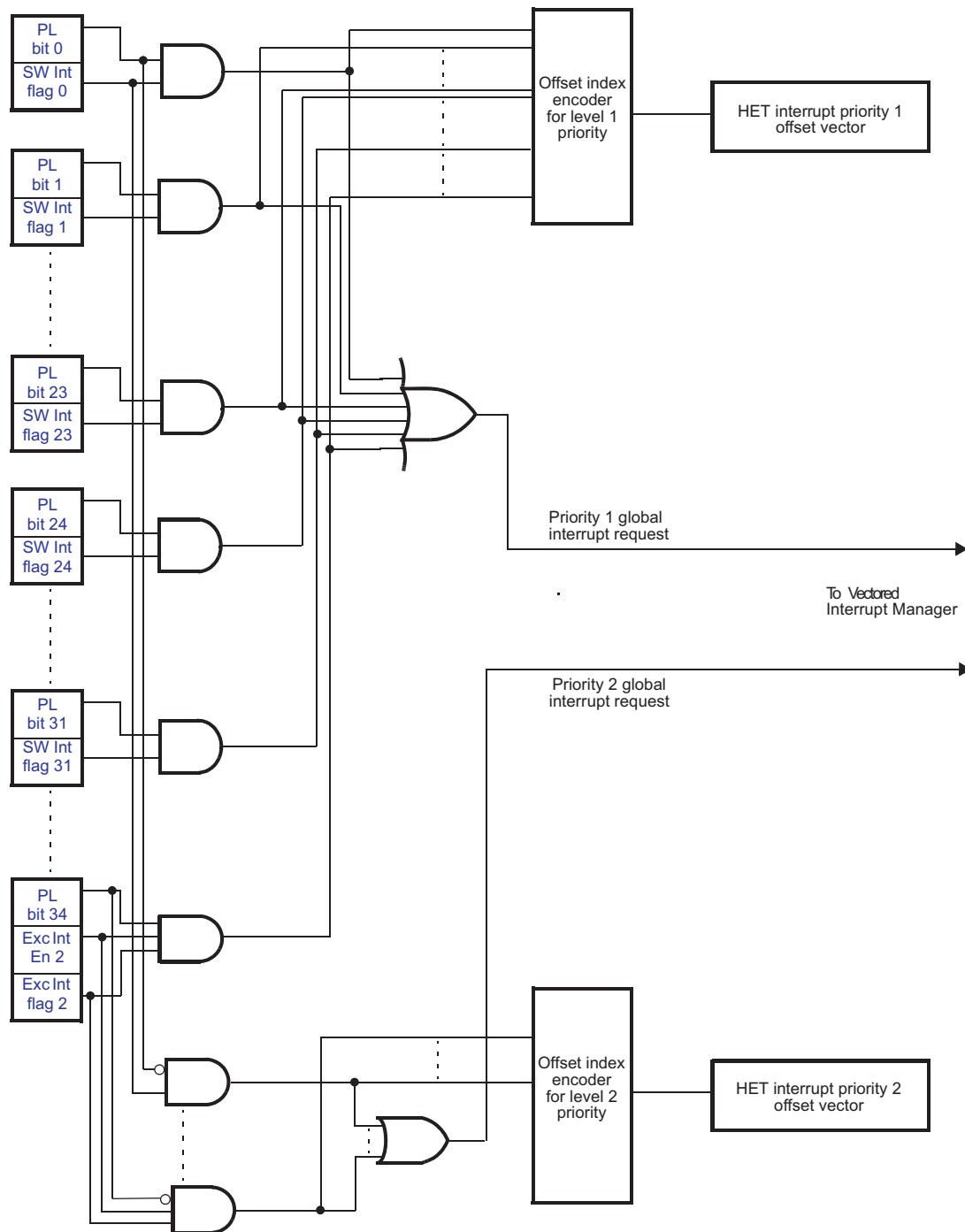
In addition to the interrupts generated by the instructions the N2HET can generate three additional exceptions:

- Program overflow
- APCNT underflow (see [Section 23.3.1.2](#))
- APCNT overflow (see [Section 23.3.1.3](#))

23.2.8 Hardware Priority Scheme

If two or more software interrupts are pending on the same priority level, the offset value will show the one with the highest priority. The interrupt with the highest priority is the one with the lower offset value. This scheme is hard-wired in the offset encoder. See [Figure 23-28](#).

Figure 23-28. Interrupt Flag/Priority Level Architecture

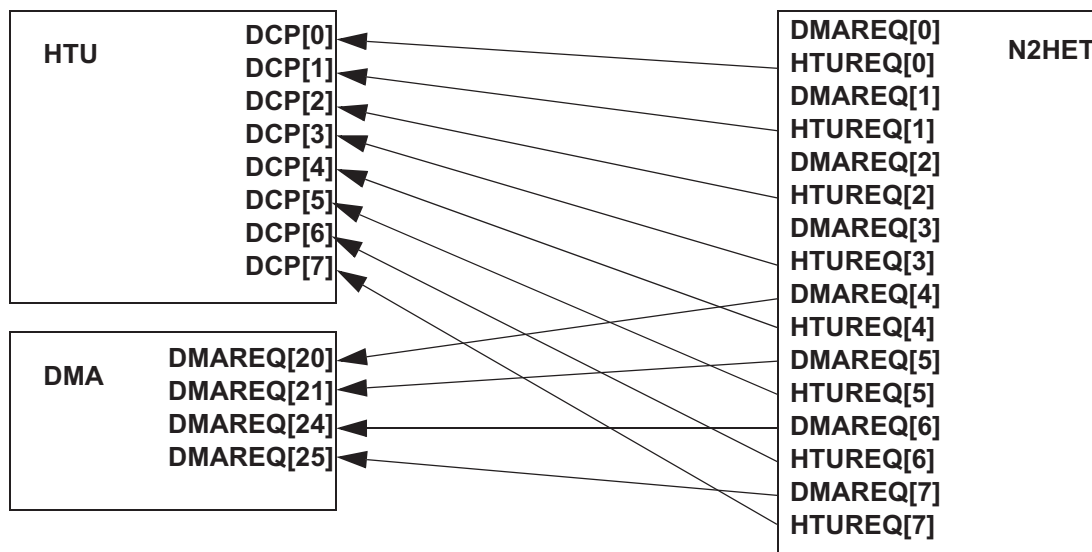


23.2.9 N2HET Requests to DMA and HTU

As described in [Section 23.6.3](#), the majority of the N2HET instructions are able to generate a transfer request to the High-End Timer Transfer Unit (HTU) and/or to the DMA module when an instruction-specific condition is true. One N2HET instruction can select one of 8 request lines by programming the “reqnum” parameter. The “request” field in an instruction is used to enable, disable, or to generate a quiet request (see [Section 23.6.2](#)) on the selected request line. Quiet requests can be used by the HTU, but not by the DMA. For quiet request, refer to the *High-End Timer Transfer Unit (HTU) Module* chapter (see [Section 24.2.4.1](#)).

The configuration of the N2HET Request Destination Select Register (HETREQDS) bits determines if a request line triggers an HTU-DCP, a DMA channel or both. This means the register bits will determine whether an N2HET instruction triggers DMAREQ[x], HTUREQ[x] or both signals (shown in [Figure 23-29](#)). The request line number x corresponds to the “reqnum” parameter used in the instruction.

Figure 23-29. Request Line Assignment Example



23.3 Angle Functions

Engine management systems require an angle-referenced time base to synchronize signals to the engine toothed wheel. The N2HET has a method to provide such a time base for low-end engine systems. The reference is created by the N2HET using three dedicated instructions with fractional angle steps equal to $/8$, $/16$, $/32$, $/64$.

23.3.1 Software Angle Generator

The N2HET provides three specialized count instructions to generate an angle referenced time base synchronized to an external reference signal (the toothed wheel signal) that defines angular reference points.

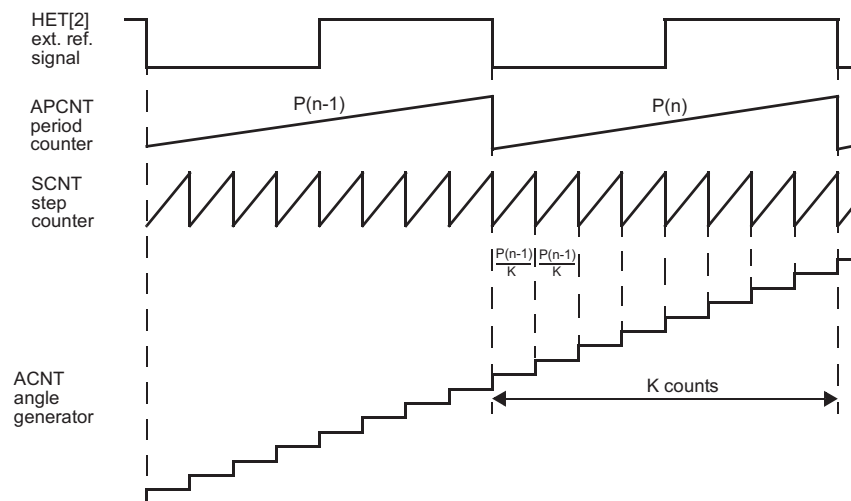
The time base is used to generate fractional angle steps between the reference points. The step width K ($= 8, 16, 32$, or 64) programmed by the user defines the angle accuracy of the time base. These fractional steps are then accumulated in an angle counter to form the absolute angle value.

The first counter, APCNT, incremented on each loop resolution clock measures the periods $P(n)$ of the external signal. The second counter SCNT counts by step K up to the previous period value $P(n-1)$, measured by APCNT, and then recycles. The resulting period of SCNT is the fraction $P(n-1) / K$. The third counter ACNT accumulates the fractions generated by SCNT.

[Figure 23-30](#) illustrates the basic operation of APCNT, SCNT, and ACNT.

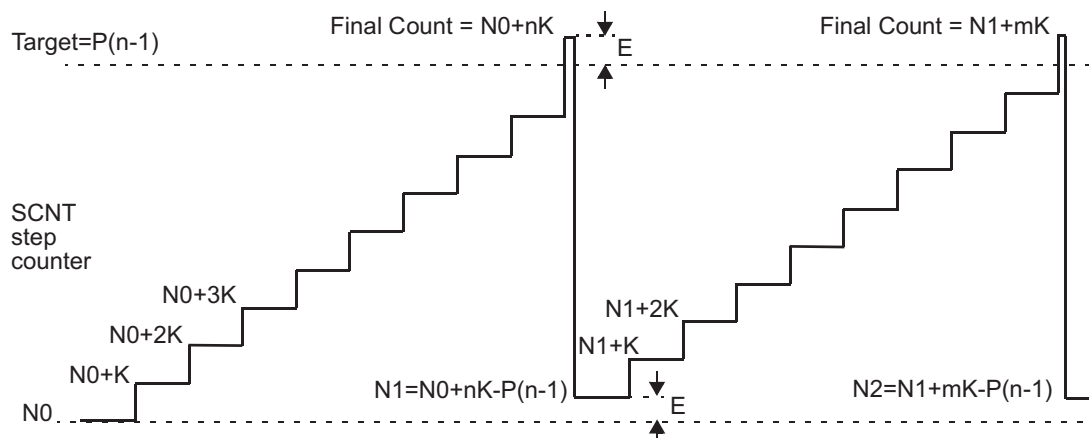
A N2HET timer program can only have one angle generator.

Figure 23-30. Operation of N2HET Count Instructions



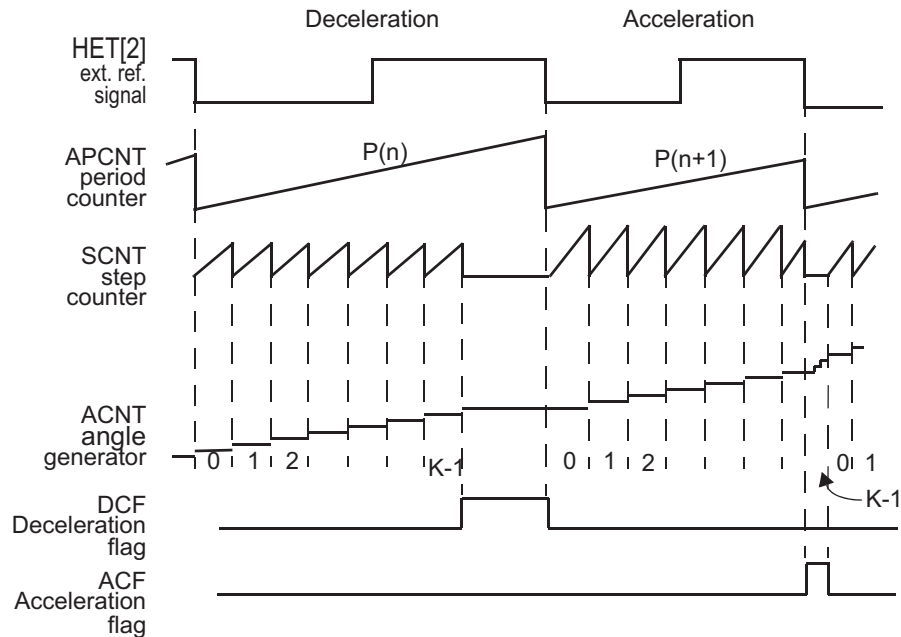
Due to stepping, the final count of SCNT does not usually exactly match the target value $P(n-1)$. [Figure 23-31](#) illustrates how SCNT compensates for this feature by starting each cycle with the remainder (final count - target) of the previous cycle.

Figure 23-31. SCNT Count Operation



ACNT detects period variations of the external signal measured by APCNT and compensates related counting errors. A period increase is flagged in the deceleration flag. A period decrease is flagged in the acceleration flag. If no variation is flagged, ACNT increments the counter value each time SCNT reaches its target. If acceleration is detected, ACNT increments the counter value on each timer resolution (fast mode). If deceleration is detected, ACNT is stopped. Figure 23-32 illustrates how the compensations for acceleration and deceleration operate.

Figure 23-32. ACNT Period Variation Compensations



23.3.1.1 Singularities

Singularities (gaps, in this case, from missing teeth in a toothed wheel) in the external reference signal can be masked. The start and end of singularities are defined by gap start and gap end values specified in SCNT and ACNT. When ACNT reaches gap start or gap end, it sets/resets the gap flag.

While the gap flag is set, new periods of the external reference signal are ignored for angle computation. SCNT uses the last period measured by APCNT just before gap start.

Figure 23-33 and Figure 23-34 illustrate the behavior of the angle generator during a gap after a deceleration or acceleration of the N2HET.

Figure 23-33. N2HET Timings Associated with the Gap Flag (ACNT Deceleration)

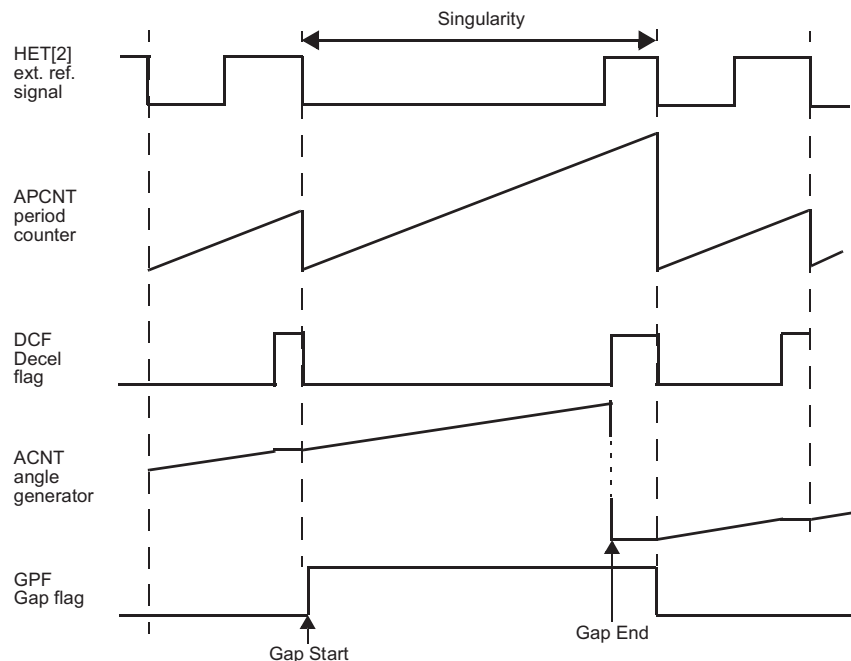
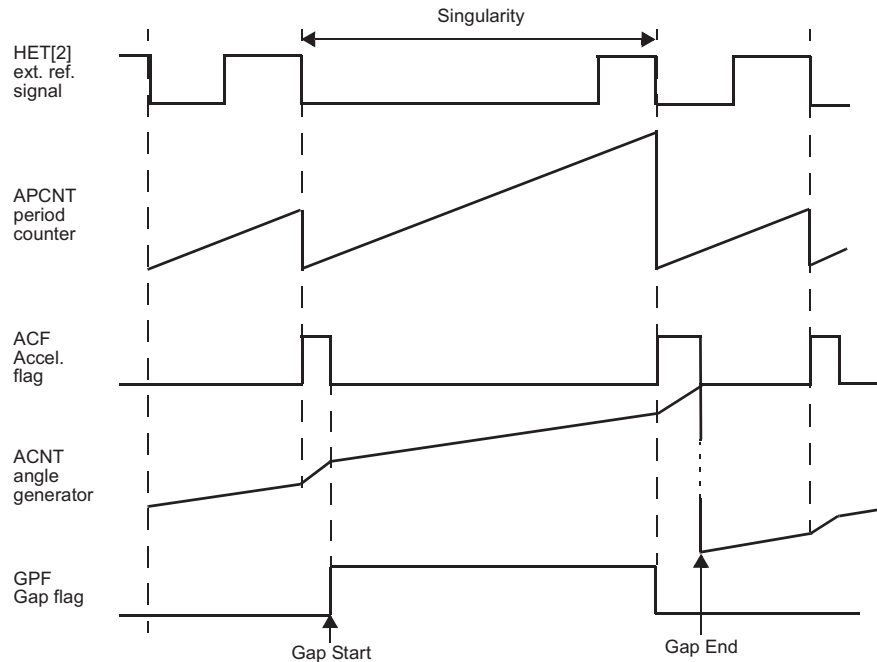


Figure 23-34. N2HET Timings Associated with the Gap Flag (ACNT Acceleration)


23.3.1.2 APCNT Underflow

The fastest valid external signal APCNT can accept must satisfy the following condition:

$$\text{Step Width } K < \text{Period Min. Resolution (LRP)}$$

This condition fixes the maximum possible step width once the minimum period and the resolution of an application are specified.

If a period value accidentally falls below the minimum allowed, APCNT stops the capture of these periods and sets the APCNT underflow interrupt flag located in the exceptions interrupt control register. In such a situation, SCNT and ACNT continue to be executed using the last valid period captured by APCNT.

23.3.1.3 APCNT Overflow

The slowest valid external signal APCNT can measure must satisfy the following condition:

$$\text{Period Max Resolution} < 33554431$$

When this limit is reached (APCNT Count equals all 1's), APCNT stays at a maximum count (stops counting). APCNT remains in this position until the next specified capture edge is detected on the selected pin and sets the APCNT overflow interrupt flag located in the exceptions interrupt control register. In this situation, SCNT and ACNT continue to be executed using the maximum APCNT period count.

23.3.2 Hardware Angle Generator (HWAG)

23.3.2.1 Overview

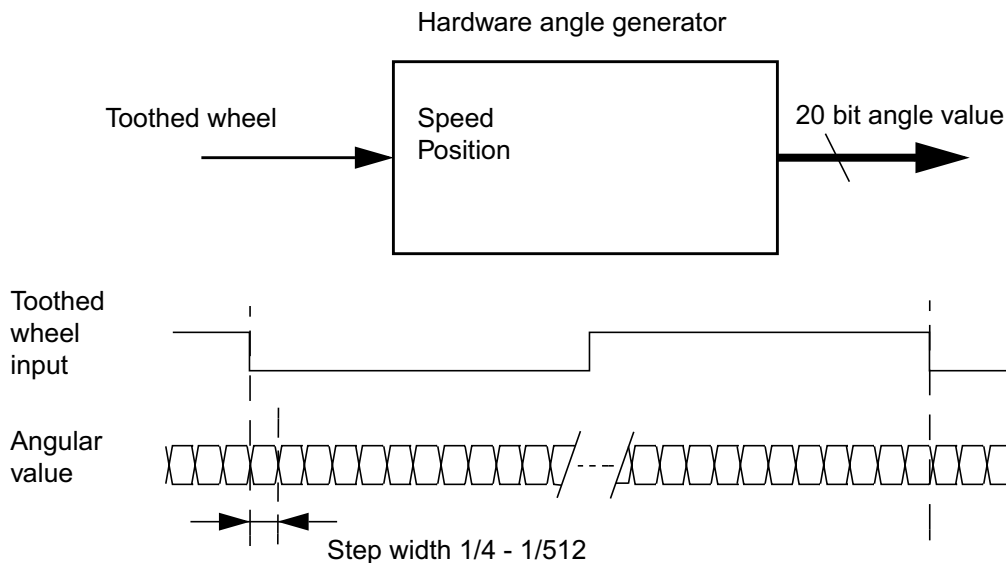
More engine control functions require powerful microcontrollers to process the timing. These controllers must generate signals such as dwell time, spark time, and fuel injection, at precise engine angles. These signals must be synchronized with the engine cycle.

The hardware angle generator (HWAG) generates angle value from toothed wheels. Because the toothed wheels are inaccurate (the most widely wheel used has 60 teeth with $6^\circ/\text{tooth}$), the period between two tooth edges (\(\Delta\)) interpolates the angle value and the step width gives the number of interpolated angles. For an example of the angle generator principle, see [Figure 23-35](#).

The HWAG can complement the high-end timer (NHET) to generate complex angle-angle or angle-time wave forms.

To work with the majority of toothed wheels, the HWAG provides registers to allow the CPU to configure step width, singularity, and filtering when initializing.

Figure 23-35. Angle Generator Principle



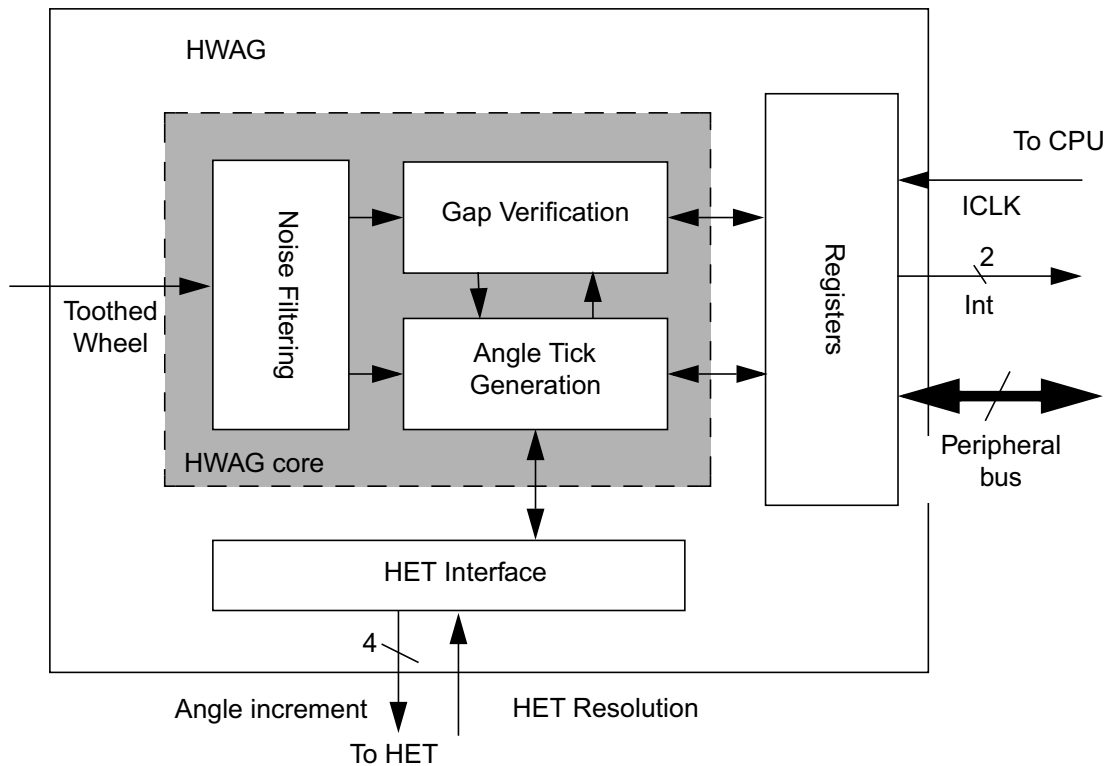
23.3.2.1.1 HWAG Features

The HWAG provides the following features:

- Programmable step width from $1/4$ to $1/512$
- Automatic synchronization check after first singularity synchronization
- Direct interface with the high-end timer
- 15 to 10,000 RPM range
- Programmable toothed-wheel input filter
- Programmable active edge on toothed-wheel
- Start bit synchronized to the tooth edge
- Pin selection capability for toothed-wheel input

23.3.2.1.2 Block Diagram

Figure 23-36. Hardware Angle Generator Block Diagram



23.3.2.2 HWAG Operation

23.3.2.2.1 Angle Tick Generation Algorithm

23.3.2.2.1.1 Angle Tick Generation Principle

The angle tick generator is the core kernel of this module. It uses the time-interpolation algorithm to generate angle ticks based on the last toothed wheel period. The angle counter is incremented at each new angle tick.

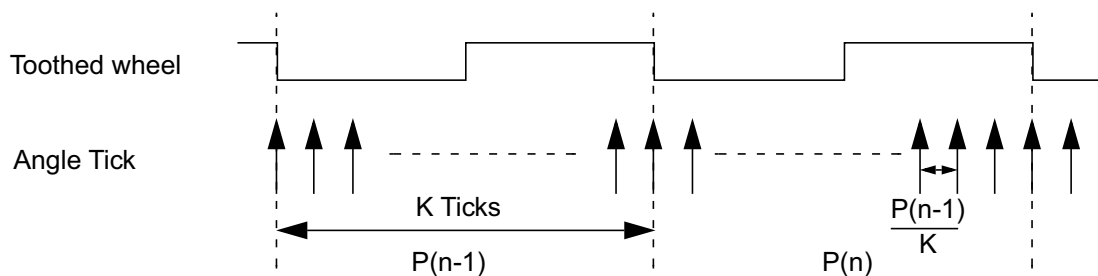
Because the toothed wheel is too inaccurate to fit with actual power-train applications, the algorithm is based on dividing the previous tooth period by K angle steps. The tooth period is the period between two active edges, which the HWAG global control register 2 (HWAGCR2) defines as the falling or the rising edge of the input signal. For an example of the angle tick generation principle, see [Figure 23-37](#).

The speed of the toothed wheel varies. This variance in speed creates some discontinuities in the angle counter behavior.

When the toothed wheel accelerates, the current period becomes shorter than the previous one and the tooth edge arrives before the last tick has been generated. To compensate for any missed ticks, the HWAG adds them to the angle counter when the active edge of the tooth arrives. The angle value is updated and resynchronized at each new active tooth edge.

When the toothed wheel decelerates, the period becomes longer than the previous period and K ticks are already counted before the active edge tooth arrives. After the last tick has been generated, the HWAG generates a tick only after the active tooth edge arrives.

Figure 23-37. Angle Tick Generation Principle



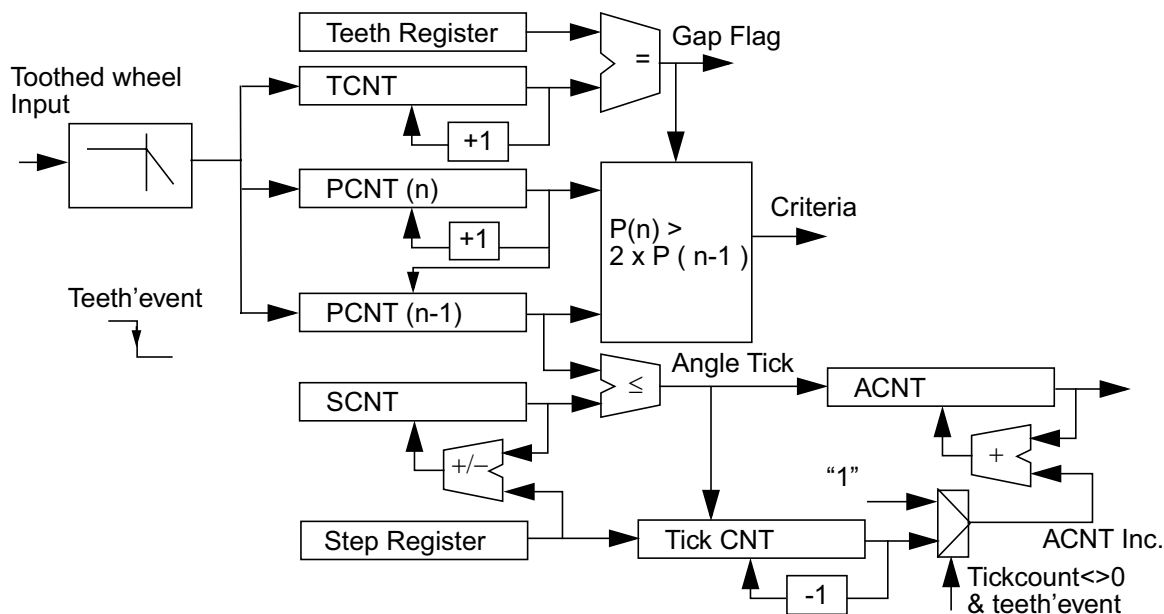
23.3.2.2.1.2 Angle Tick Generation Implementation

The time-interpolation algorithm, which generates ticks based on the toothed wheel tooth period, consists of the following five main counters linked together:

- Tooth counter (TCNT): Current tooth
- Period counter (PCNT): Period between two teeth
- Step counter (SCNT): Angle step
- Tick counter (TCKC): Angle ticks
- Angle counter (ACNT): Angle value

The algorithm also includes differences comparison, adder, and working registers as shown in [Figure 23-38](#).

Figure 23-38. New Angle Tick Generation Architecture



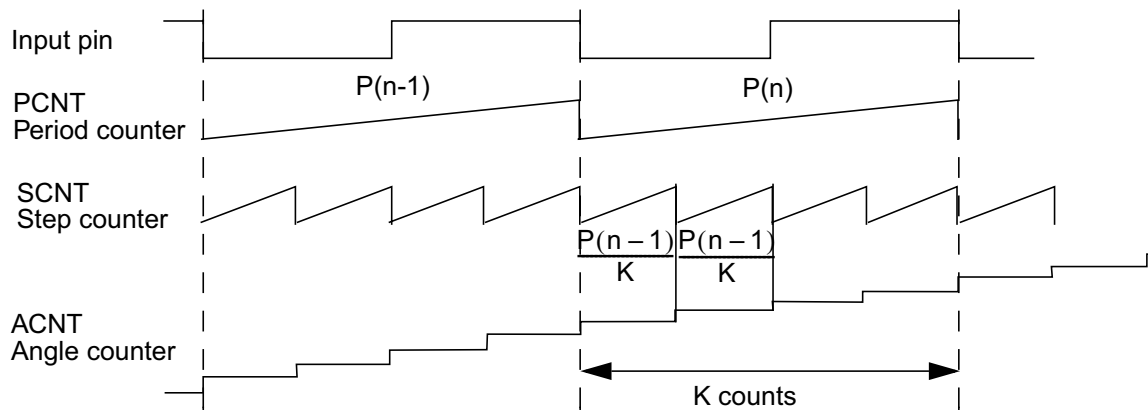
The TCNT is an 8-bit counter. It counts teeth until it reaches the teeth register value then generates a gap flag signal. The gap flag signal which changes the behavior of the HWAG during the singularity and resets the TCNT on the next active edge of the toothed wheel input.

The PCNT calculates the period $P(n)$ between two teeth (two active edges on the toothed wheel input). The active edge (falling or rising) is selected by setting the TED bit in the HWAG global control register 2 (HWAGCR2). On an active edge from the toothed wheel input, the PCNT is saved in the HWAG previous tooth period value register (HWAPCNT1).

The SCNT counts by K steps up to the previous period value, which is contained in the HWAPCNT1 register. When the SCNT overflows PCNT(n-1), an angle tick is generated and SCNT is reset to the remainder between the SCNT and PCNT(n-1). The resulting period of the SCNT is the fraction $PCNT(n-1)/K$.

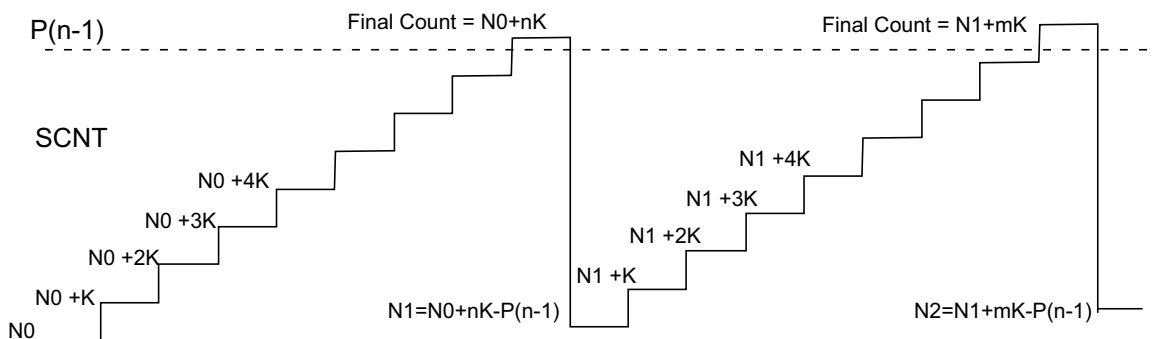
The TCKC counts every angle tick until it reaches K and then stops the SCNT. If an active edge occurs before the TCKC has reached K , the remainder is added directly to the ACNT.

When encountering an earlier active edge, the ACNT accumulates the fractions (angle ticks) generated by the SCNT and the remainder of the TCKC. For an example of angle generation using the time-based algorithm, see [Figure 23-39](#).

Figure 23-39. Angle Generation Using Time Based Algorithm


Because of stepping, the final count of the SCNT will usually be unequal to the target value PCNT(n-1) and then will overflow. To compensate for this error generated by the algorithm, reset the SCNT to the remainder of the difference between (SCNT - PCNT(n-1)).

To see how the SCNT and PCNT(n-1) generate angle ticks and compensate for the error due to the integer fractions, see [Figure 23-40](#).

Figure 23-40. SCNT Stepping Compensation


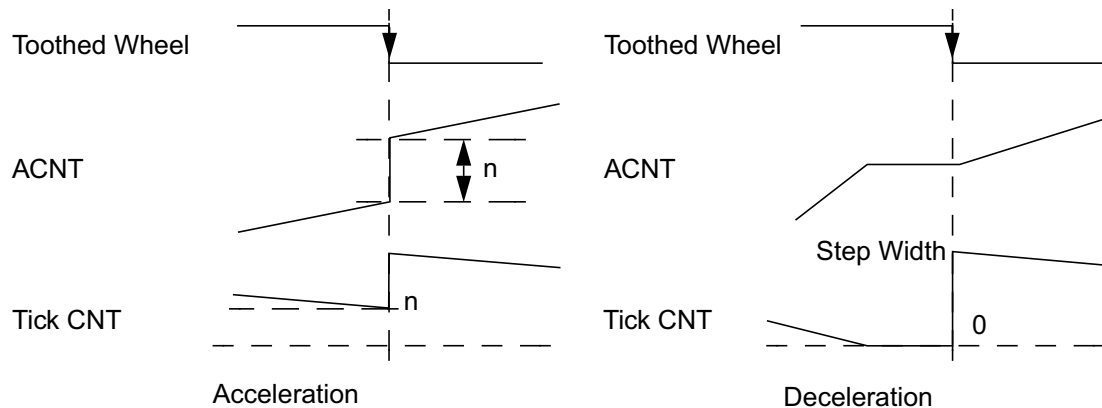
23.3.2.2.1.3 Acceleration and Deceleration

Because the toothed wheel speed is inconstant, it creates discontinuities in the angle counter behavior.

If the TCKC reaches zero before a new active tooth edge during a deceleration, the angle tick signal is no longer generated by the SCNT and PCNT(n-1). This halts the ACNT until the new active tooth arrives.

If the TCKC is unequal to zero when the new active tooth edge arrives during an acceleration (that is, the falling edge on the toothed wheel input in the example below), the rest of the tick counter increments the ACNT. For an example of the ACNT during acceleration and deceleration, see [Figure 23-41](#).

Figure 23-41. ACNT During Acceleration and Deceleration



23.3.2.2.1.4 End of Cycle

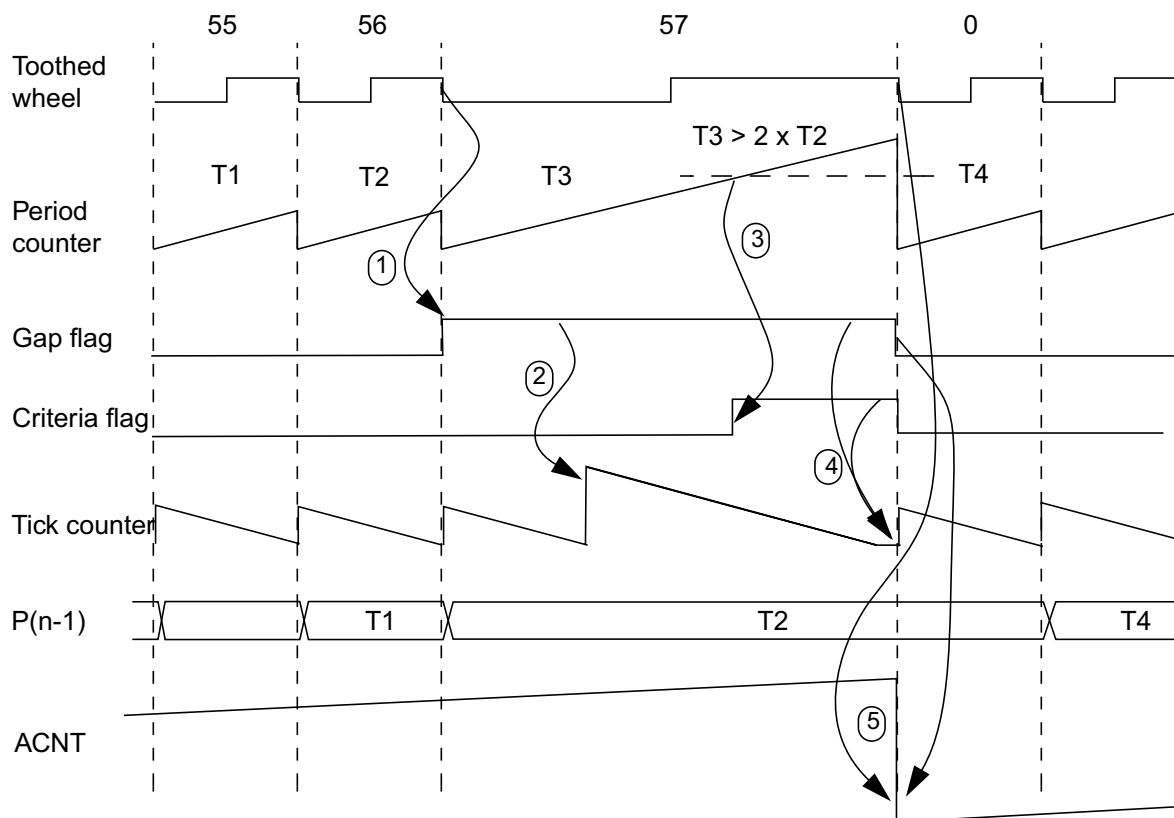
The HWAG behaves differently during the singularity tooth period of the toothed wheel. During the singularity period, the HWAG counts three virtual teeth (that is, three times the step width is added to the ACNT) to ensure that the ACNT reaches the maximum value (that is, every angle step has been counted) before resetting it.

During the singularity period, the HWAG generates angle ticks like for a normal tooth but with three times the value. To generate these angle ticks, the HWAG uses a constant period based on the previous tooth period. Because the period is based on the previous tooth period, the HWAG must recover from a deceleration or acceleration of three teeth when realizing the active edge tooth at the end of the singularity tooth.

The HWAG must ensure that the singularity occurs where expected and must verify it. When the singularity tooth arrives, TCNT reaches the teeth register, sets the signal gap flag, and then keeps PCNT(n-1) until the first tooth of the next round has passed. Because of these conditions, angle ticks before the second tooth will be based on the previous singularity tooth period.

The tick counter is first loaded with a normal value. When the counter reaches zero, it is reloaded once with twice the step width value if the criteria flag is not set. PCNT(n) continues to be incremented and to check the criteria with PCNT(n-1). For more information on gap verification, see [Section 23.3.2.2.4](#). The SCNT continues to generate angle ticks until the tick counter reaches zero the second time. The criteria flag validates the tooth in order to reset the counters. For an example of how the criteria flag validates the tooth to reset the counters, see [Figure 23-42](#).

When the tooth active edge occurs, the ACNT is incremented with the remainder value if the tick counter is not equal to zero. When the ACNT contains a value equals to K times the teeth register, the PCNT, the TCNT and the ACNT are reset to begin a new revolution.

Figure 23-42. Singularity Check, ACNT Reset and Timing Associated


- ① When TCNT = teeth register, the Gap flag is raised
- ② Tick CNT reloads automatically with 2x the step-width because the Gap flag = '1'
- ③ If PCNT (n) > 2 x PCNT (n-1) and the Gap flag = '1' then the Criteria flag is raised
- ④ The tick counter is not reloaded because the Criteria flag is raised
- ⑤ The Gap flag and tooth active edge reset, followed by ACNT

23.3.2.2.2 Angle Zero Initialization

Before any angle operation, initialize the HWAG and then initialize the angle zero as the singularity tooth. To initialize the angle zero as the singularity tooth, the HWAG can send an interrupt at each new tooth to help the software detect the first tooth if the interrupt is set. This allows you to decide which algorithm to apply to detect the zero degree tooth (by enabling the corresponding interrupt, you can also use the wired criteria).

When researching which algorithm to apply, the counters ACNT and TCNT are frozen and must be initialized to their start values. The ACNT value is equal to T times the step value (T is the tooth where the start will take effect and the initial value of the tooth counter). The counters PCNT(n) and PCNT(n-1) contain the current period and the previous period respectively. These counters allow you to set a detection criteria. When the application software sets the start bit, the software unfreezes the ACNT and TCNT counters. The counters count from the preloaded values at the next tooth active edge. The ACNT is preloaded with the value of 2 teeth and started synchronously with the next active edge of the toothed wheel. For an example of the HWAG start sequence, see [Figure 23-43](#).

Figure 23-43. Example of HWAG Start Sequence

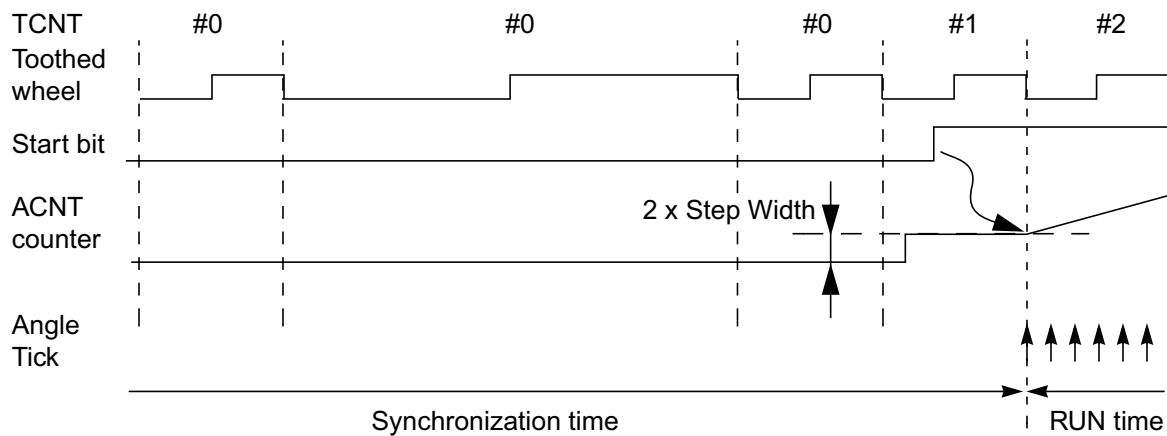


Figure 23-44 is an example of a singularity research initializing the HWAG at the second tooth to start synchronously with the third tooth. The HWAG angle value register (HWAACNT) contains 1024 (2×512) and the HWAG current teeth number register (HWATHVL) contains 2.

The code is executed in a tooth interrupt subroutine in code using the $PCNT(n-2) > PCNT(n-3) + PCNT(n-1)$ algorithm.

Figure 23-44. Code

```

TOOTH_INT LDR      R0, =HWAG_LOC; Find singularity tooth
          LDR      R1, [R0, #HWAPCNT1]; load PCNT(n-1) from HWAG
          LDR      R2, PCNT2 ; load PCNT(n-2) from memory
          LDR      R3, PCNT3 ; load PCNT(n-3) from memory
          ADD      R3, R3, R1 ; PCNT (n-1)+PCNT(n-3)
          CMP      R2, R3 ; Compare.
          BGT      SETSTRT ; Set start bit if R2 > R3
          STR      R2, PCNT3 ; PCNT2 -> PCNT3 Else Save Value
          STR      R1, PCNT2 ; PCNT1 -> PCNT2
          B        EXIT ; Wait for next tooth
SETSTRT   MOV      R10, #0xFF
          STR      R10, [R0, #HWAFLG]; Clear all the interrupt pending FLG
          LDR      R10, [R0, #HWACTL]; Load HWACTL register
          ORR      R10, R10, #0x0100 ; start bit enable
          STR      R10, [R0, #HWACTL]; Set start bit into HWAG
EXIT      SUBS     PC, LR, #4 ; Return from interruption
PCNT2     DCD      0x00000000 ; reserved memory for PCNT (n-2)
PCNT3     DCD      0x00000000 ; reserved memory for PCNT (n-3)

```

23.3.2.2.3 Stopping the HWAG

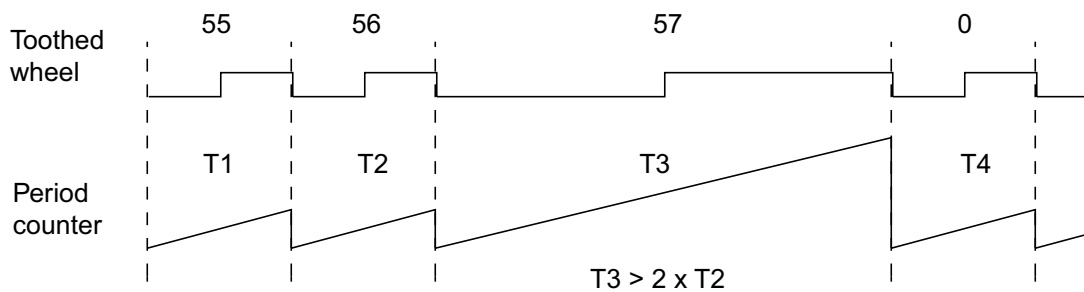
The HWAG starts synchronously with the active edge of the toothed wheel, but stops when the start (STRT) bit in the HWAG global control register 2 (HWAGCR2) is reset. Within a tooth, the HWAG can be stopped and parameters can be changed (that is, step width, angle counter, and so on) If this happens, the restart will take effect on the next active tooth edge.

NOTE: When stopping the HWAG, stop the angle increment delivered to the NHET and set it to zero. Reload the NHET counter with the same value of the angle counter ($\pm$ corrections), if restarting the HWAG.

23.3.2.2.4 Gap Verification

After the CPU sets the synchronization and puts the HWAG into RUN time (that is, the start bit is set), the tooth counter counts until reaching the teeth register (the number of real teeth of a full wheel revolution). When the tooth counter reaches the teeth register, the gap flag signal is set. For more information on the end of the cycle, see [Section 23.3.2.2.1.4](#). When the gap flag signal is set, it allows the HWAG to verify if the singularity is in the correct position (last tooth). The module then applies the $PCNT(n) > 2 \times PCNT(n-1)$ criteria by comparing $PCNT(n)$ and $PCNT(n-1)$ with one bit left shifted. If the criteria does not match when the tooth arrives, then the HWAG sends an interrupt to the CPU and does not reset the ACNT counter. The application software must recover from such an interrupt to keep the HWAG operating optimally. For an example of gap verification criteria for a 60-2 toothed wheel, see [Figure 23-45](#).

Figure 23-45. Gap Verification Criteria For a 60-2 Toothed Wheel



If the hardware criteria is not enabled, you must set the angle reset (ARST) bit in the HWAG global control register 2 (HWAGCR2) to validate the singularity. The HWAGCR2 register must validate the singularity before the active edge of the singularity tooth. If the HWAGCR2 register fails to validate the singularity, the HWAG generates an interrupt and does not clear the ACNT counter when the tooth edge occurs.

NOTE: For a 60-2 toothed wheel, set the ARST flag after the reload of the tick counter(when $PCNT(n) = PCNT(n-1)$). By verifying the criteria, the application software can set the ARST bit after this point.

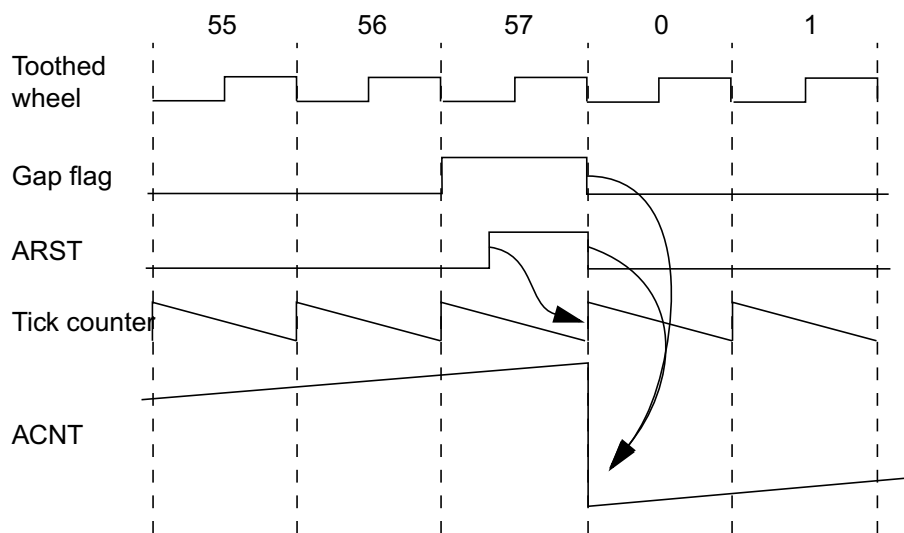
The CPU can read the PCNT counter and make a custom criteria set the ARST bit on time for the HWAG. The application software can use the gap flag interrupt to find the singularity tooth. Alternately, the CPU can verify the validity of the singularity in the second tooth with a more accurate criteria by using the HWAG previous tooth period value register (HWAPCNT1).

23.3.2.2.4.1 Use of the ARST Bit In Case of a Toothed Wheel Without Singularity

If a toothed wheel has no singularity (that is, no missing teeth), the ACNT must be reset when it reaches the angle zero point. To reset the ACNT when it reaches the angle zero point, set the ARST bit to 1.

Setting the ARST bit before the reload of the tick counter will cause the HWAG to fail to reload the tick counter. The HWAG will act like a normal tooth but the next active edge on the toothed wheel input will reset the ACNT and TCNT and clear the ARST bit. For an example of using the ARST bit in a toothed wheel without singularity, see [Figure 23-46](#).

Figure 23-46. Using the ARST Bit in a Toothed Wheel Without Singularity



23.3.2.2.5 Input Noise Filtering

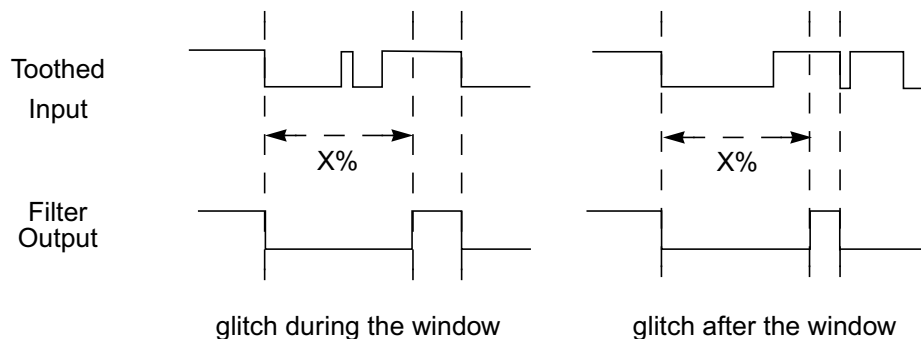
The toothed wheel input comes from an analog part and is sensitive to external noise. Due to this sensitivity, the input needs to be filtered because of glitches in the signal.

The HWAG digitally filters the toothed wheel input signal before it is used inside the core. The filter blocks the signal which negates the effect inside the HWAG. The HWAG provides two filter registers that filter the same way.

The filters validate the input signal after n angle ticks. The n angle ticks are like $X\%$ of the tick counter. The value of the remaining percentage of the tick counter ($1 - X\%$) need to be set because the tick counter is a down counter. Calculate the value to put into the filter registers from the step width value (or angle ticks value per tooth). The toothed wheel input is like a low pass filter with a cut-off frequency that functions like a toothed-wheel speed, but without acceleration and decelerations side effects. For an example of a windowing filter for a toothed wheel input on a falling active edge, see [Figure 23-47](#).

NOTE: At any time, the CPU can modify the filter values to fine tune with the application.

Figure 23-47. Windowing Filter for Toothed Wheel Input on Falling Active Edge



To calculate this number:

$$\text{Step Width} \times (1 - X\%) = \text{Filter Register Value}$$

If the step width value is equal to 512 and you want to filter 75% of the tooth, calculate the filter register as follows:

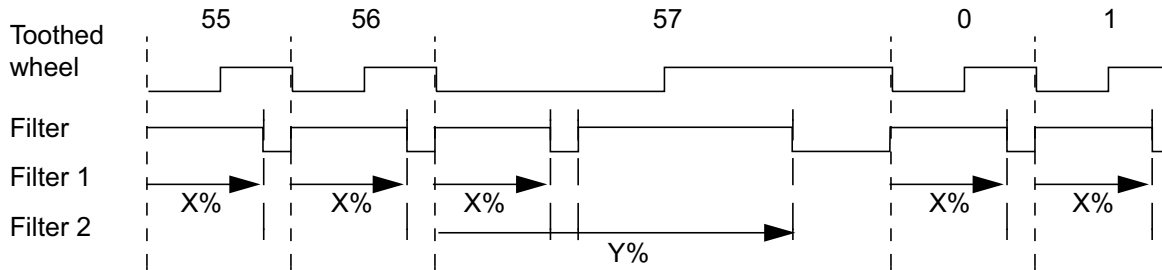
$$512 \times (1 - 0.75) = 128$$

When the tick counter reaches the filter register value, the toothed wheel input is unblocked.

23.3.2.2.5.1 Filter During Singularity Tooth

During the singularity tooth, the filter acts differently than during a normal tooth. The filter releases the input for a normal tooth. When the tick counter is reloaded, a second filter value is applied to the toothed wheel input. For an example of filtering during a singularity tooth, see [Figure 23-48](#).

Figure 23-48. Filtering During Singularity Tooth



The second filter value is set using the same equation as the first filter with the step width multiplied by 3.

To calculate this number:

$$(3 \times \text{Step Width}) \times (1 - Y\%) = \text{Second Filter Register Value} \quad (34)$$

If the step width value is equal to 512 and you want 70% of singularity tooth period to be filtered, calculate the filter register value as follows:

$$3 \times 512 \times (1 - 0.70) = 460$$

23.3.2.2.6 HWAG Interrupts

When conditions are set, the HWAG interrupts are generated.

When the interrupt condition is true, the corresponding flag is set in the HWAG interrupt flag register (HWAFLG). If the corresponding enable bit in the HWAG interrupt enable set register (HWAENASET) is also set, an interrupt request is sent to the CPU through one of the interrupt lines, depending on the priority of the interrupt (HWAG interrupt level set register (HWALVLSET)).

Because the HWAG can set interruptions, the CPU must determine which source created the interrupt request and then execute the interrupt service routine. The CPU reads the offset register (HWAOFFx) that gives the number of the source. If the CPU reads the offset register, it will automatically clear the source flag that created the request.

NOTE: If the corresponding enable bit is not set, a read in the offset register will not clear a flag. To set the bit, write a 1 in the corresponding bit within the HWAG interrupt flag register (HWAFLG).

The HWAG generates eight different interrupts:

- 0 = Overflow period
- 1 = Singularity not found
- 2 = Tooth interrupt
- 3 = ACNT overflow
- 4 = PCNT(n) > 2 × PCNT (n-1) during normal tooth
- 5 = Bad active edge tooth
- 6 = Gap flag
- 7 = Angle increment overflow

For more information on these interrupts, see [Table 23-14](#). Each interrupt source is associated with a low or high priority. When one or more interrupts with the same priority occur, a fixed priority determines the offset vector if the corresponding enable bits are set.

The HWAG generates two interrupt request signals for the central interrupt module (CIM). For information on servicing interrupts, see [Figure 23-49](#). For a list offset values, see [Table 23-13](#).

Table 23-13. HWAG Interrupt Sources and Offset Values

Source Number	Offset Value
0	1
1	2
:	:
7	8

Figure 23-49. HWAG Interrupt Block Diagram

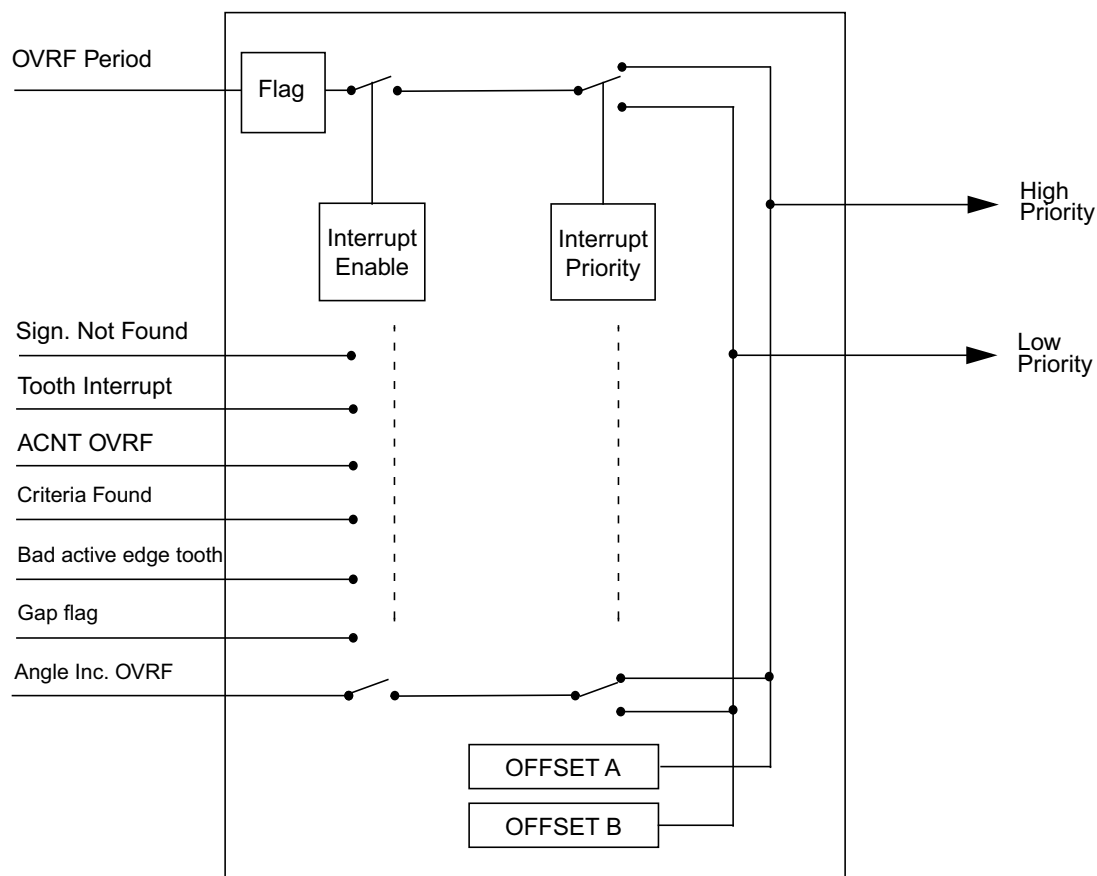


Table 23-14. HWAG Interrupt Descriptions

Interrupt Names	Interrupt Descriptions
Overflow period	Occurs when the PCNT (n) counter reaches the maximum value. Can occur if the toothed wheel input remains stable. May indicate failure of an engine stall or a toothed wheel sensor.
Singularity not found	When the TCNT counter sets the gap flag, the HWAG waits for the criteria flag to raise before the toothed wheel active edge. If the toothed wheel active edge occurs before the criteria flag, the HWAG raises the singularity not found interrupt flag.
New edge tooth	This interrupt can sync or let you control the tick generation. This interrupt indicates the new active edge tooth. This interrupt could be filtered or unfiltered (Bit FIL in control register).
Angle counter (ACNT) overflow	This interrupt occurs when the singularity is unable to be found. The angle counter (ACNT) continues until overflow.
Singularity found during normal tooth	This interrupt indicates that the period of the current tooth is at least two times longer than the previous one when the HWAG expects a normal tooth. This interrupt can detect the singularity without bit manipulation by the CPU.
Bad active edge tooth	This interrupt indicates that an active edge has occurred before the end of the filtering (toothed wheel input blocked) but the HWAG remains inactive internally. This interrupt can detect glitches on the toothed wheel input.
Gap flag	When TCNT reaches the teeth register and the HWAG raises the gap flag , This interrupt is set when the gap flag is raised by the HWAG,
Angle increment overflow	This interrupt indicates that the number of the angle increment is more than 15 since the last resolution tick. This interrupt can prevent any discrepancies between the NHET and the HWAG.

NOTE: Before enabling any interruption, clear the HWAG interrupt flag register (HWAFLG) to ensure that any interrupts have finished. If interrupts are pending, the HWAG could generate an interrupt based on an unrealistic event.

23.3.2.3 Emulation

Because the HWAG is designed to synchronize with a real-time environment, the HWAG counters continue during emulation.

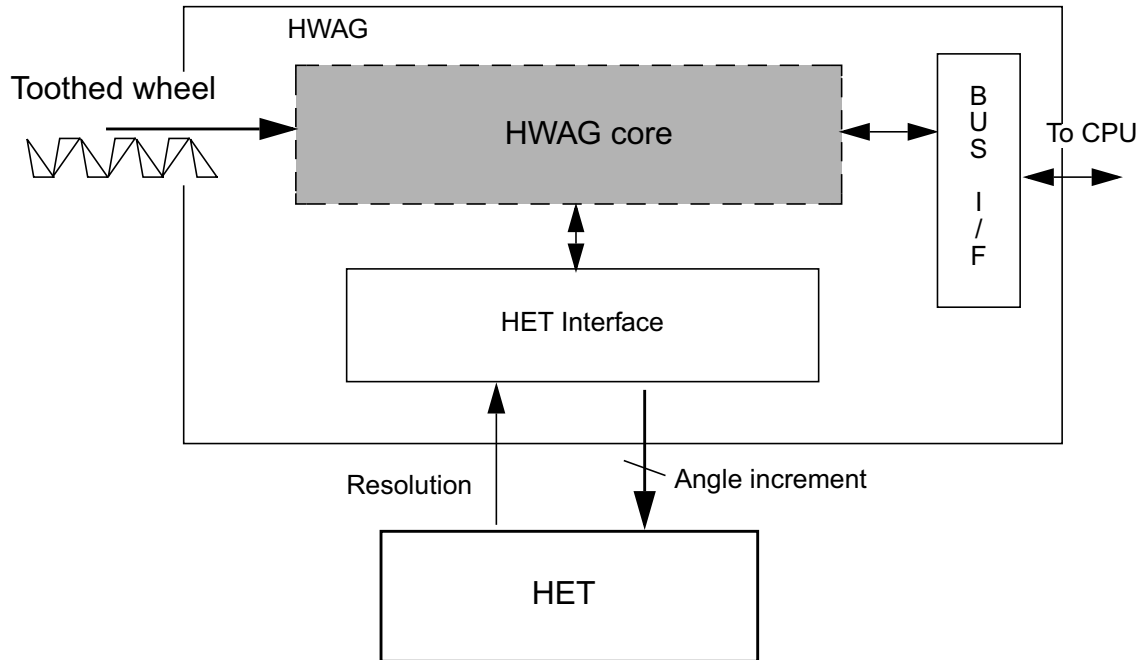
When the CPU is frozen, the HWAG continues to run and update registers. Only the offset registers remain uncleared when entering debug mode.

During debug mode, interrupts can occur and will wait until the CPU enters run mode again. If interrupts occur, they could affect synchronization with the toothed wheel

23.3.2.4 Hardware Angle Generator and High-End Timer

In the engine management application, the HWAG is usually connected to one or more high-end timers. This connection allows you to perform angle compare and angle/time compare. For an example of the hardware angle generator/high-end timer interface, see [Figure 23-50](#).

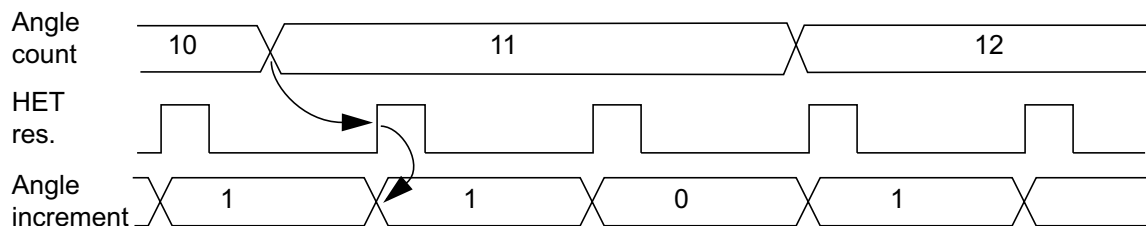
Figure 23-50. Hardware Angle Generator/High End Timer Interface



23.3.2.4.1 Signal Description

To perform a resynchronization, the HWAG interface provides to the NHET at every resolution clock an angle increment value that represents how much the angle counter of the HWAG has been incremented since the last NHET resolution clock. For an example of the angle count within the HWAG, see [Figure 23-51](#).

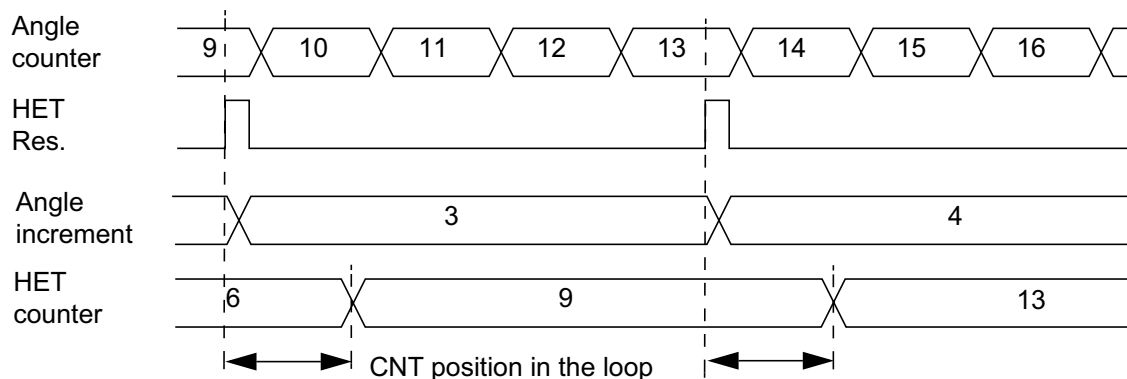
Figure 23-51. Angle Count Within the HWAG at Resolution Clock



When the engine speed increases, the angle count can increment by more than one in a NHET resolution but the HWAG will continue to provide the angle increment value at every resolution..

The NHET can then implement its own angle counter (using a CNT instruction in angle mode) which will be incremented once per resolution by the value given by the angle increment. For an example of an angle count within the NHET with increments, see [Figure 23-52](#).

Figure 23-52. Angle Count Within the NHET With Increments

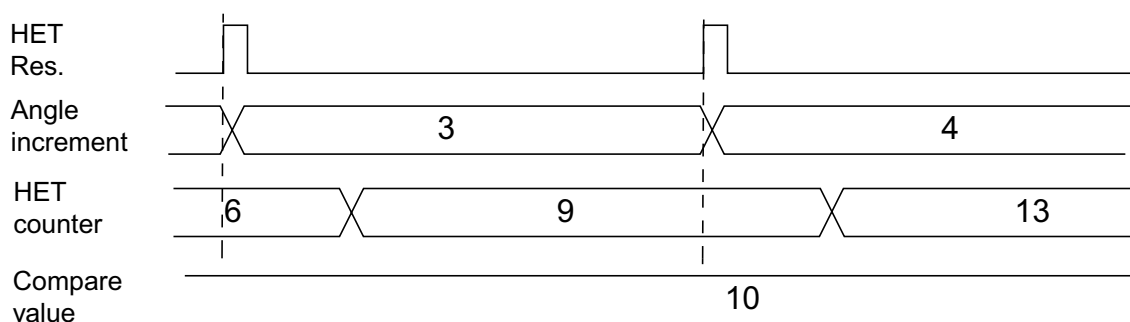


23.3.2.4.2 NHET Operation on Angle Functions (ACMP, CNT)

23.3.2.4.2.1 State of the Art

Because the angle value can be increased by more than one, the compare value could be in-between the old angle value and the new angle value of the NHET angle counter (where new angle value = old angle value + angle increment). To perform an angle compare that ensures not to miss a compare value, the NHET provides the ACMP instruction. For an example of a compare without ACMP instruction, see [Figure 23-53](#).

Figure 23-53. Compare Without ACMP Instruction

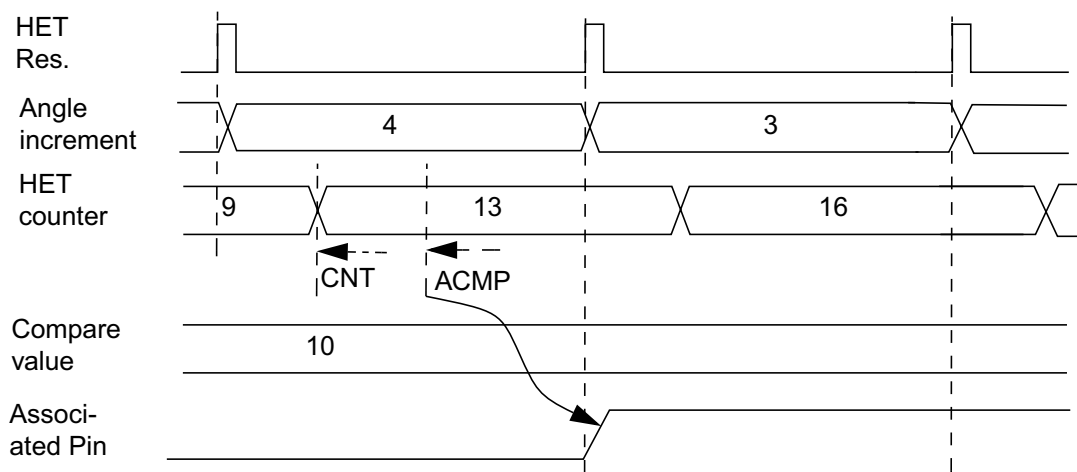


When the HET counter passes from 9 to 13, the equality compare can not match the compare value 10. Consequently, the angle position is missed!

23.3.2.4.2.2 ACMP Instruction Advantage

The ACMP instruction is more than an equality compare. ACMP instruction performs an in-between comparison (old angle value < compare value ≤ new angle value) to match the position of the toothed wheel. This instruction, where an equality compare executes every resolution, may miss a compare match. For an example of ACMP compare within the NHET, see [Figure 23-54](#).

Figure 23-54. Example of ACMP Compare Within the NHET



With the ACMP instruction, the compare that is performed will be: $9 < 10 \leq 13$

With the ACMP instruction, the compare is: $9 < 10 \leq 13$

NOTE: To avoid multiple matches, the ACMP only matches during a single resolution.

Performing the following equations at the same time implements this compare:

$CMP > NHET \text{ angle counter} - \text{Angle increment}$

$CMP \leq NHET \text{ angle counter}$

23.3.2.4.3 NHET Interface

23.3.2.4.3.1 Input Signal Selection

The input pin of the toothed-wheel signal is software selectable. In previous generations of NHET/HWAG, this was fixed to HET[2]. On this device, the input pin is programmable to provide more flexibility for the system implementation. However, the implementation is done in a way to be backward compatible.

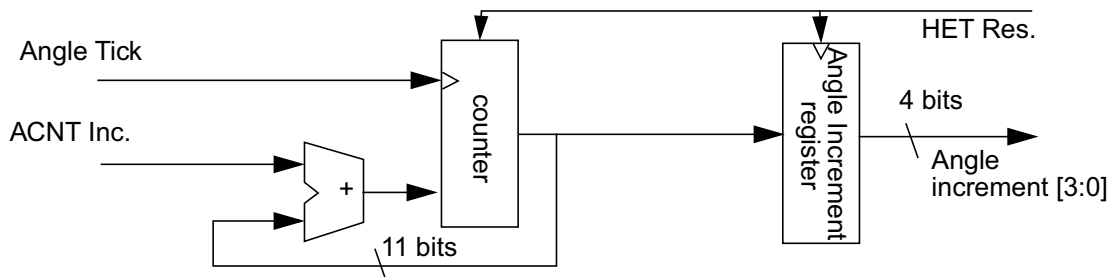
A separate register, HWAG pin select register (HWAPINSEL), is implemented to allow this selection functionality. The HWAPINSEL register should be programmed before the HWAG is turned on. The default selection will be HET[2] (PINSEL = 2h) after reset. The signals will be derived from the input buffer of each pin. This will allow configuring the pin as an output and measure back the output signal with the HWAG.

You can change the HWAPINSEL register at any time, but the proper functionality of the HWAG is not assured if the selection is changed while the HWAG is already operational. It is recommended that the input selection is done before the STRT bit in the HWAG global control register 2 (HWAGCR2) is programmed to 1.

23.3.2.4.3.2 HWAG to NHET Interface

The NHET interface is a 11-bit counter sampled by the NHET and reset by the NHET resolution. The counter contains the value of ACNT incremented during the last resolution (see [Section 23.3.2.4.1](#)). For the NHET interface block diagram, see [Figure 23-55](#).

Figure 23-55. NHET Interface Block Diagram



When the ACNT register is reset to zero, the angle increment register is not reset. The NHET software checks if its own angle register is higher than 360° and either clears it or continues to 720°. If ACNT is reset within the HWAG, the angle increment register gives the NHET the number of angle ticks from the last resolution.

During a strong acceleration after a tooth active edge, the number of angle ticks can exceed 15. If the number of ticks exceeds 15, the HWAG delivers to the NHET several angle increments at 15. This allows the NHET to follow without missing any angle positions from the HWAG. When the counter is below 15, the angle increment reflects the counter. When the angle increment overflows, it sets to 15, and if the enable bit (bit 7 in the control register) is set, the HWAG can send an interrupt to the CPU.

During a strong deceleration, the angle increment can stay null for one or more NHET resolution clocks.

To minimize the error between the fly-wheel and NHET angle counter, the step width and the NHET resolution must be set to avoid any overflow of the 11-bit counter of the NHET interface. This can happen if the number of angle ticks always exceeds 15 during one resolution.

23.3.2.5 Range of Operations

23.3.2.5.1 Intrinsic HWAG Limitation

The following factors limit the HWAG:

- SYSCLK
- PCNT counter (overflow)
- Number of teeth
- Angle step

These factors will influence the engine speed range (RPM limitation) and the maximum accuracy of the angle steps (wheel limitation).

- RPM limitation

The toothed wheel speed is limited by the period counter (PCNT) and the angle step for a given SYSCLK.

RPM minimum is related to PCNT overflow and SYSCLK.

Maximum PCNT value $\times$ SYSCLK = Maximum tooth period

$$\text{RPM} = \frac{60}{\text{TeethNumber} \times \text{ToothPeriod}}$$

PCNT is a 24-bit counter based on SYSCLK.

RPM maximum is related to the angle step and SYSCLK.

Minimum tooth period $>$ Step Width $\times$ SYSCLK

The angle ticks period could not be inferior to the SYSCLK.

Example: The toothed wheel is a 60-2, SYSCLK is 50 Mhz (20 ns), and step width is 512:

RPM minimum $\geq 16\,777\,215 \times 20\text{ ns} = 335.5443\text{ ms} \geq \sim 3\text{ RPM}$

RPM maximum $\geq 512 \times 20\text{ ns} = 10.24\text{ }\mu\text{s} \geq 97\,656\text{ RPM}$

NOTE: With a 60-2 toothed wheel, the tooth period is the reverse of the RPM number.

- Wheel Limitation

The HWAG is limited by the number of teeth and the increments in a revolution.

The maximum number of teeth is 256. This limits the number of increments per revolution to 512 steps $\times$ 256 teeth = 131 072 angle increments.

23.3.2.5.2 HWAG-NHET Limitation

The maximum angle accuracy is a function of the angle step and the NHET loop resolution.

The increment per resolution limits the interface between the HWAG and the NHET. The maximum angle increment per NHET resolution is 15 increments/NHET_res, which is an angular speed. If the angle increment overflows 15 during a constant speed, the system is diverging.

In the HWAG, the angular speed is given by the relation:

$$\text{Angular Speed} = \frac{\text{Step Width}}{\text{Minimum ToothPeriod}}$$

To ensure that the values are correct, they must satisfy the following equation:

$$\frac{\text{MaxHET resolution} \times \text{Step Width}}{\text{Minimum ToothPeriod}} < 15$$

Then,

$$\text{MaxHET resolution} = \frac{15 \times \text{Min ToothPeriod}}{\text{Step Width}}$$

Example: For a 60-2 at 10000 RPM, the tooth period is 100 μs and the step width is 512:

$$\text{MaxHET resolution} = \frac{15 \times 100}{512} = 2.93 \mu\text{s}$$

23.3.2.6 Tricks

23.3.2.6.1 Using HWAG Previous Tooth Period Value Register (HWAPCNT1)

The HWAG previous tooth period value register (HWAPCNT1) can compensate for errors because of acceleration or deceleration.

If there is a variation of the toothed wheel, the ACNT register will have a discontinuity. For an explanation of acceleration and deceleration, see [Section 23.3.2.2.1.3](#). Avoid this discontinuity by giving the HWAPCNT1 register a smaller or larger value, depending of the variation. When HWAPCNT1 is modified, the angle tick period is also be modified which causes faster or slower tick generation and decreases the discontinuity on the next falling edge.

Because of this compensation, the NHET interface will not overflow and fewer errors will occur on the NHET angle counter in case of strong acceleration.

NOTE: Reading the angle increment will give the application the amount of the acceleration. However, adding the value directly to the NHET counter will result in a discontinuity in the compare sequence. Particularly angle based compare could be missed.

23.3.2.6.2 Using the Singularity During Normal Tooth Interrupt

This interrupt detects if the HWAG is desynchronized with the toothed wheel and resynchronizes the HWAG.

Because the criteria was set during a tooth other than the singularity tooth, the interrupt occurs. Because the criteria is based on $PCNT > 2 \times PCNT(n-1)$, this interrupt is likely due to the singularity.

The following steps explain how to resynchronize the HWAG with this interrupt:

1. Stop the HWAG
2. Reset ACNT
3. Reset tooth counter
4. Reset interrupt
5. Set start bit.

The HWAG will restart on the tooth zero.

23.4 N2HET Control Registers

Table 23-15 summarizes all the N2HET registers. The base address for the control registers is FFF7 B800h for N2HET1 and FFF7 B900h for N2HET2.

Table 23-15. N2HET Registers

Offset	Acronym	Register Description	Section
00h	HETGCR	Global Configuration Register	Section 23.4.1
04h	HETPFR	Prescale Factor Register	Section 23.4.2
08h	HETADDR	NHET Current Address Register	Section 23.4.3
0Ch	HETOFF1	Offset Index Priority Level 1 Register	Section 23.4.4
10h	HETOFF2	Offset Index Priority Level 2 Register	Section 23.4.5
14h	HETINTENAS	Interrupt Enable Set Register	Section 23.4.6
18h	HETINTENAC	Interrupt Enable Clear Register	Section 23.4.7
1Ch	HETEXC1	Exception Control Register 1	Section 23.4.8
20h	HETEXC2	Exception Control Register 2	Section 23.4.9
24h	HETPRY	Interrupt Priority Register	Section 23.4.10
28h	HETFLG	Interrupt Flag Register	Section 23.4.11
2Ch	HETAND	AND Share Control Register	Section 23.4.12
34h	HETHRSH	HR Share Control Register	Section 23.4.13
38h	HETXOR	HR XOR-Share Control Register	Section 23.4.14
3Ch	HETREQENS	Request Enable Set Register	Section 23.4.15
40h	HETREQENC	Request Enable Clear Register	Section 23.4.16
44h	HETREQDS	Request Destination Select Register	Section 23.4.17
4Ch	HETDIR	NHET Direction Register	Section 23.4.18
50h	HETDIN	NHET Data Input Register	Section 23.4.19
54h	HETDOUT	NHET Data Output Register	Section 23.4.20
58h	HETDSET	NHET Data Set Register	Section 23.4.21
5Ch	HETDCLR	NHET Data Clear Register	Section 23.4.22
60h	HETPDR	NHET Open Drain Register	Section 23.4.23
64h	HETPULDIS	NHET Pull Disable Register	Section 23.4.24
68h	HETPSL	NHET Pull Select Register	Section 23.4.25
74h	HETPCR	Parity Control Register	Section 23.4.26
78h	HETPAR	Parity Address Register	Section 23.4.27
7Ch	HETPPR	Parity Pin Register	Section 23.4.28
80h	HETSFPRLD	Suppression Filter Preload Register	Section 23.4.29
84h	HETSFENA	Suppression Filter Enable Register	Section 23.4.30
8Ch	HETLBPSEL	Loop Back Pair Select Register	Section 23.4.31
90h	HETLBPDIR	Loop Back Pair Direction Register	Section 23.4.32
94h	HETPINDIS	NHET Pin Disable Register	Section 23.4.33

23.4.1 Global Configuration Register (HETGCR)

N2HET1: offset = FFF7 B800h; **N2HET2:** offset = FFF7 B900h

Figure 23-56. Global Configuration Register (HETGCR) [offset = 00h]

31										25										24									
Reserved																		HET_PIN_ENA											
R-0																		R/W-1											
23				22				21				20				19				18				17				16	
Reserved				MP				Reserved				PPF				IS				CMS									
R-0				R/W-0				R-0				R/W-0				R/W-0				R/W-0									
15																		1		0									
Reserved																		TO											
R-0																		R/W-0											

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-16. Global Configuration Register (HETGCR) Field Descriptions

Bit	Field	Value	Description
31-25	Reserved	0	Reads return 0. Writes have no effect.
24	HET_PIN_ENA	0 1	Enables the output buffers of the pin structures depending on the value of nDIS and DIR.x when PINDIS.x is set. Note: This bit will automatically get cleared when nDIS pin (input port) value is 0. No affect on the pin output buffer structure. Enables the pin output buffer structure when DIR = output, PINDIS.x is set and nDIS = 1.
23	Reserved	0	Reads return 0. Writes have no effect.
22-21	MP	0 1h 2h 3h	Master Priority The NHET can prioritize master accesses to N2HET RAM between the HET Transfer Unit and another arbiter, which outputs the access of one of the remaining masters. The MP bits allow the following selections: The HTU has lower priority to access the N2HET RAM than the arbiter output. The HTU has higher priority to access the N2HET RAM than the arbiter output. The HTU and the arbiter output use a round robin scheme to access the N2HET RAM. Reserved
20-19	Reserved	0	Reads return 0. Writes have no effect.
18	PPF	0 1	Protect Program Fields The PPF bit together with the Turn On/Off bit (TO) allows to protect the program fields of all instructions in N2HET RAM. When TO = 0: All masters can read and write the program fields. All masters can read and write the program fields.
		0 1	When TO = 1: All masters can read and write the program fields. The program fields are readable but not writable for all masters, which could access the N2HET RAM. Possible masters are the CPU, HTU, DMA and a secondary CPU (if available). Writes initiated by these masters are discarded.
17	IS	0 1	Ignore Suspend When Ignore Suspend = 0, the timer operation is stopped on suspend (the current timer instruction is completed). Timer RAM can be freely accessed during suspend. When set to 1, the suspend is ignored and the N2HET continues operating. N2HET stops when in suspend mode. N2HET ignores suspend mode and continues operation.

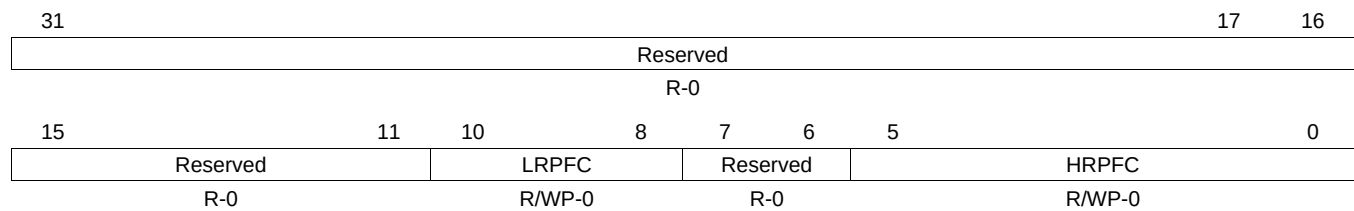
Table 23-16. Global Configuration Register (HETGCR) Field Descriptions (continued)

Bit	Field	Value	Description
16	CMS		Clk_master/slave This bit is used to synchronize multi-N2HETs. If set (N2HET is master), the N2HET outputs a signal to synchronize the prescalers of the slave N2HET. By default, this bit is reset, which means a slave configuration. Note: This bit must be set to one (1) for single-N2HET configuration.
		0	N2HET is configured as a slave.
		1	N2HET is configured as a master.
15-1	Reserved	0	Reads return 0. Writes have no effect.
0	TO		Turn On/Off TO does not affect the state of the pins. You must set/reset the timer pins when they are turned off, or re-initialize the timer RAM and control registers before a reset. After a device reset, the timer is turned off by default.
		0	N2HET is OFF. The timer program stops executing. Turn-off is automatically delayed until the current timer program loop is completed. Turn-off does not affect the content of the timer RAM, ALU registers, or control registers. Turn-off resets all flags.
		1	N2HET is ON. The timer program execution starts synchronously to the Loop clock. In case of multiple N2HETs configuration, the slave N2HETs are waiting for the loop clock to come from the master before starting execution. Then, the timer address points automatically address 00h (corresponding to program start).

23.4.2 Prescale Factor Register (HETPFR)

N2HET1: offset = FFF7 B804h; **N2HET2:** offset = FFF7 B904h

Figure 23-57. Prescale Factor Register (HETPFR)



LEGEND: R/W = Read/Write; R = Read only; WP = Write in privileged mode only; -n = value after reset

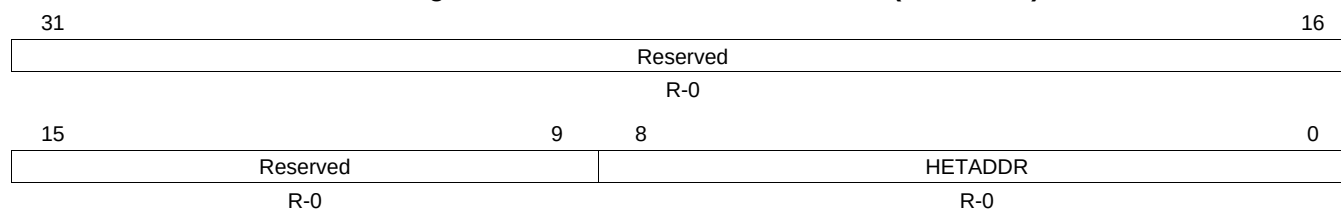
Table 23-17. Prescale Factor Register (HETPFR) Field Descriptions

Bit	Field	Value	Description
31-11	Reserved	0	Reads return 0. Writes have no effect.
10-8	LRPFC	0 1h 2h 3h 4h 5h 6h 7h	Loop-Resolution Pre-scale Factor Code. LRPFC determines the loop-resolution prescale divide rate (lr). /1 /2 /4 /8 /16 /32 /64 /128
7-6	Reserved	0	Reads return 0. Writes have no effect.
5-0	HRPFC	0 1h 2h 3h : 3Dh 3Eh 3Fh	High-Resolution Pre-scale Factor Code. HRPFC determines the high-resolution prescale divide rate (hr). /1 /2 /3 /4 : /62 /63 /64

23.4.3 N2HET Current Address Register (HETADDR)

N2HET1: offset = FFF7 B808h; **N2HET2:** offset = FFF7 B908h

Figure 23-58. N2HET Current Address (HETADDR)



LEGEND: R = Read only; -n = value after reset

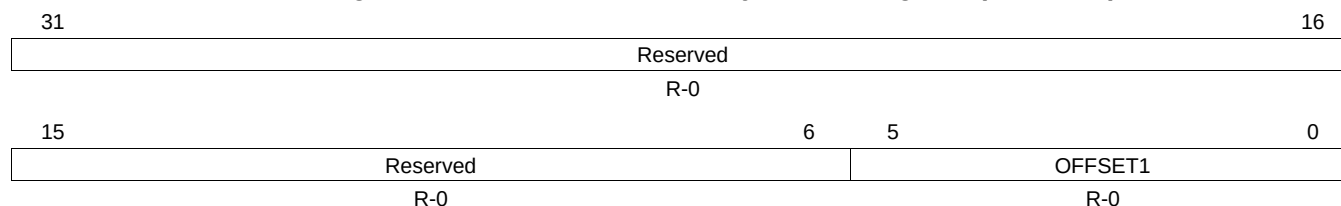
Table 23-18. N2HET Current Address (HETADDR) Field Descriptions

Bit	Field	Value	Description
31-9	Reserved	0	Reads return 0. Writes have no effect.
8-0	HETADDR		N2HET Current Address Read: Returns the current N2HET program address. Write: Writes have no effect.

23.4.4 Offset Index Priority Level 1 Register (HETOFF1)

N2HET1: offset = FFF7 B80Ch; **N2HET2:** offset = FFF7 B90Ch

Figure 23-59. Offset Index Priority Level 1 Register (HETOFF1)



LEGEND: R = Read only; -n = value after reset

Table 23-19. Offset Index Priority Level 1 Register (HETOFF1) Field Descriptions

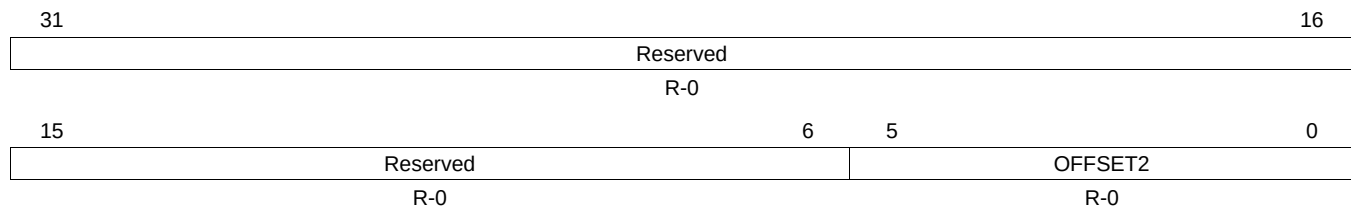
Bit	Field	Value	Description
31-6	Reserved	0	Reads return 0. Writes have no effect.
5-0	OFFSET1		OFFSET1 indexes the currently pending high-priority interrupt. Offset values and sources are listed in Table 23-20 . Read: Read of these bits determines the pending N2HET interrupt. Write: Writes have no effect. Note: In any read operation mode, the corresponding flag (in the HETFLG) is also cleared. In Emulation mode the corresponding flag is not cleared.

Table 23-20. Interrupt Offset Encoding Format

Offset Value	Source No.
0	No interrupt
1	Instruction 0, 32, 64...
2	Instruction 1, 33, 65...
:	:
32	Instruction 31, 63, 95...
33	Program Overflow
34	APCNT Underflow
35	APCNT Overflow

23.4.5 Offset Index Priority Level 2 Register (HETOFF2)

N2HET1: offset = FFF7 B810h; N2HET2: offset = FFF7 B910h

Figure 23-60. Offset Index Priority Level 2 Register (HETOFF2)


LEGEND: R = Read only; -n = value after reset

Table 23-21. Offset Index Priority Level 2 Register (HETOFF2) Field Descriptions

Bit	Field	Value	Description
31-6	Reserved	0	Reads return 0. Writes have no effect.
5-0	OFFSET2		OFFSET2 indexes the currently pending low-priority interrupt. Offset values and sources are listed in Table 23-20. Read: Read of these bits determines the pending N2HET interrupt. Write: Writes have no effect. Note: In any read operation mode, the corresponding flag (in the HETFLG) is also cleared. In Emulation mode, the corresponding flag is not cleared.

23.4.6 Interrupt Enable Set Register (HETINTENAS)

N2HET1: offset = FFF7 B814h; **N2HET2:** offset = FFF7 B914h

Figure 23-61. Interrupt Enable Set Register (HETINTENAS)

31	16
HETINTENAS	
R/W-0	
15	0
HETINTENAS	
R/W-0	

LEGEND: R/W = Read/Write; -n = value after reset

Table 23-22. Interrupt Enable Set Register (HETINTENAS) Field Descriptions

Bit	Field	Value	Description
31-0	HETINTENAS[n]		Interrupt Enable Set bits. HETINTENAS is readable and writable in any operation mode. Writing a 1 to bit x enables the interrupts of the N2HET instructions at N2HET addresses x+0, x+32, x+64, and so on. Generating an interrupt requires to set bit x in HETINTENAS and to enable the interrupt bit in one of the instructions at addresses x+0, x+32, x+64, and so on. To avoid ambiguity, only one of the instructions x+0, x+32, x+64, and so on, should have the interrupt enable bit (inside the instruction) set. Writing a 0 to HETINTENAS has no effect. When reading from HETINTENAS bit x gives the information, if N2HET instructions x+0, x+32, x+64, and so on, have the interrupt enabled or disabled.
		0	Read: Interrupt is disabled. Write: Writes have no effect.
		1	Read: Interrupt is enabled. Write: Interrupt is enabled.

23.4.7 Interrupt Enable Clear Register (HETINTENAC)

N2HET1: offset = FFF7 B818h; **N2HET2:** offset = FFF7 B918h

Figure 23-62. Interrupt Enable Clear (HETINTENAC)

31	16
HETINTENAC	
R/W-0	
15	0
HETINTENAC	
R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-23. NHET Interrupt Enable Clear (HETINTENAC) Field Descriptions

Bit	Field	Value	Description
31-0	HETINTENAC[n]		Interrupt Enable Clear bits. HETINTENAC is readable and writable in any operation mode. Writing a 1 to bit x disables the interrupts of the N2HET instructions at N2HET addresses x+0, x+32, x+64, and so on. (See also description in Table 23-22). Writing a 0 to HETINTENAC has no effect. When reading from HETINTENAC bit x gives the information, if N2HET instructions x+0, x+32, x+64, and so on, have the interrupt enabled or disabled.
		0	Read: Interrupt is disabled. Write: Writes have no effect.
		1	Read: Interrupt is enabled. Write: Interrupt is disabled.

23.4.8 Exception Control Register 1 (HETEXC1)

N2HET1: offset = FFF7 B81Ch; **N2HET2:** offset = FFF7 B91Ch

Figure 23-63. Exception Control Register (HETEXC1)

31	25			24
Reserved			APCNT_OVRFL_ENA	
R-0			R/W-0	
23	17			16
Reserved			APCNT_UNRFL_ENA	
R-0			R/W-0	
15	9			8
Reserved			PRGM_OVRFL_ENA	
R-0			R/W-0	
7	3	2	1	0
Reserved		APCNT_OVRFL_PRY	APCNT_UNRFL_PRY	PRGM_OVRFL_PRY
R-0		R/W-0	R/W-0	R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

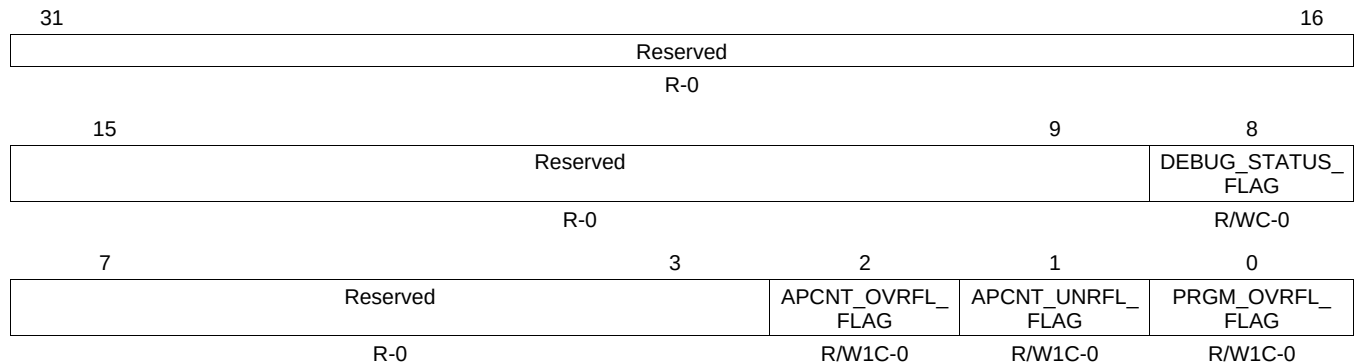
Table 23-24. Exception Control Register 1 (HETEXC1) Field Descriptions

Bit	Field	Value	Description
31-17	Reserved	0	Reads return 0. Writes have no effect.
24	APCNT_OVRFL_ENA	0 1	APCNT Overflow Enable APCNT overflow exception is not enabled. Enables the APCNT overflow exception.
23-17	Reserved	0	Reads return 0. Writes have no effect.
16	APCNT_UNRFL_ENA	0 1	APCNT Underflow Enable APCNT underflow exception is not enabled. Enables the APCNT underflow exception.
15-9	Reserved	0	Reads return 0. Writes have no effect.
8	PRGM_OVRFL_ENA	0 1	Program Overflow Enable The program overflow exception is not enabled. Enables the program overflow exception.
7-3	Reserved	0	Reads return 0. Writes have no effect.
2	APCNT_OVRFL_PRY	0 1	APCNT Overflow Exception Interrupt Priority Exception priority level 2. Exception priority level 1.
1	APCNT_UNRFL_PRY	0 1	APCNT Underflow Exception Interrupt Priority Exception priority level 2. Exception priority level 1.
0	PRGM_OVRFL_PRY	0 1	ProgramOverflow Exception Interrupt Priority Exception priority level 2. Exception priority level 1.

23.4.9 Exception Control Register 2 (HETEXC2)

N2HET1: offset = FFF7 B820h; **N2HET2:** offset = FFF7 B920h

Figure 23-64. Exception Control Register 2 (HETEXC2)



LEGEND: R/W = Read/Write; R = Read only; W1C = Write 1 to clear; -n = value after reset

Table 23-25. Exception Control Register 2 (HETEXC2) Field Descriptions

Bit	Field	Value	Description
31-9	Reserved	0	Reads return 0. Writes have no effect.
8	DEBUG_STATUS_FLAG	0	Debug Status Flag. This flag is set when N2HET has stopped at a breakpoint. Also generates a debug request to halt the ARM CPU. Read: N2HET is either running, or stopped, flag cleared but not yet restarted. Write: No effect.
		1	Read: N2HET is stopped at a breakpoint. Write: Clears the bit. To restart N2HET clear this bit and then restart the ARM CPU. The N2HET and ARM CPU will start synchronously.
7-3	Reserved	0	Reads return 0. Writes have no effect.
2	APCNT_OVRFL_FLAG	0	APCNT Overflow Flag Read: Exception has not occurred since the flag was cleared. Write: No effect.
		1	Read: Exception has occurred since the flag was cleared. Write: Clears the bit.
1	APCNT_UNDFL_FLAG	0	APCNT Underflow Flag Read: Exception has not occurred since the flag was cleared. Write: No effect.
		1	Read: Exception has occurred since the flag was cleared. Write: Clears the bit.
0	PRGM_OVERFL_FLAG	0	Program Overflow Flag Read: Exception has not occurred since the flag was cleared. Write: No effect.
		1	Read: Exception has occurred since the flag was cleared Write: Clears the bit.

23.4.12 AND Share Control Register (HETAND)

N2HET1: offset = FFF7 B82Ch; **N2HET2:** offset = FFF7 B92Ch

Figure 23-67. AND Share Control Register (HETAND)

Reserved															
R-0															
15	14	13	12	11	10	9	8								
AND SHARE31/30	AND SHARE29/28	AND SHARE27/26	AND SHARE25/24	AND SHARE23/22	AND SHARE21/20	AND SHARE19/18	AND SHARE17/16								
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0								
7	6	5	4	3	2	1	0								
AND SHARE15/14	AND SHARE13/12	AND SHARE11/10	AND SHARE9/8	AND SHARE7/6	AND SHARE5/4	AND SHARE3/2	AND SHARE1/0								
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0								

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-28. AND Share Control Register (HETAND) Field Descriptions

Bit	Field	Value	Description
31-16	Reserved	0	Reads return 0. Writes have no effect.
15-0	ANDSHARE n+1 / n		AND Share Enable Enable the AND sharing of the same pin for two HR structures. For example, if bit ANDSHARE1/0 is set, the pin HET[0] will then be commanded by a logical AND of both HR structures 0 and 1. Note: If HR AND SHARE bits are used, pins not connected to HR structures (the odd number pin in each pair) can be accessed as general inputs/outputs. 0 HR Output of HET[n+1] and HET[n] are not AND shared. 1 HR Output of HET[n+1] and HET[n] are AND shared onto pin HET[n].

23.4.13 HR Share Control Register (HETHRSH)

N2HET1: offset = FFF7 B834h; **N2HET2:** offset = FFF7 B934h

Figure 23-68. HR Share Control Register (HETHRSH)

31								16							
Reserved															
R-0															
15		14		13		12		11		10		9		8	
HR SHARE31/30		HR SHARE29/28		HR SHARE27/26		HR SHARE25/24		HR SHARE23/22		HR SHARE21/20		HR SHARE19/18		HR SHARE17/16	
R/W-0		R/W-0		R/W-0		R/W-0		R/W-0		R/W-0		R/W-0		R/W-0	
7		6		5		4		3		2		1		0	
HR SHARE15/14		HR SHARE13/12		HR SHARE11/10		HR SHARE9/8		HR SHARE7/6		HR SHARE5/4		HR SHARE3/2		HR SHARE1/0	
R/W-0		R/W-0		R/W-0		R/W-0		R/W-0		R/W-0		R/W-0		R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-29. HR Share Control Register (HETHRSH) Field Descriptions

Bit	Field	Value	Description
31-16	Reserved	0	Reads return 0. Writes have no effect.
15-0	HRSHARE _{n+1 / n}	0 1	HR Share Bits Enables the share of the same pin for two HR structures. For example, if bit HRSHARE1/0 is set, the pin HET[0] will then be connected to both HR input structures 0 and 1. Note: If HR share bits are used, pins not connected to HR structures (the odd number pin in each pair) can be accessed as general inputs/outputs. 0 HR Input of HET[n+1] and HET[n] are not shared. 1 HR Input of HET[n+1] and HET[n] are shared; both measure pin HET[n].

23.4.14 XOR Share Control Register (HETXOR)

N2HET1: offset = FFF7 B838h; **N2HET2:** offset = FFF7 B938h

Figure 23-69. XOR Share Control Register (HETXOR)

Reserved															
R-0															
15	14	13	12	11	10	9	8								
XOR SHARE31/30	XOR SHARE29/28	XOR SHARE27/26	XOR SHARE25/24	XOR SHARE23/22	XOR SHARE21/20	XOR SHARE19/18	XOR SHARE17/16								
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0								
7	6	5	4	3	2	1	0								
XOR SHARE15/14	XOR SHARE13/12	XOR SHARE11/10	XOR SHARE9/8	XOR SHARE7/6	XOR SHARE5/4	XOR SHARE3/2	XOR SHARE1/0								
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0								

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-30. XOR Share Control Register (HETXOR) Field Descriptions

Bit	Field	Value	Description
31-16	Reserved	0	Reads return 0. Writes have no effect.
15-0	XORSHARE n+1 / n		XOR Share Enable Enable the XOR-share of the same pin for two output HR structures. For example, if bit XORSHARE1/0 is set, the pin HET[0] will then be commanded by a logical XOR of both HR structures 0 and 1. Note: If XOR share bits are used, pins not connected to HR structures (the odd number pin in each pair) can be accessed as general inputs/outputs. 0 HR Output of HET[n+1] and HET[n] are not XOR shared. 1 HR Output of HET[n+1] and HET[n] are XOR shared onto pin HET[n].

23.4.15 Request Enable Set Register (HETREQENS)

N2HET1: offset = FFF7 B83Ch; **N2HET2:** offset = FFF7 B93Ch

Figure 23-70. Request Enable Set Register (HETREQENS)

31																8															
Reserved																															
R-0																															
7				6				5				4				3				2				1				0			
REQENA7				REQENA6				REQENA5				REQENA4				REQENA3				REQENA2				REQENA1				REQENA0			
R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0			

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-31. Request Enable Set Register (HETREQENS) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	REQENAn		Request Enable Bits
		0	Read: Returns the information that request line n is disabled. Write: Writing a 0 has no effect.
		1	Read: Returns the information that request line n is enabled. Write: Writing a 1 to bit n enables the N2HET request line n. Note: The request line can trigger a DMA control packet (DMA channel), an HTU double control packet (DCP) or both simultaneously. The HETREQDS register determines to which module(s) the N2HET request line n is assigned. Note: A disabled request line does not memorize old requests. So there are no pending requests to service after enabling request line n.

23.4.16 Request Enable Clear Register (HETREQENC)

N2HET1: offset = FFF7 B840h; **N2HET2:** offset = FFF7 B940h

Figure 23-71. Request Enable Clear Register (HETREQENC)

31							8								
Reserved															
R-0															
7		6		5		4		3		2		1		0	
REQDIS7		REQDIS6		REQDIS5		REQDIS4		REQDIS3		REQDIS2		REQDIS1		REQDIS0	
R/W-0		R/W-0		R/W-0		R/W-0		R/W-0		R/W-0		R/W-0		R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-32. Request Enable Clear Register (HETREQENC) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	REQDISn		Request Disable Bits
		0	Read: Returns the information that request line n is disabled. Write: Writing a 0 has no effect.
		1	Read: Returns the information that request line n is enabled. Write: Writing a 1 to bit n disables the N2HET request line n.

23.4.17 Request Destination Select Register (HETREQDS)

N2HET1: offset = FFF7 B844h; **N2HET2:** offset = FFF7 B944h

Figure 23-72. Request Destination Select Register (HETREQDS) [offset = FFF7 B844h]

31		24	23	22	21	20	19	18	17	16
Reserved			TDBS7	TDBS6	TDBS5	TDBS4	TDBS3	TDBS2	TDBS1	TDBS0
R-0			R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
15		8	7	6	5	4	3	2	1	0
Reserved			TDS7	TDS6	TDS5	TDS4	TDS3	TDS2	TDS1	TDS0
R-0			R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-33. Request Destination Select Register (HETREQDS) Field Descriptions

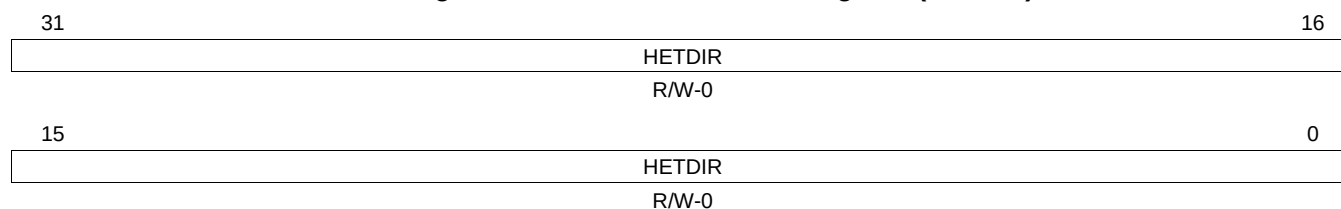
Bit	Field	Value	Description
31-24	Reserved	0	Reads return 0. Writes have no effect.
23-16	TDBSn	0	HTU, DMA or Both Select Bits N2HET request line n is assigned to the module specified by TDS bit n.
		1	N2HET request line n is assigned to both DMA and HTU. TDS bit n is ignored in this case.
15-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	TDSn		HTU or DMA Select Bits Note: It must be ensured in the N2HET program, that one request line is triggered by only one N2HET instruction.
		0	N2HET request line n is assigned to HTU (TDBS bit n is zero).
		1	N2HET request line n is assigned to DMA (TDBS bit n is zero).

NOTE: Please refer to the device data sheet how each of the 8 N2HET request lines are connected to these modules. See also [Section 23.2.9](#).

23.4.18 NHET Direction Register (HETDIR)

N2HET1: offset = FFF7 B84Ch; **N2HET2:** offset = FFF7 B94Ch

Figure 23-73. N2HET Direction Register (HETDIR)



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-34. N2HET Direction Register (HETDIR) Field Descriptions

Bit	Field	Value	Description
31-0	HETDIR[n]	0	Data direction of NHET pins Pin HET[n] is an input (and its output buffer is tristated).
		1	Pin HET[n] is an output.

NOTE: [Table 23-9](#) shows how the register bits of DIR, PULDIS and PULSEL are affecting the N2HET pins.

23.4.19 N2HET Data Input Register (HETDIN)

N2HET1: offset = FFF7 B850h; **N2HET2:** offset = FFF7 B950h

Figure 23-74. N2HET Data Input Register (HETDIN)

31	16
HETDIN	
R-x	
15	0
HETDIN	
R-x	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset;

Table 23-35. N2HET Data Input Register (HETDIN) Field Descriptions

Bit	Field	Value	Description
31-0	HETDIN[n]	0	Data input. This bit displays the logic state of the pin. Pin HET[n] is at logic low (0).
		1	Pin HET[n] is at logic high (1).

23.4.20 N2HET Data Output Register (HETDOUT)

N2HET1: offset = FFF7 B854h; **N2HET2:** offset = FFF7 B954h

Figure 23-75. N2HET Data Output Register (HETDOUT)

31	16
HETDOUT	
R/W-0	
15	0
HETDOUT	
R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-36. N2HET Data Output Register (HETDOUT) Field Descriptions

Bit	Field	Value	Description
31-0	HETDOUT[n]	0	Data out write. Writes to this bit will only take effect when the pin is configured as an output. The current logic state of the pin will be displayed by this bit even when the pin state is changed by writing to HETDSET or HETDCLR. Pin HET[n] is at logic low (0).
		1	Pin HET[n] is at logic high (1) if the HETPDR[n] bit = 0 or the output is in high-impedance state if the HETPDR[n] bit = 1.

23.4.21 NHET Data Set Register (HETDSET)

N2HET1: offset = FFF7 B858h; **N2HET2:** offset = FFF7 B958h

Figure 23-76. N2HET Data Set Register (HETDSET)

31	16
HETDSET	
R/WS-0	
15	0
HETDSET	
R/WS-0	

LEGEND: R/W = Read/Write; R = Read only; S = Set; -n = value after reset

Table 23-37. N2HET Data Set Register (HETDSET) Field Descriptions

Bit	Field	Value	Description
31-0	HETDSET[n]		This register allows bits of HETDOUT to be set while avoiding the pitfalls of a read-modify-write sequence in a multitasking environment.
		0	Write: HETDOUT[n] is unchanged.
		1	Write: HETDOUT[n] is set.

23.4.22 N2HET Data Clear Register (HETDCLR)

N2HET1: offset = FFF7 B85Ch; **N2HET2:** offset = FFF7 B95Ch

Figure 23-77. N2HET Data Clear Register (HETDCLR)

31	16
HETDCLR	
R/WC-0	
15	0
HETDCLR	
R/WC-0	

LEGEND: R/W = Read/Write; R = Read only; C = Clear; -n = value after reset

Table 23-38. N2HET Data Clear Register (HETDCLR) Field Descriptions

Bit	Field	Value	Description
31-0	HETDCLR[n]		This register allows bits of HETDOUT to be cleared while avoiding the pitfalls of a read-modify-write sequence in a multitasking environment.
		0	Write: HETDOUT[n] is unchanged.
		1	Write: HETDOUT[n] is cleared.

23.4.23 N2HET Open Drain Register (HETPDR)

Values in this register enable or disable the open drain capability of the data pins.

N2HET1: offset = FFF7 B860h; **N2HET2:** offset = FFF7 B960h

Figure 23-78. N2HET Open Drain Register (HETPDR)

31	16
HETPDR	
R/W-0	
15	0
HETPDR	
R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-39. N2HET Open Drain Register (HETPDR) Field Descriptions

Bit	Field	Value	Description
31-0	HETPDR[n]	0	Open drain control for HET[n] pins The pin is configured in push/pull mode.
		1	The pin is configured in open drain mode. The HETDOUT register controls the state of the output buffer: HETDOUT[n] = 0 The output buffer of pin HET[n] is driven low. HETDOUT[n] = 1 The output buffer of pin HET[n] is tristated.

23.4.24 N2HET Pull Disable Register (HETPULDIS)

Values in this register enable or disable the pull-up/-down functionality of the pins.

N2HET1: offset = FFF7 B864h; **N2HET2:** offset = FFF7 B964h

Figure 23-79. N2HET Pull Disable Register (HETPULDIS)

31	16
HETPULDIS	
R/W-n	
15	0
HETPULDIS	
R/W-n	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset; n is device dependent, see device specific data sheet

Table 23-40. N2HET Pull Disable Register (HETPULDIS) Field Descriptions

Bit	Field	Value	Description
31-0	HETPULDIS[n]	0	Pull disable for N2HET pins The pull functionality is enabled on pin HET[n].
		1	The pull functionality is disabled on pin HET[n].

NOTE: See device data sheet for which pins provide programmable pullups/pulldowns.

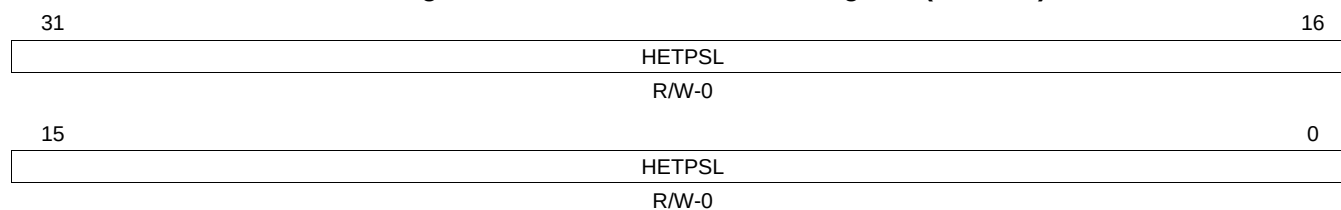
[Table 23-9](#) shows how the register bits of HETDIR, HETPULDIS, and HETPSL are affecting the N2HET pins.

23.4.25 N2HET Pull Select Register (HETPSL)

Values in this register select the pull-up or pull-down functionality of the pins.

N2HET1: offset = FFF7 B868h; **N2HET2:** offset = FFF7 B968h

Figure 23-80. N2HET Pull Select Register (HETPSL)



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-41. N2HET Pull Select Register (HETPSL) Field Descriptions

Bit	Field	Value	Description
31-0	HETPSL[n]	0	Pull select for NHET pins The pull down functionality is enabled if corresponding bit in HETPULDIS is 0.
		1	The pull up functionality is enabled if corresponding bit in HETPULDIS is 0.

NOTE: See device data sheet for which pins provide programmable pullups/pulldowns.

[Table 23-9](#) shows how the register bits of HETDIR, HETPULDIS and HETPSL are affecting the N2HET pins.

The information of this register is also used to define the pin states after a parity error:

After a parity error all N2HET pins, which are

1. Defined as output pins in the HETDIR register
2. Not defined as open drain pins (with the HETPDR register)
3. Selected with the HETPPR register, will remain outputs, but automatically change their levels in the following way:
 - If the HETPSL register specifies 0 for the pin, it will switch to low level.
 - If the HETPSL register specifies 1 for the pin, it will switch to high level.

This behavior is independent of the value, which register HETPULDIS specifies for the corresponding pin.

23.4.26 Parity Control Register (HETPCR)

N2HET1: offset = FFF7 B874h; **N2HET2:** offset = FFF7 B974h

Figure 23-81. Parity Control Register (HETPCR)

31																16
Reserved																
R-0																
15								9	8	7				4	3	0
Reserved								TEST		Reserved					PARITY_ENA	
R-0								R/WP-0		R-0					R/WP-5h	

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privileged mode only; -n = value after reset

Table 23-42. Parity Control Register (HETPCR) Field Descriptions

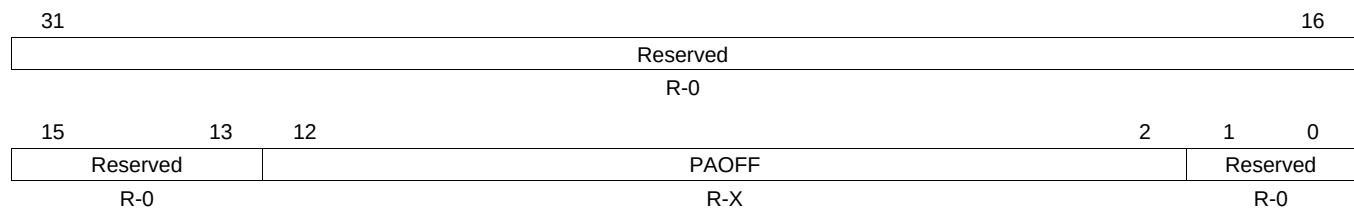
Bit	Field	Value	Description
31-9	Reserved	0	Reads return 0. Writes have no effect.
8	TEST	0	Test Bit. When this bit is set, the parity bits are mapped into the peripheral RAM frame to make them accessible by the CPU. Read: Parity bits are not memory mapped. Write: Disable mapping.
		1	Read: Parity bits are memory mapped. Write: Enable mapping.
7-4	Reserved	0	Reads return 0. Writes have no effect.
3-0	PARITY_ENA	5h	Enable/disable parity checking. This bit field enables or disables the parity check on read operations and the parity calculation on write operations. If parity checking is enabled and a parity error is detected the N2HET_UERR signal is activated. Read: Parity check is disabled. Write: Disable checking.
		Others	Read: Parity check is enabled. Write: Enable checking.

NOTE: It is recommended to write Ah to enable error detection, to guard against soft errors flipping PARITY_ENA to a disable state.

23.4.27 Parity Address Register (HETPAR)

N2HET1: offset = FFF7 B878h; **N2HET2:** offset = FFF7 B978h

Figure 23-82. Parity Address Register (HETPAR)



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset; X = Value unchanged after reset

Table 23-43. Parity Address Register (HETPAR) Field Descriptions

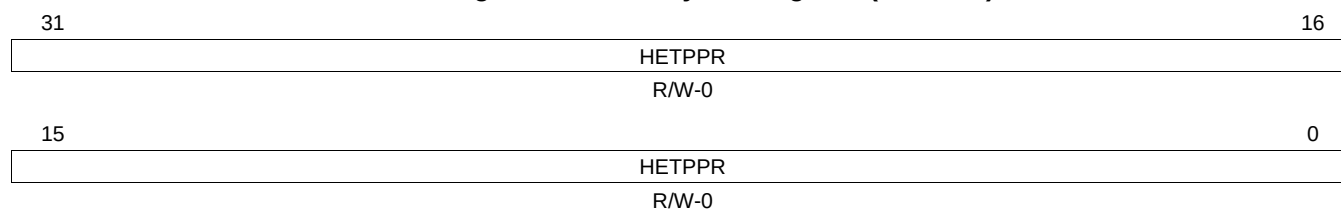
Bit	Field	Value	Description
31-13	Reserved	0	Reads return 0. Writes have no effect.
12-2	PAOFF		Parity Error Address Offset. This register holds the offset address of the first parity error, which is detected in N2HET RAM. This error address is frozen from being updated until it is read by the CPU. During emulation mode, this address is frozen even when read. In case of a N2HET RAM parity error, PAOFF will contain the offset address of the erroneous 32-bit N2HET RAM field counted from the beginning of the N2HET RAM. Examples: The 32-bit program field of instruction 0 will return 0, the 32-bit control field of instruction 0 will return 1, ..., the 32-bit control field of instruction 1 will return 5, and so on. Read: Returns the offset address of the erroneous 32-bit word in bytes from the beginning of the N2HET RAM. Write: Writes have no effect.
1-0	Reserved	0	Reads return 0. Writes have no effect.

NOTE: The Parity Error Address Register will not be reset, neither by PORRST nor by any other reset source.

23.4.28 Parity Pin Register (HETPPR)

N2HET1: offset = FFF7 B87Ch; **N2HET2:** offset = FFF7 B97Ch

Figure 23-83. Parity Pin Register (HETPPR)



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-44. Parity Pin Register (HETPPR) Field Descriptions

Bit	Field	Value	Description
31-0	HETPPR[n]	0	NHET Parity Pin Select Bits. Allows HET[n] pins to be configured to drive to a known state when an N2HET parity error is detected.
		1	Pin HET[n] is not affected by the detection of an N2HET parity error.
		1	Pin HET[n] is driven to a known state when an N2HET parity error is detected. The known state is a function of bits HETDIR[n], HETPSL[n], HETPDR[n] as described in Table 23-45 (this state is also independent of HETPULDIS[n]).

Table 23-45. Known State on Parity Error

HETDIR[n]	HETPDR[n]	HETPSL[n]	Known State on Parity Error
0	x	x	High Impedance
1	0	0	Drive Logic 0
1	0	1	Drive Logic 1
1	1	x	High Impedance

23.4.29 Suppression Filter Preload Register (HETSFPRLD)

N2HET1: offset = FFF7 B880h; **N2HET2:** offset = FFF7 B980h

Figure 23-84. Suppression Filter Preload Register (HETSFPRLD)

31	18	17	16
Reserved			CCDIV
R-0			R/W-0
15	10	9	0
Reserved		CPRLD	
R-0		R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-46. Suppression Filter Preload Register (HETSFPRLD) Field Descriptions

Bit	Field	Value	Description
31-18	Reserved	0	Reads return 0. Writes have no effect.
17-16	CCDIV	0 1h 2h 3h	Counter Clock Divider CCDIV determines the ratio between the counter clock and VCLK2. CCLK = VCLK2 CCLK = VCLK2 / 2 CCLK = VCLK2 / 3 CCLK = VCLK2 / 4
15-10	Reserved	0	Reads return 0. Writes have no effect.
9-0	CPRLD		Counter Preload Value CPRLD contains the preload value for the counter clock.

23.4.30 Suppression Filter Enable Register (HETSFENA)

N2HET1: offset = FFF7 B884h; **N2HET2:** offset = FFF7 B984h

Figure 23-85. Suppression Filter Enable Register (HETSFENA)

31	16
HETSFENA	
R/W-0	
15	0
HETSFENA	
R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-47. Suppression Filter Enable Register (HETSFENA) Field Descriptions

Bit	Field	Value	Description
31-0	HETSFENA[n]	0 1	Suppression Filter Enable Bits Note: If the pin is configured as an output by the N2HET Direction Register (HETDIR), the filter is automatically disabled independent on the bit in HETSFENA. The input noise suppression filter for pin HET[n] is disabled. The input noise suppression filter for pin HET[n] is enabled.

23.4.31 Loop Back Pair Select Register (HETLBPSEL)

Refer to [Section 23.2.5.7](#) for a description of loopback test functions.

N2HET1: offset = FFF7 B88Ch; **N2HET2:** offset = FFF7 B98Ch

Figure 23-86. Loop Back Pair Select Register (HETLBPSEL)

31	30	29	28	27	26	25	24
LBPTYPE31/30	LBPTYPE29/28	LBPTYPE27/26	LBPTYPE25/24	LBPTYPE23/22	LBPTYPE21/20	LBPTYPE19/18	LBPTYPE17/16
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
23	22	21	20	19	18	17	16
LBPTYPE15/14	LBPTYPE13/12	LBPTYPE11/10	LBPTYPE9/8	LBPTYPE7/6	LBPTYPE5/4	LBPTYPE3/2	LBPTYPE1/0
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
15	14	13	12	11	10	9	8
LBPSEL31/30	LBPSEL29/28	LBPSEL27/26	LBPSEL25/24	LBPSEL23/22	LBPSEL21/20	LBPSEL19/18	LBPSEL17/16
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
7	6	5	4	3	2	1	0
LBPSEL15/14	LBPSEL13/12	LBPSEL11/10	LBPSEL9/8	LBPSEL7/6	LBPSEL5/4	LBPSEL3/2	LBPSEL1/0
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-48. Loop Back Pair Select Register (HETLBPSEL) Field Descriptions

Bit	Field	Value	Description
31-16	LBPTYPE n+1 / n	0 1	Loop Back Pair Type Select Bits These bits are valid only when Loopback mode is enabled (HETLBPDIR[19:16] = 1010). Digital loopback is selected for HR structures on pins HET[n+1] and HET[n]. Analog loopback is selected for HR structures on pins HET[n+1] and HET[n].
15-0	LBPSEL n+1 / n		Loop Back Pair Select Bits These bits are valid only when Loopback mode is enabled (HETLBPDIR[19:16] = 1010). If bit x is set, the HR structures on pins HET[n+1] and HET[n] are connected in a loop back mode. The direction is given by LBPDIR n+1/n and type is selected by LBPTYPE n+1/n. The pin which is not driven by the N2HET pin actions can still be used as normal GIO pin.

23.4.32 Loop Back Pair Direction Register (HETLBPDIR)

Refer to [Section 23.2.5.7](#) for a description of loopback test functions.

N2HET1: offset = FFF7 B890h; **N2HET2:** offset = FFF7 B990h

Figure 23-87. Loop Back Pair Direction Register (HETLBPDIR)

31																20				19		16									
Reserved																		LBPTSTENA													
R-0																R/WP-5h															
15				14				13				12				11				10				9				8			
LBPDIR31/30				LBPDIR29/28				LBPDIR27/26				LBPDIR25/24				LBPDIR23/22				LBPDIR21/20				LBPDIR19/18				LBPDIR17/16			
R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0			
7				6				5				4				3				2				1				0			
LBPDIR15/14				LBPDIR13/12				LBPDIR11/10				LBPDIR9/8				LBPDIR7/6				LBPDIR5/4				LBPDIR3/2				LBPDIR1/0			
R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0			

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privileged mode only; -n = value after reset

Table 23-49. Loop Back Pair Direction Register (HETLBPDIR) Field Descriptions

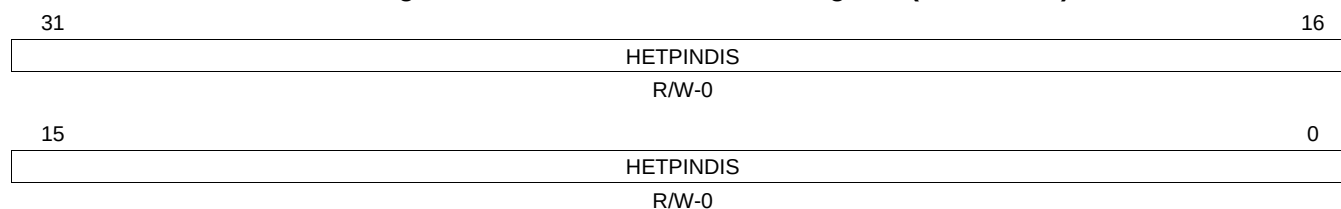
Bit	Field	Value	Description
31-20	Reserved	0	Reads return 0. Writes have no effect.
19-16	LBPTSTENA	5h Ah Others	Loopback Test Enable Key Loopback Test is disabled. Loopback Test is enabled. Loopback Test is disabled.
15-0	LBPDIR n+1 / n	0 1	Loop Back Pair Direction Bits The HR structures on pins HET[n+1] and HET[n] are internally connected with HET[n] as input and HET[n+1] as output. The HR structures on pins HET[n+1] and HET[n] connected with HET[n] as output and HET[n+1] as input.

NOTE: The loop back direction can be selected independent on the HETDIR register setting.

23.4.33 N2HET Pin Disable Register (HETPINDIS)

N2HET1: offset = FFF7 B894h; **N2HET2:** offset = FFF7 B994h

Figure 23-88. N2HET Pin Disable Register (HETPINDIS)



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-50. NHET Pin Disable Register (HETPINDIS) Field Descriptions

Bit	Field	Value	Description
31-0	HETPINDIS[n]	0	N2HET Pin Disable Bits
		1	Logic low: No affect on the output buffer enable of the pin (is controlled by the value of the HETDIR[n] bit).
		1	Logic high: Output buffer of the pin is enabled if pin nDIS = 1, HET_PIN_ENA = 1, and HETDIR = 1; or disabled if nDIS = 0, HETDIR = 0, or HET_PIN_ENA = 0.

23.5 HWAG Registers

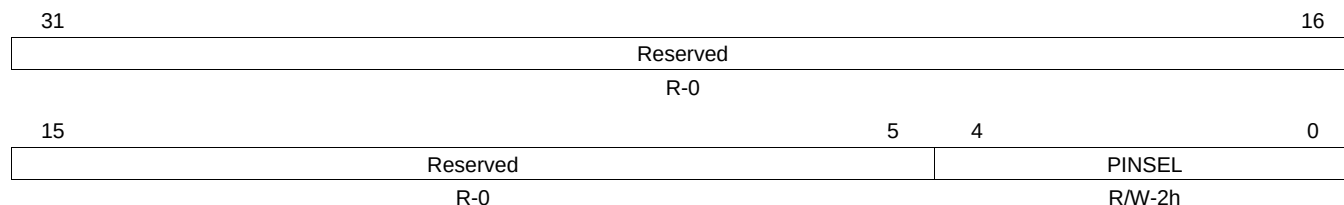
[Table 23-51](#) lists the HWAG registers.

Table 23-51. HWAG Registers

Offset	Acronym	Register Description	Section
9Ch	HWAPINSEL	HWAG Pin Select Register	Section 23.5.1
A0h	HWAGCR0	HWAG Global Control Register 0	Section 23.5.2
A4h	HWAGCR1	HWAG Global Control Register 1	Section 23.5.3
A8h	HWAGCR2	HWAG Global Control Register 2	Section 23.5.4
ACH	HWAENASET	HWAG Interrupt Enable Set Register	Section 23.5.5
B0h	HWAENACLR	HWAG Interrupt Enable Clear Register	Section 23.5.6
B4h	HWALVLSET	HWAG Interrupt Level Set Register	Section 23.5.7
B8h	HWALVLCLR	HWAG Interrupt Level Clear Register	Section 23.5.8
BCh	HWAFLG	HWAG Interrupt Flag Register	Section 23.5.9
C0h	HWAOFF0	HWAG Interrupt Offset Register 1	Section 23.5.10
C4h	HWAOFF1	HWAG Interrupt Offset Register 2	Section 23.5.11
C8h	HWAACNT	HWAG Angle Value Register	Section 23.5.12
CCh	HWAPCNT1	HWAG Previous Tooth Period Value Register	Section 23.5.13
D0h	HWAPCNT	HWAG Current Tooth Period Value Register	Section 23.5.14
D4h	HWASTWD	HWAG Step Width Register	Section 23.5.15
D8h	HWATHNB	HWAG Teeth Number Register	Section 23.5.16
DCh	HWATHVL	HWAG Current Teeth Number Register	Section 23.5.17
E0h	HWAFIL	HWAG Filter Register	Section 23.5.18
E8h	HWAFIL2	HWAG Filter Register 2	Section 23.5.19
F0h	HWAANGI	HWAG Angle Increment Register	Section 23.5.20

23.5.1 HWAG Pin Select Register (HWAPINSEL)

Figure 23-89. HWAG Pin Select Register (HWAPINSEL)



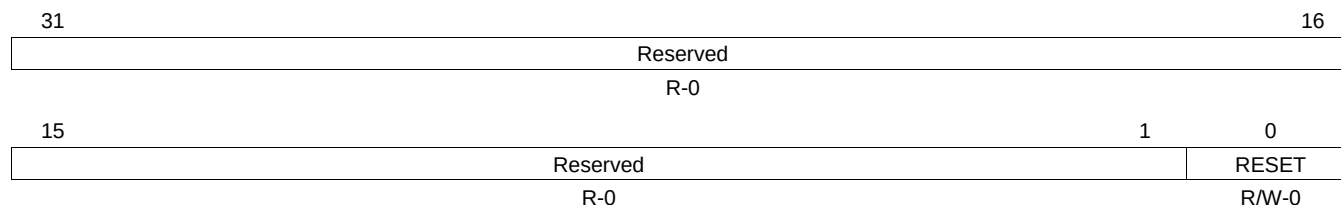
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-52. HWAG Pin Select Register (HWAPINSEL) Field Descriptions

Bit	Field	Value	Description
31-5	Reserved	0	Reads return 0. Writes have no effect.
4-0	PINSEL	0	HWAG Pin Select. Selects from which NHET pin input buffer the HWAG toothed-wheel signal is derived. Read: Pin HET[0] is selected. Write: Selects pin HET[0].
		1h	Read: Pin HET[1] is selected Write: Selects pin HET[1].
		2h	Read: Pin HET[2] is selected Write: Selects pin HET[2]. Default after reset for backwards compatibility
		: 1Fh	: Read: Pin HET[31] selected Write: Selects pin HET[31].

23.5.2 HWAG Global Control Register 0 (HWAGCR0)

Figure 23-90. HWAG Global Control Register 0 (HWAGCR0)



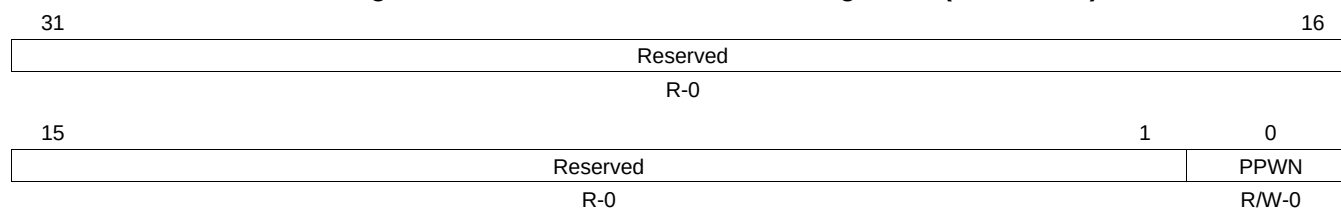
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-53. HWAG Global Control Register 0 (HWAGCR0) Field Descriptions

Bit	Field	Value	Description
31-1	Reserved	0	Reads return 0. Writes have no effect.
0	RESET	0	HWAG Module Reset.
		0	HWAG module is reset.
		1	HWAG module is not in reset.

23.5.3 HWAG Global Control Register 1 (HWAGCR1)

Figure 23-91. HWAG Global Control Register 1 (HWAGCR1)



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-54. HWAG Global Control Register 1 (HWAGCR1) Field Descriptions

Bit	Field	Value	Description
31-1	Reserved	0	Reads return 0. Writes have no effect.
0	PPWN	0	HWAG Module Power Down. This bit is implemented for legacy purposes, but has no functionality, however the HWAG module power down is controlled by the NHET power down. The HWAG cannot be powered down separately.

23.5.4 HWAG Global Control Register 2 (HWAGCR2)

Figure 23-92. HWAG Global Control Register 2 (HWAGCR2)

31	25	24	23	18	17	16
Reserved	ARST	Reserved	TED	CRI		
R-0	R/W-0	R-0	R/W-0	R/W-0		
15	9	8	7	1	0	
Reserved	FIL	Reserved	STRT			
R-0	R/W-0	R-0	R/W-0			

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-55. HWAG Global Control Register 2 (HWAGCR2) Field Descriptions

Bit	Field	Value	Description
31-25	Reserved	0	Reads return 0. Writes have no effect.
24	ARST	0 1	Angle Reset. This bit is used by the HWAG to validate the singularity when the hardware criteria is not used. The bit is cleared when the HWAG angle value register (HWAACNT) is cleared by the HWAG, when the last tooth edge occurs. If this bit is not set before the tooth edge during an singularity tooth, the HWAG generates an interruption "singularity not found", if the interrupt is enabled. 0 Do not reset ACNT once it reaches the angle zero point. 1 Reset ACNT once it reaches the angle zero point.
23-18	Reserved	0	Reads return 0. Writes have no effect.
17	TED	0 1	Tooth Edge. This bit is used to select which edge of the tooth wheel must be considered as active. 0 Falling edge 1 Rising edge
16	CRI	0 1	Criteria enable. This bits is used to control whether the criteria are applied. You could set your own criteria filter by disabling the hardwired criteria. 0 Criteria is disabled. 1 Criteria is enabled.
15-9	Reserved	0	Reads return 0. Writes have no effect.
8	FIL	0 1	Input Filter Enable. This bit is used to enable the toothed wheel input filter. 0 Filter is disabled. 1 Filter is enabled.
7-1	Reserved	0	Reads return 0. Writes have no effect.
0	STRT	0 1	Start bit. Put the HWAG into run time. Allows the HWAG to start counting ACNT, TCNT and criteria mechanism (if set). The HWAG starts at the next active edge from the toothed wheel, once set. If the start bit is cleared to 0, the HWAG is stopped immediately. 0 Do not start counting. 1 Start counting.

23.5.5 HWAG Interrupt Enable Set Register (HWAENASET)

Figure 23-93. HWAG Interrupt Enable Set Register (HWAENASET)

31																8															
Reserved																															
R-0																															
7				6				5				4				3				2				1				0			
SETINTENA7				SETINTENA6				SETINTENA5				SETINTENA4				SETINTENA3				SETINTENA2				SETINTENA1				SETINTENA0			
R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0			

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-56. HWAG Interrupt Enable Set Register (HWAENASET) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	SETINTENA[n]	0	Enable interrupt. See Table 23-57 . Read: Corresponding interrupt is not enabled. Write: No effect.
		1	Read: Corresponding interrupt is enabled. Write: Enable corresponding interrupt.

Table 23-57. HWAG Interrupts

Bit	Interrupt
0	Overflow period
1	Singularity not found
2	Tooth interrupt
3	ACNT overflow
4	PCNT(n) > 2 x PCNT (n-1) during normal tooth
5	Bad active edge tooth
6	Gap flag
7	Angle increment overflow

23.5.6 HWAG Interrupt Enable Clear Register (HWAENACLR)

Figure 23-94. HWAG Interrupt Enable Clear Register (HWAENACLR)

31	Reserved							8
R-0								
7	6	5	4	3	2	1	0	
CLRINTENA7	CLRINTENA6	CLRINTENA5	CLRINTENA4	CLRINTENA3	CLRINTENA2	CLRINTENA1	CLRINTENA0	
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-58. HWAG Interrupt Enable Clear Register (HWAENACLR) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	CLRINTENA[n]	0	Disable interrupt. See Table 23-57 . Read: Corresponding interrupt is not enabled. Write: No effect.
		1	Read: Corresponding interrupt is enabled. Write: Disable corresponding interrupt.

23.5.7 HWAG Interrupt Level Set Register (HWALVLSET)

Figure 23-95. HWAG Interrupt Level Set Register (HWALVLSET)

31																8															
Reserved																															
R-0																															
7				6				5				4				3				2				1				0			
SETINTLVL7				SETINTLVL6				SETINTLVL5				SETINTLVL4				SETINTLVL3				SETINTLVL2				SETINTLVL1				SETINTLVL0			
R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0			

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-59. HWAG Interrupt Level Set Register (HWALVLSET) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	SETINTLVL[n]		Set Interrupt Level. See Table 23-57 .
		0	Read: Low-priority interrupt. Write: No effect.
		1	Read: High-priority interrupt. Write: Set interrupt priority to high.

23.5.8 HWAG Interrupt Level Clear Register (HWALVLCLR)

Figure 23-96. HWAG Interrupt Level Clear Register (HWALVLCLR)

31																8															
Reserved																															
R-0																															
7				6				5				4				3				2				1				0			
CLRINTLVL7				CLRINTLVL6				CLRINTLVL5				CLRINTLVL4				CLRINTLVL3				CLRINTLVL2				CLRINTLVL1				CLRINTLVL0			
R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0				R/W-0			

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-60. HWAG Interrupt Level Clear Register (HWALVLCLR) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	CLRINTLVL[n]		Clear Interrupt Level. See Table 23-57 .
		0	Read: Low-priority interrupt. Write: No effect.
		1	Read: High-priority interrupt. Write: Set interrupt priority to low.

23.5.9 HWAG Interrupt Flag Register (HWAFLG)

Figure 23-97. HWAG Interrupt Flag Register (HWAFLG)

31																8															
Reserved																															
R-0																															
7				6				5				4				3				2				1				0			
INTFLG7				INTFLG6				INTFLG5				INTFLG4				INTFLG3				INTFLG2				INTFLG1				INTFLG0			
R/W1C-0				R/W1C-0				R/W1C-0				R/W1C-0				R/W1C-0				R/W1C-0				R/W1C-0				R/W1C-0			

LEGEND: R/W = Read/Write; R = Read only; W1C = Write 1 to clear; -n = value after reset

Table 23-61. HWAG Interrupt Flag Register (HWAFLG) Field Descriptions

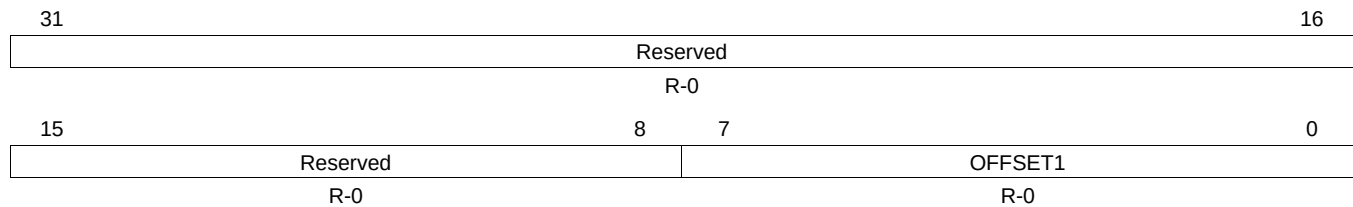
Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	INTFLG[n]	0	Interrupt Flag. These bit are set when an interrupt condition has occurred inside the HWAG. The interrupt is sent to the CPU if, and only if, the corresponding enable bit is set. HWAFLG is cleared by either reading the HWAOFF0 or HWAOFF1 register (if the corresponding bit is set) or by writing 1 to the bit. If HWAFLG is 1 but the corresponding interrupt is not enabled then it will not generate an interrupt, also the OFFSET index will not be generated for that particular HWAFLG bit. So, a read of HWAOFF registers will not clear a HWAFLG bit that is not enabled. See Table 23-57 . Read: No interrupt is pending. Write: No effect.
		1	Read: Interrupt is pending. Write: Clear the corresponding interrupt flag.

23.5.10 HWAG Interrupt Offset Register 0 (HWAOFF0)

This register is a read-only register and provides a numerical value that represents the pending interrupt with a high priority. The index can be used to locate the interrupt routine position in the vector table. A read to this register clears the corresponding interrupt pending bit in the HWAG interrupt flag register (HWAFLG). An interrupt pending bit in the HWAFLG register is the bit for which the corresponding interrupt enable bit is set.

During suspend mode, a read to this register does not clear the corresponding interrupt bit.

Figure 23-98. HWAG Interrupt Offset Register 0 (HWAOFF0)



LEGEND: R = Read only; -n = value after reset

Table 23-62. HWAG Interrupt Offset Register 0 (HWAOFF0) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	OFFSET1		High-Priority Interrupt Offset. These bits give the offset for the corresponding interrupts.
		0	Phantom interrupt
		1	Overflow period
		2	Singularity not found
		3	Tooth interrupt
		4	ACNT overflow
		5	PCNT(n) > 2 × PCNT (n-1) during normal tooth
		6	Bad active edge tooth
		7	Gap flag
		8	Angle increment overflow

23.5.11 HWAG Interrupt Offset Register 1 (HWAOFF1)

This register is a read-only register and provides a numerical value that represents the pending interrupt with a low priority. The index can be used to locate the interrupt routine position in the vector table. A read to this register clears the corresponding interrupt pending bit in the HWAG interrupt flag register (HWAFLG). An interrupt pending bit in the HWAFLG register is the bit for which the corresponding interrupt enable bit is set.

During suspend mode, a read to this register does not clear the corresponding interrupt bit.

Figure 23-99. HWAG Interrupt Offset Register 1 (HWAOFF1)

31	Reserved																16
R-0																	
15	Reserved								8	7	OFFSET2						0
R-0									R-0								

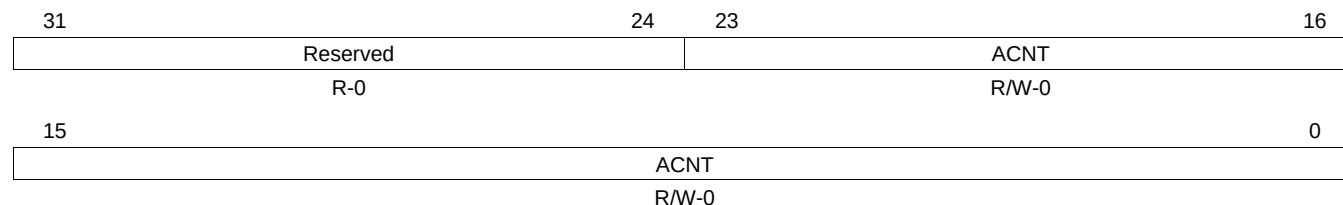
LEGEND: R = Read only; -n = value after reset

Table 23-63. HWAG Interrupt Offset Register 1 (HWAOFF1) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	OFFSET2	0	Low-Priority Interrupt Offset.. These bits give the offset for the corresponding interrupts.
		0	Phantom interrupt
		1	Overflow period
		2	Singularity not found
		3	Tooth interrupt
		4	ACNT overflow
		5	PCNT(n) > 2 × PCNT (n-1) during normal tooth
		6	Bad active edge tooth
		7	Gap flag
		8	Angle increment overflow

23.5.12 HWAG Angle Value Register (HWAACNT)

Figure 23-100. HWAG Angle Value Register (HWAACNT)



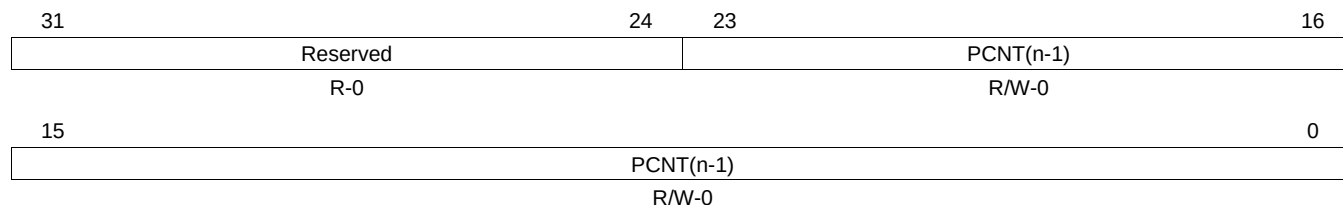
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-64. HWAG Angle Value Register (HWAACNT) Field Descriptions

Bit	Field	Value	Description
31-24	Reserved	0	Reads return 0. Writes have no effect.
23-0	ACNT	0-FF FFFFh	Angle Value. Provides the current angle value from the toothed wheel. This is equal to step width × teeth value.

23.5.13 HWAG Previous Tooth Period Value Register (HWAPCNT1)

Figure 23-101. HWAG Previous Tooth Period Value Register (HWAPCNT1)



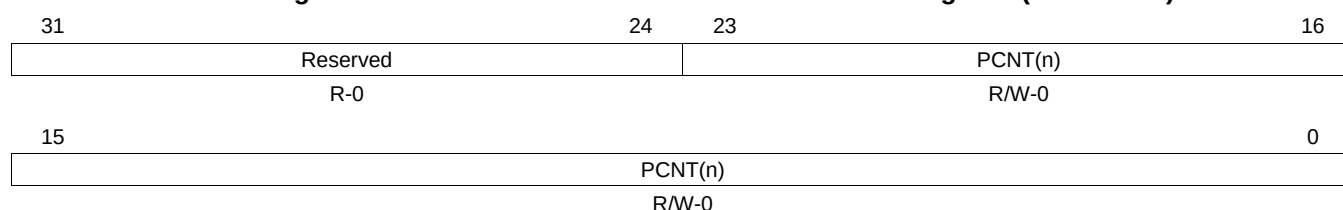
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-65. HWAG Previous Tooth Period Value Register (HWAPCNT1) Field Descriptions

Bit	Field	Value	Description
31-24	Reserved	0	Reads return 0. Writes have no effect.
23-0	PCNT(n-1)	0-FF FFFFh	Period (n-1) Value. Gives the period value of the previous tooth.

23.5.14 HWAG Current Tooth Period Value Register (HWAPCNT)

Figure 23-102. HWAG Current Tooth Period Value Register (HWAPCNT)



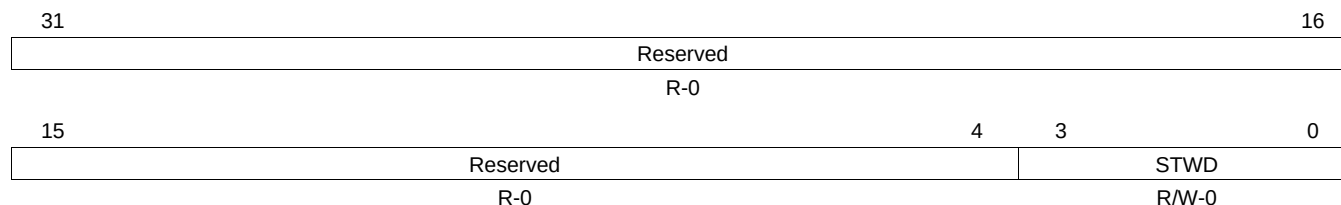
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-66. HWAG Current Tooth Period Value Register (HWAPCNT) Field Descriptions

Bit	Field	Value	Description
31-24	Reserved	0	Reads return 0. Writes have no effect.
23-0	PCNT(n)	0-FF FFFFh	Period (n) Value. Provides the current period since the beginning of the last tooth active edge seen by the HWAG (PCNT (n)). This period would not be accurate due to the fact that the PCNT counter is running at VCLK2 and that the peripheral bus is running at VCLK. Then, the value will have changed when used.

23.5.15 HWAG Step Width Register (HWASTWD)

Figure 23-103. HWAG Step Width Register (HWASTWD)



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-67. HWAG Step Width Register (HWASTWD) Field Descriptions

Bit	Field	Value	Description
31-4	Reserved		Reads return 0. Writes have no effect.
3-0	STWD		Step Width. Sets the step width for the tick generation, dividing the period into K steps. (131072, 65536, ..., 8, 4). The step count is decoded from the three LSBs using the following encoding:
		0h	4 ticks per period
		1h	8 ticks per period
		2h	16 ticks per period
		:	:
		Eh	65536 ticks per period
		Fh	131072 ticks per period

23.5.16 HWAG Teeth Number Register (HWATHNB)

Figure 23-104. HWAG Teeth Number Register (HWATHNB)

31											16	
Reserved												
R-0												
15						8	7					0
Reserved						THNB						
R-0						R/W-0						

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-68. HWAG Teeth Number Register (HWATHNB) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	THNB	0-FFh	Teeth Number. Sets the teeth number with the maximum value of the toothed wheel. This must be equal to N-1 real teeth (that is, 57 for a 60-2 toothed wheel).

23.5.17 HWAG Current Teeth Number Register (HWATHVL)

Figure 23-105. HWAG Current Teeth Number Register (HWATHVL)

31											16	
Reserved												
R-0												
15						8	7					0
Reserved						THVL						
R-0						R/W-0						

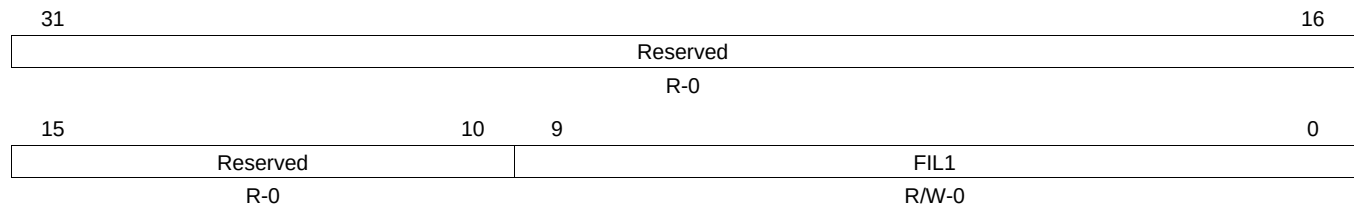
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-69. HWAG Current Teeth Number Register (HWATHVL) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	THVL	0-FFh	Teeth Value. Provides the current teeth number.

23.5.18 HWAG Filter Register (HWAFIL)

Figure 23-106. HWAG Filter Register (HWAFIL)



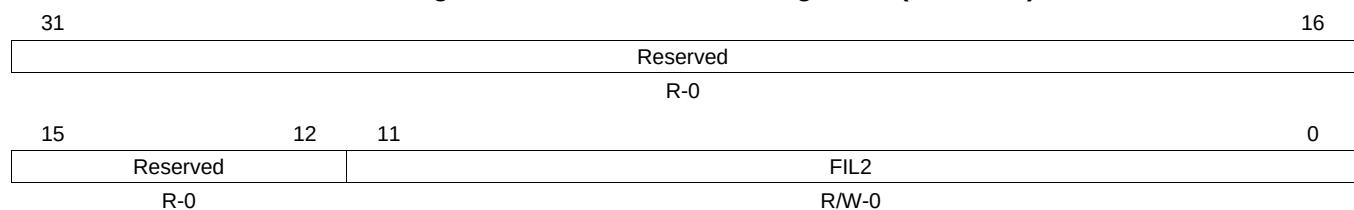
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-70. HWAG Filter Register (HWAFIL) Field Descriptions

Bit	Field	Value	Description
31-10	Reserved	0	Reads return 0. Writes have no effect.
9-0	FIL1	0-3FFh	Filter Value. Contains the value to be compared to the tick counter. It allows the tooth signal to be taken into account by the HWAG. This function works only if the mode filtering is set. The value is calculated as shown in Section 23.3.2.2.5 .

23.5.19 HWAG Filter Register 2 (HWAFIL2)

Figure 23-107. HWAG Filter Register 2 (HWAFIL2)



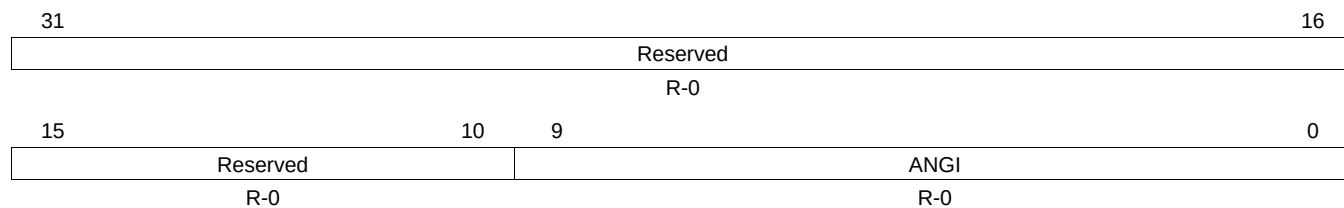
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 23-71. HWAG Filter Register 2 (HWAFIL2) Field Descriptions

Bit	Field	Value	Description
31-12	Reserved	0	Reads return 0. Writes have no effect.
11-0	FIL2	0-FFFh	Filter Value 2. Contains the value to be compared to the tick counter during the singularity tooth. It allows the tooth signal to be taken into account by the HWAG. This function works only if the mode filtering is set. The value is calculated as shown in Section 23.3.2.2.5.1 .

23.5.20 HWAG Angle Increment Register (HWAANGI)

Figure 23-108. HWAG Angle Increment Register (HWAANGI)



LEGEND: R = Read only; -n = value after reset

Table 23-72. HWAG Angle Increment Register (HWAANGI) Field Descriptions

Bit	Field	Value	Description
31-10	Reserved	0	Reads return 0. Writes have no effect.
9-0	ANGI	0-3FFh	Angle Increment Value. Provides the current angle increment value. The value is incremented by the tick counter and is decremented by the NHET resolution clock.

23.6 Instruction Set

23.6.1 Instruction Summary

Table 23-73 presents a list of the instructions in the N2HET instruction set. The pages following describe each instruction in detail.

Table 23-73. Instruction Summary

Abbreviation	Instruction Name	Opcode	Sub-Opcode	Cycles ⁽¹⁾
ACMP	Angle Compare	Ch	-	1
ACNT	Angle Count	9h	-	2
ADCNST	Add Constant	5h	-	2
ADC	Add with Carry and Shift	4h	C[25:23] = 011, C5 = 1	1-3
ADD	Add and Shift	4h	C[25:23] = 001, C5 = 1	1-3
ADM32	Add Move 32	4h	C[25:23] = 000, C5 = 1	1-2
AND	Bitwise AND and Shift	4h	C[25:23] = 010, C5 = 1	1-3
APCNT	Angle Period Count	Eh	-	1-2
BR	Branch	Dh	-	1
CNT	Count	6h	-	1-2
DADM64	Data Add Move 64	2h	-	2
DJZ	Decrement and Jump if -zero	Ah	P[7:6] = 10	1
ECMP	Equality Compare	0h	C[6:5] = 00	1
ECNT	Event Count	Ah	P[7:6] = 01	1
MCMP	Magnitude Compare	0h	C[6] = 1	1
MOV32	Move 32	4h	C[5] = 0	1-2
MOV64	Move 64	1h	-	1
OR	Bitwise OR	4h	C[25:23] = 100, C5 = 1	1-3
PCNT	Period/Pulse Count	7h	-	1
PWCNT	Pulse Width Count	Ah	P[7:6] = 11	1
RADM64	Register Add Move 64	3h	-	1
RCNT	Ratio Count	Ah	P[7:6] = 00, P[0] = 1	3
SBB	Subtract with Borrow and Shift	4h	C[25:23] = 110, C[5] = 1	1-3
SCMP	Sequence Compare	0h	C[6:5] = 01	1
SCNT	Step Count	Ah	P[7:6] = 00, P[0] = 0	3
SHFT	Shift	Fh	C[3] = 0	1
SUB	Subtract and Shift	4h	C[25:23] = 101, C[5] = 1	1-3
WCAP	Software Capture Word	Bh	-	1
WCAPE	Software Capture Word and Event Count	8h	-	1
XOR	Bitwise Exclusive-Or and Shift	4h	C[25:23] = 111, C[5] = 1	1-3

⁽¹⁾ Cycles refers to the clock cycle of the N2HET module; which on most devices is VCLK2. (Check the device datasheet description of clock domains to confirm). If the high-resolution prescale value is set to /1, then this is also the same as the number of HR clock cycles.

Table 23-74. FLAGS Generated by Instruction

Abbreviation	Flag Name	Set/Reset by	Used by
C	Carry Flag	ADC, ADD, AND, OR, RCNT, SBB, SUB, XOR	ADC, BR, SBB
N	Negative Flag	ADC, ADD, AND, OR, SBB, SUB, XOR	BR
V	Overflow Flag	ADC, ADD, AND, OR, SBB, SUB, XOR	BR
Z	Zero flag	ACNT, ADC, ADD, AND, APCNT, CNT, OR, PCNT, SBB, SCNT, SHFT, SUB, XOR	ACMP, ACNT, BR, ECMP, MCMP, MOV32, RCNT, SCMP, SHFT
X	Angle Compare Match Flag	ACMP	SCMP
SWF 0-1	Step Width flags	SCNT	ACNT
NAF	New Angle Flag	ACNT	NAF_global
NAF_global	New Angle Flag (global)	HWAG or NAF	ACMP, BR, CNT, ECMP, ECNT
ACF	Acceleration Flag	ACNT	,ACNT, SCNT
DCF	Deceleration Flag	ACNT	,ACNT, SCNT
GPF	Gap Flag	ACNT	ACNT, APCNT

The instructions capable of generating software interrupts are listed in [Table 23-75](#).

Table 23-75. Interrupt Capable Instructions

Interrupt Capable Instructions	Non Interrupt Capable Instructions
ACMP	ADC
ACNT	ADCNST
APCNT	ADD
BR	ADM32
CNT	AND
DJZ	DADM32
ECMP	MOV32
ECNT	MOV64
MCMP	OR
PCNT	RADM64
PWCNT	RCNT
SCMP	SBB
SHFT	SCNT
WCAP	SUB
WCAPE	XOR

23.6.2 Abbreviations, Encoding Formats and Bits

Abbreviations marked with a star (*) are available only on specific instructions.

U	Reading a bit marked with U will return an indeterminate value.
BRK	Defines the software breakpoint for the device software debugger. Default: OFF Location: Program field [22]
next	Defines the program address of the next instruction in the program flow. This value may be a label or an 9-bit unsigned integer. Default: Current instruction + 1 Location: Program field [21:13]
reqnum*	Defines the number of the request line (0,1,...,7) to trigger either the HTU or the DMA. Default: 0 Location: Program field [25:23]
request*	Allows to select between no request (NOREQ), request (GENREQ) and quiet request (QUIET). See Section 23.2.9 . Default: No request Location: Control Field [28:27]

Request	C[28]	C[27]	To HTU	To DMA
NOREQ	0	0	no request	no request
	1	0		
GENREQ	0	1	request	request
QUIET	1	1	quiet request	no request

remote*	Determines the 9-bit address of the remote address for the instruction. Default: Current instruction + 1 Location: Program field [8:0]
control	Determines whether the immediate data field [31:0] is cleared when it is read. When the bit is not set, reads do not clear the immediate data field. Default: OFF Location: Control field [26]
en_pin_action*	Determines whether the selected pin is ON so that the action occurs on the chosen pin Default: OFF Location: Control field [22]
Cond_addr*	Conditional address (optional): Defines the address of the next instruction when the condition occurs. Default: Current address + 1 Location: Control field [21:13]
Pin*	Pin Select: Selects the pin on which the action occurs. Enter the pin number. Default: pin 0 Location: Control field [12:8] except PCNT

The format CC{pin number} is also supported.

MSB				LSB	Description
0	0	0	0	0	Select HET[0]
0	0	0	0	1	Select HET[1]
(Each pin may be selected by writing its number in binary)					
1	1	1	1	0	Select HET[30]
1	1	1	1	1	Select HET[31]

Reg*

Register select: Selects the register for data comparison and storage

Default: No register (None)

Location: Control field [2:1] except for CNT instruction.

Extended Register Select C[7] is available for ACMP, ADC, ADD, ADM32, AND, DADM64, ECMP, ECNT, MCMP, MOV32, MOV64, OR, RADM64, SBB, SHFT, SUB, WCAP, WCAPE instructions.

Register	Ext Reg. C[7]	C[2]	C[1]
A	0	0	0
B	0	0	1
T	0	1	0
None	0	1	1
R	1	0	0
S	1	0	1
Reserved	1	1	0
Reserved	1	1	1

Action*

(2 Action Option) Either sets or clears the pin

Default: Clear

Location: Control Field [4]

Action	C[4]
Clear	0
Set	1

Action*

(4 Action Option) Either sets, clears, pulse high or pulse low on the pin. Set/clear are single pin actions, pulse high/low include the opposite pin action.

Default: Clear

Location: Control Field [4:3]

Action	Action Type	C[4]	C[3]
Clear	Set low on match	0	0
Set	Set high on match	1	0
Pulse Low	Set low on match + reset to high on Z=1 (opposite action)	0	1
Pulse High	Set high on match + reset to low on Z=1 (opposite action)	1	1

hr_lr* Specifies HIGH/LOW data resolution. If the hr_lr field is HIGH, the instruction uses the hr_data field. If the hr_lr field is LOW, the hr_data field is ignored.
Default: HIGH
Location: Program Field [8]

hr_lr	Prog. field [8]
LOW	1
HIGH	0

prv* Specifies the initial value defining the previous bit (see [Section 23.2.5.8](#)). A value of ON sets the previous pin-level bit to 1. A value of OFF sets the initial value of the previous (prv) bit to 0. The prv bit is overwritten (set or reset) by the N2HET the first time the instruction is executed.

Default: OFF

Location: Control Field [25]

cntl_val* Available for DADM64, MOV64, and RADM64, this bit field allows the user to specify the replacement value for the remote control field.

comp_mode* Specifies the compare mode. This field is used with the 64-bit move instructions. This field ensures that the sub-opcodes are moved correctly.

Default: ECMP

Location: Control Field [6:5]

Action	C[6]	C[5]	Order
ECMP	0	0	
SCMP	0	1	
MCMP1	1	0	REG_GE_DATA
MCMP2	1	1	DATA_GE_REG

23.6.3 Instruction Description

The following sections provide information for individual instructions.

Parameters in [] are optional. Refer to the N2HET assembler user guide for the default values when parameters are omitted.

23.6.3.1 ACMP (Angle Compare)

Syntax

```

ACMP {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    [control={OFF | ON}]
    [en_pin_action={OFF | ON}]
    [cond_addr={label | 9-bit unsigned integer}]
    pin={pin number}
    [action={CLEAR | SET}]
    reg={A | B | R | S | T | NONE}
    [irq ={OFF | ON}]
    data={25-bit unsigned integer}
}
    
```

Figure 23-109. ACMP Program Field (P31:P0)

31	26	25	23	22	21	13	12	9	8	0
0	Request Number			BRK	Next program address					
6	3			1	9					
						1100				Reserved
						4				9

Figure 23-110. ACMP Control Field (C31:C0)

31	29	28	27	26	25	24	23	22	21					16	
Reserved		Request type		Control	Cout prv	Reserved		En. pin action	Conditional address						
3			2		1	1	2		1	9					
15	13		12				8	7	6	5	4	3	2	1	0
Conditional address		Pin select					Ext. Reg	Reserved		Pin action	Res.	Register select		Int. ena	
9			5			1		2		1	1	2		1	

Figure 23-111. ACMP Data Field (D31:D0)

31	7	6	0
Data			Reserved
25			7

Cycles One

Register modified Selected register (A, B, R, S, or T)

The purpose of the comparison is to assert pin action when the angle compare value lies between the old counter value and the new counter value (held in the selected register). Since the angle increment varies from one loop resolution clock to another, an exact equality test cannot be applied. Instead, the following inequality is used to determine the occurrence of a match:

$$\text{Old counter value} < \text{Angle compare value} \leq \text{New counter value}$$

This is done by performing following comparisons:

Selected register value minus angle increment < angle compare value

Angle compare value ≤ Selected register value

register	Register B is recommended for typical applications with ACMP.
irq	Specifies whether or not an interrupt is generated. Specifying ON generates an interrupt when the edge state is satisfied and the gap flag is set. Specifying OFF prevents an interrupt from being generated. Default: OFF.
data	Specifies the 25-bit angle compare value.

Execution

```

X = 0;
If (Data <= Selected Register)
    Cout = 0;
else
    Cout = 1;
If (Z == 0 AND (Selected Register - Angle Inc. < Data ) AND Cout == 0) OR
    (Z == 1 AND (Cout_prv == 1 OR Cout == 0))
{
    X = 1;
    If (Enable Pin Action == 1)
        Selected Pin = Pin Action AT next loop resolution clock;

    If (Interrupt Enable == 1)
        HETFLG[n] = 1; /* n depends on address */
    If ([C28:C27] == 01)
        Generate request on request line [P25:P23];
    If ([C28:C27] == 11)
        Generate quiet request on request line [P25:P23];

    Jump to Conditional Address;
}
else
    Jump to Next Program Address;

Cout_prv = Cout (always executed)

```

NOTE: Carry-Out Signal (Cout)

Cout is the carry-out signal of the adder. Even if it is not a flag, it is valid all along ACMP instruction execution.

Angle inc. = NAF_global or hardware angle generator 11-bit input.

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).

23.6.3.2 ACNT (Angle Count)

Syntax

```
ACNT {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    edge={RISING | FALLING}
    [irq = {OFF | ON}]
    [control={OFF | ON}]
    [prv={OFF | ON}]
    gapend = {25-bit unsigned integer}
    data={25-bit unsigned integer}
}
```

Figure 23-112. ACNT Program Field (P31:P0)

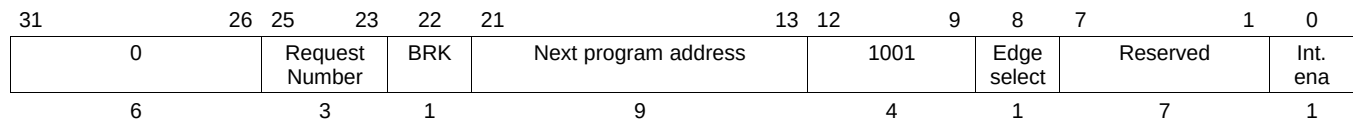


Figure 23-113. ACNT Control Field (C31:C0)

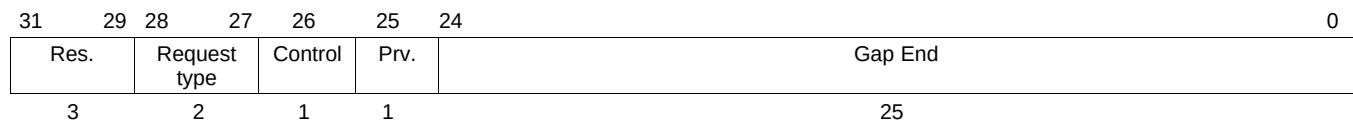
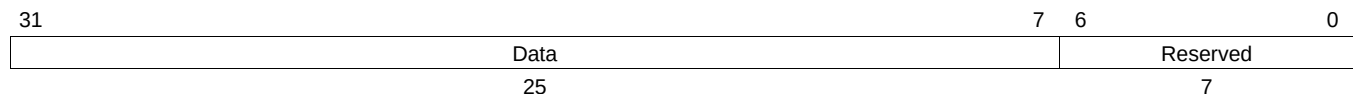


Figure 23-114. ACNT Data Field (D31:D0)



Cycles

Two, as follows:

- First cycle: Angle increment condition and gap end comparison.
- Second cycle: Gap start comparison.

Register modified

Register B (angle value)

Description

This instruction defines a specialized virtual timer used after SCNT and APCNT to generate an angle-referenced time base that is synchronized to an external signal (that is, a toothed wheel signal). ACNT uses pin HET[2] exclusively. The edge select must be the same as the HET[2] edge which was selected in the previous APCNT.

ACNT refers to the same step width selection that the previous SCNT saved in flags SWF0 and SWF1 (see information on SCNT).

ACNT detects period variations of the external signal measured by APCNT and compensates related count errors.

A period increase is flagged in the deceleration flag (DCF). A period decrease is flagged in the acceleration flag (ACF). If no variation is detected, ACNT increments the counter value each time SCNT reaches its target.

If acceleration is detected, ACNT increments the counter value on each timer resolution. If deceleration is detected ACNT does not increment and is thus saturated.

ACNT also specifies the gap end angle value defining the end value of a gap range in ACNT where period measurements in APCNT are temporarily stopped to mask singularities in the external signal. ACNT uses register A containing gap start and register B to store the counter value.

Edge Specifies the edge for the input capture pin (HET[2]).

Action	P8	Edge Select
Rising	1	Detects a rising edge of HET[2]
Falling	0	Detects a falling edge of HET[2]

irq ON generates an interrupt when the edge state is satisfied and the gap flag is set. OFF prevents an interrupt from being generated.
Default: OFF.

gapend Defines the 25-bit end value of a gap range. The start value is defined in the SCNT instruction.
 $GAPEND = (\text{Step Value} * (\# \text{ of teeth on the toothed wheel} + \# \text{ of missing teeth})) - 1$

data Specifies the 25-bit initial count value for the data field.
Default: 0.

NOTE: Target Edge Field

The target edge field represents the three LSBs of data field register in case of step width = 8, four LSBs for step width = 16, five LSBs for step width = 32 and six LSBs for step width = 64.

Execution

Increment Condition: $((Z = 1 \text{ AND } DCF = 0) \text{ OR } ACF = 1)$

Pin Edge Condition: Specified edge detected on HET[2]

Target Edge Condition: (Target Edge field in data field = 0) AND (Angle Increment condition is true) AND (GPF = 0)

If (Angle Increment Condition) is false

```
{
    NAF = 0;
    Register B = Data field register;
}
else
{
    NAF = 1;
    If (Counter value != GapEnd)
    {
        Register B = Data field register + 1;
        Data Field Register = Counter value + 1;
    }
}
```

```

        else
        {
            Register B = 0;
            Data Field Register = 0;
            If (ACF == 0) DCF = 1;
        }
    }

    Z = 0;

    If (Data field register == GapStart)
    {
        GPF = 1;
        If (Target Edge condition is true)
        {
            ACF = 0;
            If ((specified edge is not detected on pin HET[2]) AND (data
            field register != 0) AND (ACF == 0) AND (angle increment condition
            is true))
                DCF = 1;
        }
        If (specified edge is detected on pin HET[2])
        {
            DCF = 0;
            If ((target_edge_field != 0) AND (DCF == 0)) ACF = 1;
            If (GPF == 1)
            {
                GPF = 0;
                Z = 1;
                If (Interrupt Enable == 1)
                {
                    HETFLG[n] = 1;      /* n depends on address */
                    If ([C28:C27] == 01)
                        Generate request on request line [P25:P23];;
                    If ([C28:C27] == 11)
                        Generate quiet request on request line
                        [P25:P23];
                }
            }
        }
    }

    If ((target_edge_field != 0) and (pin_edge_cond == 1))
    {
        pin_update = 0;
    }
    else if (target_edge_field == 0)
    {
        pin_update = 1;
    }

    If (pin_update is true in next loop clock cycle)
    {
        Prv bit = Current Lx value of HET[2] pin;
    }

    Jump to next program address;

```

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).

23.6.3.3 ADCNST (Add Constant)

Syntax

```

ADCNST {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [control={OFF | ON}]
    remote={label | 9-bit unsigned integer}
    min_off={25-bit unsigned integer}
    data={25-bit unsigned integer}
    [hr_data={7-bit unsigned integer}]
}

```

Figure 23-115. ADCNST Program Field (P31:P0)

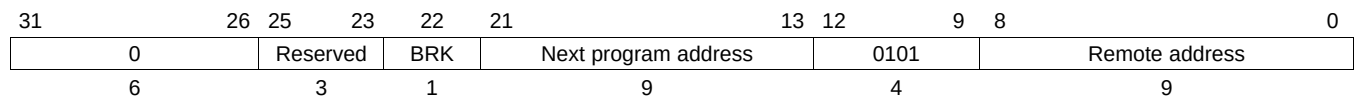


Figure 23-116. ADCNST Control Field (C31:C0)

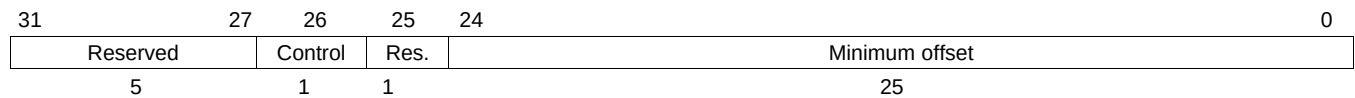
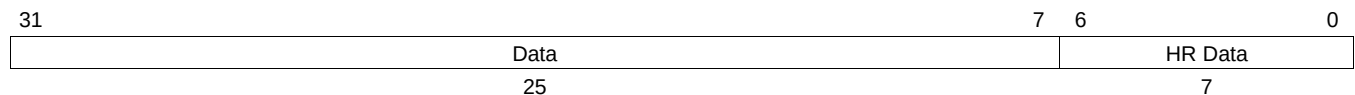


Figure 23-117. ADCNST Data Field (D31:D0)



Cycles Two

Register modified Register T (implicity)

Description ADCNST is an extension of ADM32. ADCNST first checks whether the data field value at the remote address is zero; it then performs different adds and moves on the result. ADCNST is typically used to extend the counter value of PWCNT.

min_off A 25-bit constant value that is added to the data field value if the remote data field is null.

data A 25-bit value that is always added to the remote data field.
Default: 0.

hr_data Seven least significant bits of the data addition to the remote data field.
Default: 0.

Figure 23-118 and Figure 23-119 illustrate the behavior of ADCNST if the remote data field is zero or is not zero.

Figure 23-118. ADCNST Operation If Remote Data Field[31:7] Is Not Zero

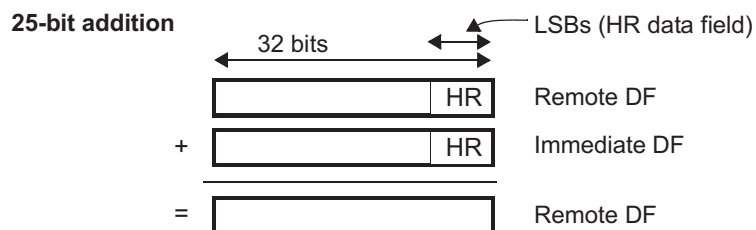
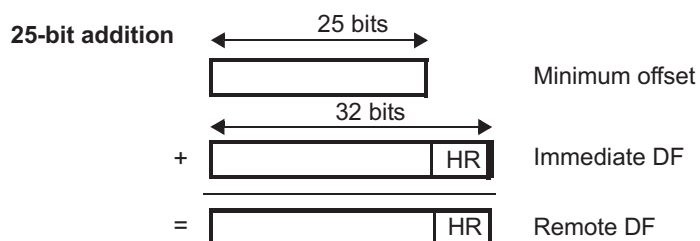


Figure 23-119. ADCNST Operation if Remote Data Field [31:7] Is Zero



Execution

```
If (Remote Data Field Value [31:7] != 0)
    Remote Data Field = Immediate Data Field + Remote Data Field;
else
    Remote Data Field = Immediate Data Field + min. offset(bits C24:C0);

Jump to Next Program Address;
```

23.6.3.4 ADC, ADD, AND, OR, SBB, SUB, XOR

Syntax

```

ADC | ADD | AND | OR | SBB | SUB | XOR {
src1 = { ZERO | IMM | A | B | R | S | T | ONES | REM | REMP }
src2 = { ZERO | IMM | A | B | R | S | T | ONES }
dest = { NONE | IMM | A | B | R | S | T }
[rdest = { NONE | REM | REMP }]
[brk={OFF | ON}]
[next={label | 9-bit unsigned integer}]
[remote={label | 9-bit unsigned integer}]
[control={OFF | ON}]
[init={OFF | ON}]
[smode = {LSL | CSL | LSR | CSR | RR | CRR | ASR }]
[scount = {5 bit unsigned integer}]
[data={25-bit unsigned integer}]
[hr_data={7-bit unsigned integer}]
}

```

Figure 23-120. ADC, ADD, AND, OR, SBB, SUB, XOR Program Field (P31:P0)

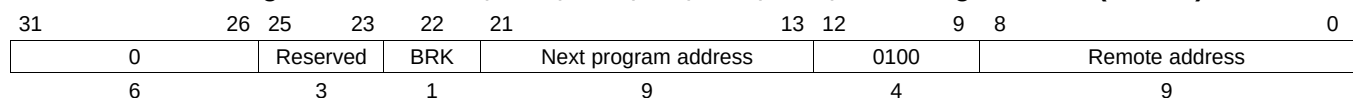


Figure 23-121. ADC, ADD, AND, OR, SBB, SUB, XOR Control Field (C31:C0)

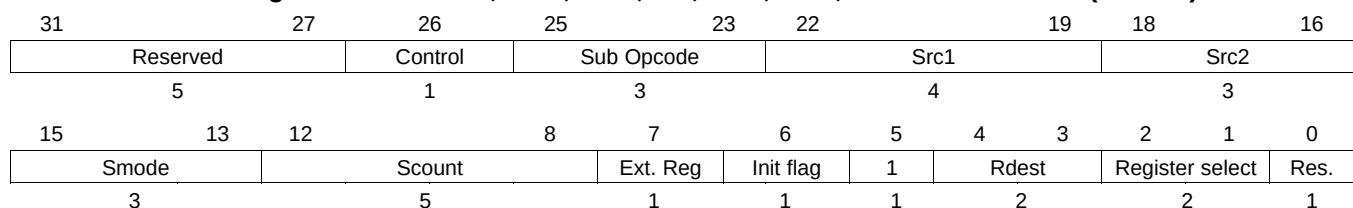
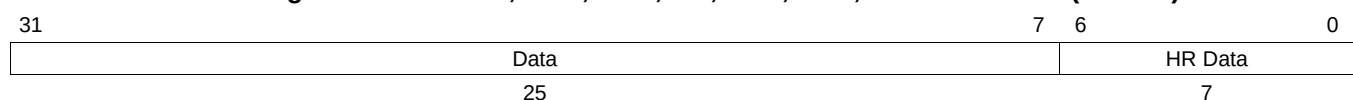


Figure 23-122. ADC, ADD, AND, OR, SBB, SUB, XOR Data Field (D31:D0)



Cycles	One to three cycles, depending on operands selected. (See Table 23-80)
Register modified	Selected register (A, B, R, S, T, or NONE)
Description	This instruction performs the specified 32-bit arithmetic or logical operation on operands src1 and src2, followed by an optional shift/rotate step. The result of this operation is then stored to either an N2HET register or the immediate data field of the instruction. In addition, the same result may be stored in a remote data field or the least significant bits of a remote instruction program field (P[8:0]). Bits P[8:0] of the program field are used by most instructions formats to hold the remote address that the instruction operates on, so the ability to update this field programatically makes it easier to write subroutines that operate on different data sets.

The Sub-Opcode field C[25:3] determines which type of operation (ADD, ADC, AND, OR, SBB, SUB, XOR) is executed by the instruction. A list of these operations and the corresponding Sub-Opcode encoding can be found in [Table 23-76](#).

All arithmetic is performed using 32-bit integer math. However, source and destination operands vary in width and can be 9 bits (REMP), 25 bits (A, B) or 32 bits (R,S,T, IMM, REM). Source operands REMP, A,B are extended to 32-bits before being operated on. Also the result of the computation needs to be truncated before being written back to REMP, A, or B when these are selected as destination operands. [Table 23-77](#) provides a list of source operand options, how they are expanded to 32-bit integers (if applicable) and the control field encoding to select the option for src1 and src2 operands.

[Table 23-78](#) provides a similar list of destination operands and their encodings. Up to two destination operands may be selected for each instruction, a register/immediate destination and a remote destination may be selected simultaneously. Truncation is performed independently for each destination operand as appropriate to its size.

An optional shift step following the arithmetic or logical operation may be selected through the smode and scout operands. The shift or rotate type is selected by the smode field; [Table 23-79](#) illustrates the options that are available for smode. The number of bits shifted is determined by the scout operand.

Table 23-76. Arithmetic / Bitwise Logic Sub-Opcodes

Instruction	Description	Operation	Sub-Opcode
ADC	Add with Carry	result = src1 + src2 + C	C[25:23] = 011
ADD	Add	result = src1 + src2	C[25:23] = 001
AND	Bitwise Logic And	result = src1 & src2	C[25:23] = 010
OR	Bitwise Logic Or	result = src1 src2	C[25:23] = 100
SBB	Subtract with Borrow	result = src1 - src2 - C	C[25:23] = 110
SUB	Subtract	result = src1 - src2	C[25:23] = 101
XOR	Bitwise Logic Exclusive Or	result = src1 ^ src2	C[25:23] = 111

Table 23-77. Source Operand Choices

Source Operand	32-bit value	Address	src1	src2
A	{A[24:0], 0x00}	n/a	C[22:19] = 0010	C[18:16] = 010
B	{B[24:0], 0x00}	n/a	C[22:19] = 0011	C[18:16] = 011
R	R[31:0]	n/a	C[22:19] = 0100	C[18:16] = 100
S	S[31:0]	n/a	C[22:19] = 0101	C[18:16] = 101
T	T[31:0]	n/a	C[22:19] = 0110	C[18:16] = 110
IMM	D[31:0]	current instruction address	C[22:19] = 0001	C[18:16] = 001
ZERO	0x00000000	n/a	C[22:19] = 0000	C[18:16] = 000
ONES	0xFFFFFFFF	n/a	C[22:19] = 0111	C[18:16] = 111
REM	D[31:0]	specified by remote[8:0]	C[22:19] = 1000	n/a
REMP	{0x000000, P[8:0]}	specified by remote[8:0]	C[22:19] = 1001	n/a

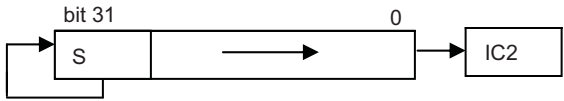
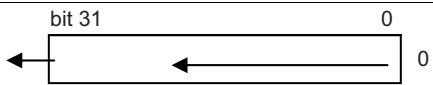
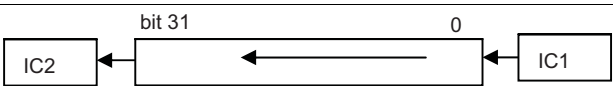
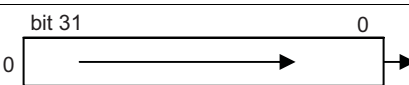
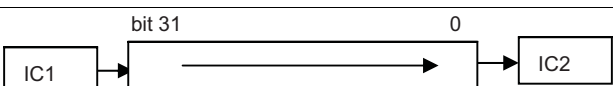
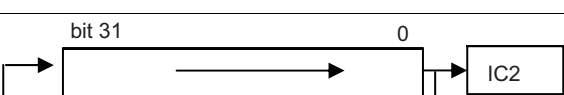
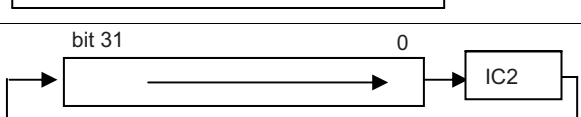
Table 23-78. Destination Operand Choices

Destination Operand	Stored Value	Address	dest	rdest
A	A[24:0] = result [31:8]	n/a	C[7] = 0, C[2:1] = 00	n/a
B	B[24:0] = result [31:8]	n/a	C[7] = 0, C[2:1] = 01	n/a
R	R[24:0] = result [31:0]	n/a	C[7] = 1, C[2:1] = 00	n/a
S	S[24:0] = result [31:0]	n/a	C[7] = 1, C[2:1] = 01	n/a
T	T[24:0] = result [31:0]	n/a	C[7] = 0, C[2:1] = 10	n/a

Table 23-78. Destination Operand Choices (continued)

Destination Operand	Stored Value	Address	dest	rdest
IMM	D[31:0] = result [31:0]	current instruction address	C[7] = 1, C[2:1] = 10	n/a
NONE	n/a	n/a	C[7] = 0, C[2:1] = 11	C[4:3] = 00
REM	D[31:0] = result [31:0]	specified by remote[8:0]	n/a	C[4:3] = 01
REMP	P[8:0] = result [8:0]	specified by remote[8:0]	n/a	C[4:3] = 10

Table 23-79. Shift Encoding

Shift Type	C[15:13] smode	Operation Illustrated ⁽¹⁾
No Shift Applied	0 0 0	n/a - no shift
ASR-Arithmetic Shift Right	0 0 1	
LSL-Logical Shift Left	0 1 0	
CSL-Carry Shift Left	0 1 1	
LSR-Logical Shift Right	1 0 0	
CSR-Carry Shift Right	1 0 1	
RR - Rotate Right	1 1 0	
CRR – Carry Rotate Right	1 1 1	

⁽¹⁾ IC1 is the carry flag after the arithmetic / logical operation is performed. IC2 is the updated carry flag after the shift operation is performed. s is the sign bit.

Table 23-80. Execution Time for ADC, ADD, AND, OR, SBB, SUB, XOR Instructions

src1	dest	rdest	remote[8:0]	Cycle s
ZERO, IMM, A, B, R, S, T, or ONES	A,B,R,S,T, or NONE	NONE	!= next[8:0]	1
REM or REMP	A,B,R,S,T, or NONE	NONE	!= next[8:0]	2
ZERO, IMM, A, B, R, S, T, or ONES	IMM	REM	!= next[8:0]	2
ZERO, IMM, A, B, R, S, T, or ONES	A,B,R,S,T, or NONE	REMP	!= next[8:0]	2
ZERO, IMM, A, B, R, S, T, or ONES	A,B,R,S,T, or NONE	NONE	== next[8:0]	2
REM or REMP	IMM	REM	x	3
x	IMM	REMP	x	3
REM or REMP	x	REM	== next[8:0]	3
x	x	REMP	== next[8:0]	3

Execution

```

    / Notes: IR1, IR2 are 32-bit intermediate results
    // SRC1, SRC2 are 32-bit sources selected
    //          by fields src1, src2
    // IC1, IC2 are intermediate values of the carry flag
    // IZ1, IZ2 are intermediate values of the zero flag
    // IN1, IN2 are intermediate values of the negative flag
    // IV1, IV2 are intermediate values of the overflow flag
    // scount is the shift count (0 to 31) specified by C12:C8

    /***** SOURCE OPERAND DECODING STAGE *****/
    switch (C22:C19)
    {
        case 0000:SRC1[31:0] = 0x00000000
        case 0001:SRC1[31:0] = Immediate Data Field D[31:0]
        case 0010:SRC1[31:8] = A[24:0]; SRC1[6:0] = 0
        case 0011:SRC1[31:8] = B[24:0]; SRC1[6:0] = 0
        case 0100:SRC1[31:0] = R[31:0]
        case 0101:SRC1[31:0] = S[31:0]
        case 0110:SRC1[31:0] = T[31:0]
        case 0111:SRC1[31:0] = 0xFFFFFFFF
        case 1000:SRC1[31:0] = Remote Data Field D[31:0]
        case 1001:SRC1[31:9] = 0; SRC1[8:0] = Remote Program Field P[8:0]
    }

    switch (C18:C16)
    {
        case 000:SRC2[31:0] = 0x00000000
        case 001:SRC2[31:0] = Immediate Data Field[31:0]
        case 010:SRC2[31:8] = A[24:0]; SRC2[6:0] = 0
        case 011:SRC2[31:8] = B[24:0]; SRC2[6:0] = 0
        case 100:SRC2[31:0] = R[31:0]
        case 101:SRC2[31:0] = S[31:0]
        case 110:SRC2[31:0] = T[31:0]
        case 111:SRC2[31:0] = 0xFFFFFFFF
    }

    /***** ARITHMETIC / LOGICAL OPERATION STAGE *****/
    switch (C[25:23])
    {
        case 011:IR1 = src1 + src2 + C // ADC
        case 001:IR1 = src1 + src2 // ADD
        case 010:IR1 = src1 & src2 // AND
        case 100:IR1 = src1 | src2 // OR
        case 110:IR1 = src1 - src2 - C // SBB
        case 101:IR1 = src1 - src2 // SUB
        case 111:IR1 = src1 ^ src2 // XOR
    }

    IC1 = Carry Out if Operation is ADD, ADC, SUB, SBB
        = 0 if Operation is AND, OR, XOR
    IZ1 = Set if IR1 is zero, Clear if IR1 is non-zero
    IN1 = IR[31]
    IV1 = (IC1 XOR IR1[31]) AND NOT(SRC1[31] XOR SRC2[31])

    /***** SHIFT STAGE *****/
    switch (C15:C13)
    {
        case 000: // smode = No Shift
            IR2 = IR1
            IC2 = IC1; IZ2 = IZ1; IN2 = IN1; IV2 = IV1;

        case 001: // smode = Arithmetic Shift Right
            IR2[31 - scount : 0] = IR1[31:scount]

            if (scount>0) {
                IR2[31 : 31 - scount + 1] = IR1[31]
            }
    }

```

```

        IC2 = IR1[scount-1]
    }
    else {
        IC2 = IC1
    }

    IN2 = IR2[31];
    if (IR2 == 0) { IZ2 = 1 } else {IZ2 = 0};
    IV2 = (IR2[31] XOR IR1[31]) OR IV1

case 010: // smode = Logical Shift Left
    IR2[31 : scount] = IR1[31 - scount: 0]

    if (scount > 0) {
        IR2[scount - 1 : 0] = 0
    }

    IC2 = IC1
    IN2 = IR2[31];
    if (IR2 == 0) { IZ2 = 1 } else {IZ2 = 0};
    IV2 = (IR2[31] XOR IR1[31]) OR IV1

case 011: // smode = Carry Shift Left
    IR2[31 : scount] = IR1[31 - scount: 0]

    if (scount>0) {
        IR2[scount - 1 : 0] = [IC1,...IC1]
        IC2 = IR1[31 - scount + 1]
    }
    else
    {
        IC2 = IC1
    }

    IN2 = IR2[31];
    if (IR2 == 0) { IZ2 = 1 } else {IZ2 = 0};
    IV2 = (IR2[31] XOR IR1[31]) OR IV1

case 100: // smode = Logical Shift Right
    IR2[31 - scount : 0] = IR1[31:scount]

    if (scount>0) {
        IR2[31 : 31 - scount + 1] = 0
    }

    IC2 = IC1
    IN2 = IR2[31];
    if (IR2 == 0) { IZ2 = 1 } else {IZ2 = 0};
    IV2 = (IR2[31] XOR IR1[31]) OR IV1

case 101: // smode = Carry Shift Right
    IR2[31 - scount : 0] = IR1[31:scount]

    if(scount>0) {
        IR2[31:31-scount + 1] = [IC1,...IC1]
        IC2 = IR1[scount-1]
    }
    else {
        IC2 = IC1
    }

    IN2 = IR2[31];
    IZ2 = Set if IR2 == 0;
    IV2 = (IR2[31] XOR IR1[31]) OR IV1

case 110: // smode = Rotate Right

```

```

        IR2[31 - scount : 0] = IR1[31:scount]

        if(scount>0) {
            IR2[31:31-scount+1] = IR1[scount-1:0]
            IC2 = IR1[scount-1]
        }
        else {
            IC2 = IC1
        }

        IN2 = IR2[31];
        if (IR2 == 0) { IZ2 = 1 } else {IZ2 = 0};
        IV2 = (IR2[31] XOR IR1[31]) OR IV1

case 111: // smode = Carry Rotate Right
    IR2[31 - scount : 0] = IR1[31:scount]

    if (scount == 0) {
        IC2 = IC1
    }
    else if (scount == 1) {
        IR2[31] = IC1
        IC2 = IR1[0]
    }
    else {
        IR2[31:31-scount+1] = {IR1[scount-2:0],IC1}
        IC2 = IR1[scount - 1]
    }

    IN2 = IR2[31];
    if (IR2 == 0) { IZ2 = 1 } else {IZ2 = 0};
    IV2 = (IR2[31] XOR IR1[31]) OR IV1
}
/***** WRITE REGISTER DESTINATION STAGE *****/
switch (C7, C2:C1)
{
    case 000:A[24:0] = IR2[31:8]
    case 001:B[24:0] = IR2[31:8]
    case 010:T[31:0] = IR2[31:0]
    case 011:IR2 is not stored in register, immediate
    case 100:R[31:0] = IR2[31:0]
    case 101:S[31:0] = IR2[31:0]
    case 110:Immediate Data Field[31:0] = IR2
    case 111:IR2 is not stored in register, immediate
}
/***** WRITE REMOTE DESTINATION STAGE *****/
switch (C4:3)
{
    case 00:IR2 is not stored in remote field
    case 01:Remote Data Field D[31:0] = IR2
    case 10:Remote Program Field P[8:0] = IR2[8:0]
    case 11:IR2 is not stored in remote field
}
/***** UPDATE FLAGS STAGE *****/
C FLAG = IC2
N FLAG = IN2
Z FLAG = IZ2
V FLAG = IV2
If (Init Flag == 1)
{
    ACF = 0;
    DCF = 1;
    GPF = 0;
    NAF = 0;
}
else ACF, DCF, GPF, NAF remain unchanged;

```

23.6.3.5 ADM32 (Add Move 32)

Syntax

```

ADM32 {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    remote={label | 9-bit unsigned integer}
    [control={OFF | ON}]
    [init={OFF | ON}]
    type={IM&REGTOREG | REM&REGTOREG | IM&REMTOREG |
    IM&REGTOREM}
    reg={A | B | R | S | T }
    data={25-bit unsigned integer}
    [hr_data={7-bit unsigned integer}]
}

```

Figure 23-123. ADM32 Program Field (P31:P0)

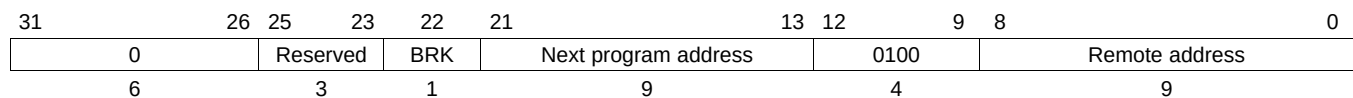


Figure 23-124. ADM32 Control Field (C31:C0)

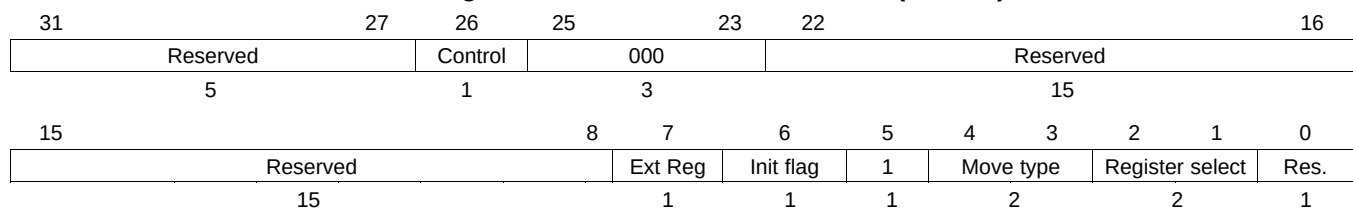
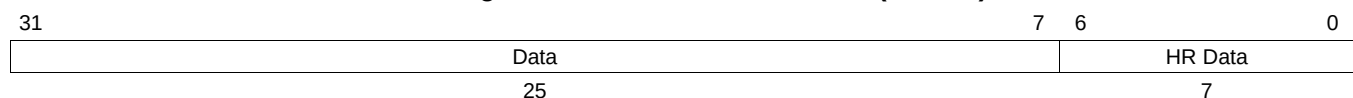


Figure 23-125. ADM32 Data Field (D31:D0)



Cycles One or two cycles (see [Table 23-81](#))

Register modified Selected register (A, B, R, S, or T)

Description This instruction modifies the selected ALU register or data field values at the remote address depending on the move type. The modified value results from adding the immediate or remote data field to the ALU register or the remote data field, depending on the move type. Table description shows the C2 and C1 bit encoding for determining which register is selected.

init (Optional) Determines whether or not system flags are initialized. A value of ON reinitializes the following system flags to these states:

- Acceleration flag (ACF) = 0
- Deceleration flag (DCF) = 1
- Gap flag (GPF) = 0

type

New angle flag (NAF) = 0
 A value of OFF results in no change to the system flags.
 Default: OFF

Specifies the move type to be executed.

Table 23-81. Move Types for ADM32

Type	C4	C3	Add	Destination(s)	Cycles
IM®TOREG	0	0	Imm. data field + Reg. A, B, R, S, or T	Register A, B, R, S, or T	1
REM®TOREG	0	1	Remote data field + Reg. A, B, R, S, or T	Register A, B, R, S, or T	2
IM&REMTOREG	1	0	Imm. data field + Remote data field	Register A, B, R, S, or T	2
IM®TOREM	1	1	Imm. data field + Reg. A, B, R, S, or T	Remote data field	1

If selected register is R, S, or T, the operation is a 32-bit Addition/move. If A or B register is selected, it is limited to 25-bit operation since A and B only support 25-bit.

data

Specifies the 25-bit integer value for the immediate data field.

hr_data

Specifies the 7 least significant bits of the immediate data field.
 Default: 0.

Execution

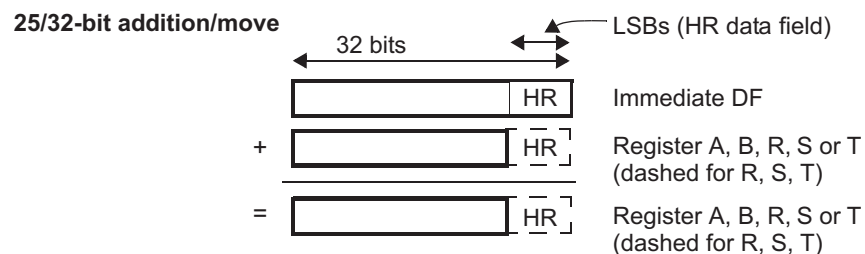
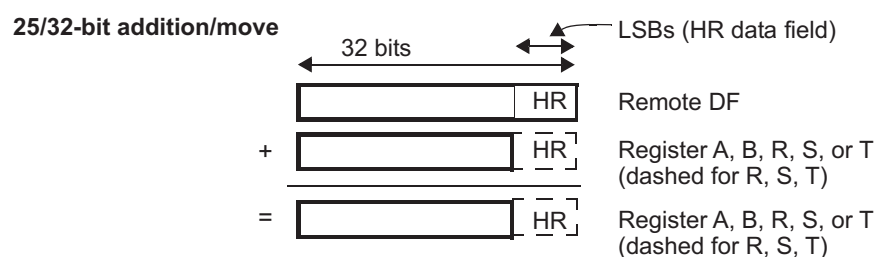
```

switch (C4:C3)
{
    case 00:
        Selected register = Selected register + Immediate Data Field;
    case 01:
        Selected register = Selected register + Remote Data Field;
    case 10:
        Selected register = Immediate Data Field + Remote Data Field;
    case 11:
        Remote Data Field = Selected register + Immediate Data Field;
}

If (Init Flag == 1)
{
    ACF = 0;
    DCF = 1;
    GPF = 0;
    NAF = 0;
}
else
    All flags remain unchanged;

Jump to Next Program Address;
  
```

Figure 23-126 and Figure 23-127 illustrate the ADM32 operation for various cases.

Figure 23-126. ADM32 Add and Move Operation for IM®TOREG (Case 00)

Figure 23-127. ADM32 Add and Move Operation for REM®TOREG (Case 01)


23.6.3.6 APCNT (Angle Period Count)

Syntax

```

APCNT {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    [irq={OFF | ON}]
    type={FALL2FALL | RISE2RISE}
    [control={OFF | ON}]
    prv={OFF | ON}}
    period={25-bit unsigned integer}
    data={25-bit unsigned integer}
}
    
```

Figure 23-128. APCNT Program Field (P31:P0)

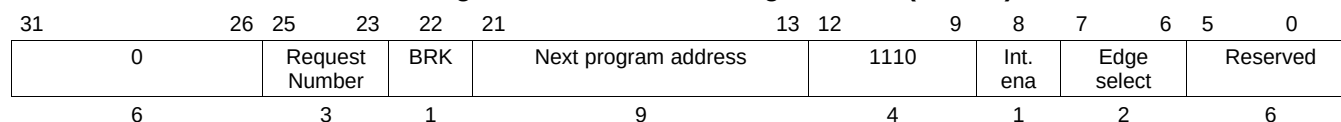


Figure 23-129. APCNT Control Field (C31:C0)

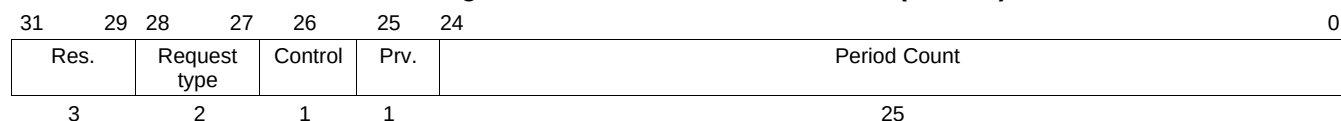
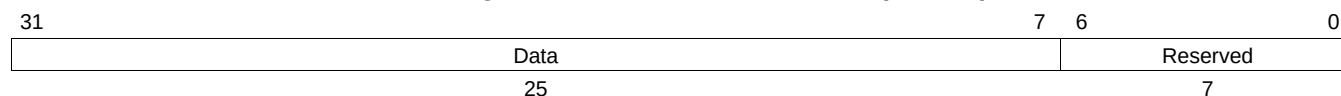


Figure 23-130. APCNT Data Field (D31:D0)



Cycles

One or two cycles

- Cycle 1: edge detected (normal operation)
 - Cycle 2: edge detected and GPF = 1 and underflow condition is true
- One cycle (normal operation) two cycles (edge detected)

Register modified

Register A and T (implicitly)

Description

This instruction is used before SCNT and ACNT to generate an angle-referenced time base synchronized to an external signal (that is, a toothed wheel signal). It is assumed that the pin and edge selections are the same for APCNT and ACNT.

APCNT is restricted to pin HET[2]. The toothed wheel must then be connected to pin HET[2].

APCNT uses the gap flag (GPF) defined by ACNT to start or stop captures in the period count field [C24:C0]. When GPF = 1, the previous period value is held in the control field and in register T. When GPF = 0, the current period value is captured in the control field and in register T.

APCNT uses the step width flags (SWF0 and SWF1) defined by SCNT to detect period durations shorter than one step, and then disables capture. The edge select encoding is shown in [Table 23-82](#).

irq ON generates an interrupt when the edge state is satisfied. OFF prevents an interrupt from being generated.
Default: OFF.

type Specifies the edge type that triggers the instruction.
Default: Fall2Fall.

Table 23-82. Edge Select Encoding for APCNT

type	P7	P6	Selected Condition
Fall2Fall	1	0	Falling edge
Rise2Rise	1	1	Rising edge

period Contains the 25-bit count value from the previous APCNT period.

data 25-bit value serving as a counter.
Default: 0.

Execution

```

Z = 0;

If (Data field register != 1FFFFFFh)
{
    Register A = Data field register + 1;
    Data field register = Data field register + 1;
}
elseif (specified edge not detected on HET[2])
{
    Register A = 1FFFFFFh;
    APCNT Ovflw flag = 1;
}

If (specified edge detected on HET[2])
{
    Z = 1;

    If (Data field register == 1FFFFFFh)
    {
        Register A = 1FFFFFFh;
        Register T = 1FFFFFFh; Period count = 1FFFFFFh;
        Period count = 1FFFFFFh;
    }
    elseif (GPF == 0 AND Data Field register >= Step width)
    {
        Register A = Data field register + 1;
        Register T = Register A;
        Period count = Register T;

        If (Interrupt Enable == 1)
            HETFLG[n] = 1; /* n depends on address */
        If ([C28:C27] == 01)
            Generate request on request line [P25:P23];
        If ([C28:C27] == 11)
            Generate quiet request on request line [P25:P23];
    }

    If (GPF == 1)
        Register T = Period count;
    If (Data Field register < Step width)
    {
        Register T = Period count;
        APCNT Undflw flag = 1;
        Period Count = 000000h;
    }

    Data field register = 000000h;
}
else
{
    Register T = Period count;
}

Prv bit = Current Lx value of HET[2] pin;

Jump to Next Program Address;

```

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).

23.6.3.7 BR (Branch)

Syntax

```
BR {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    [control={OFF | ON}]
    [prv={OFF | ON}]
    cond_addr={label | 9-bit unsigned integer}
    [pin= {pin number}]
    event={NOCOND | FALL | RISE | BOTH | ZERO | NAF | LOW | HIGH | C | NC
    | EQ | Z | NE | NZ | N | PZ | V | NV | ZN | P | GE | LT | GT | LE | LO | HS }
    [irq={OFF | ON}]
}
```

Figure 23-131. BR Program Field (P31:P0)

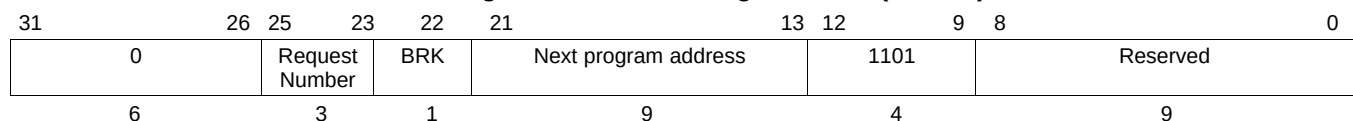


Figure 23-132. BR Control Field (C31:C0)

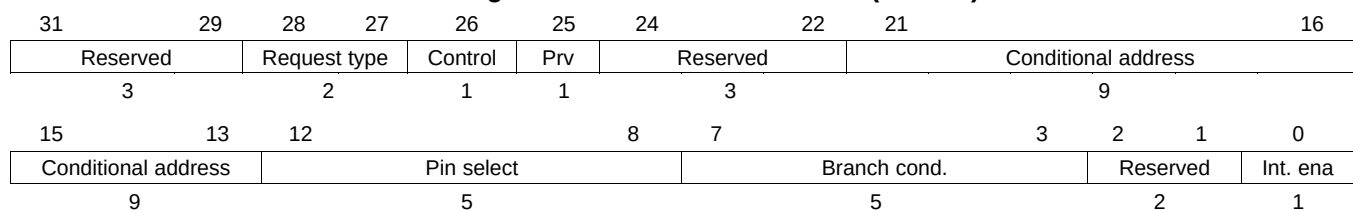
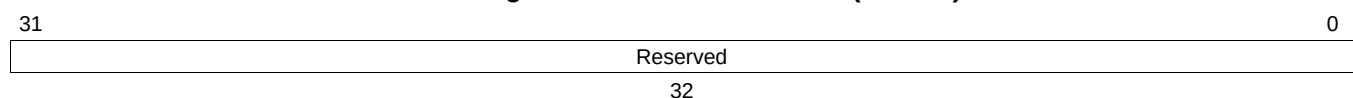


Figure 23-133. BR Data Field (D31:D0)



Cycles

One

Register modified

None

Description

This instruction executes a jump to the conditional address [C21:C13] on a pin or a flag condition, and can be used with all pins.

[Table 23-83](#) provides the branch condition encoding.

event

Specifies the event that triggers a jump to the indexed program address.

Default: FALL

irq

ON generates an interrupt when the event occurs that triggers the jump. If irq is set to OFF, no interrupt is generated.

Default: OFF.

Table 23-83. Branch Condition Encoding for BR

Event	C7	C6	C5	C4	C3	Branch Condition
NOCOND	0	0	0	0	0	Always
FALL	0	0	1	0	0	On falling edge on the selected pin
RISE	0	1	0	0	0	On rising edge on selected pin
BOTH	0	1	1	0	0	On rising or falling edge on selected pin
ZERO	1	0	0	0	0	If Zero flag is set
NAF	1	0	1	0	0	If NAF_global flag is set
LOW	1	1	0	0	0	On LOW level on selected pin
HIGH	1	1	1	0	0	On HIGH level on selected pin
C	0	0	0	0	1	Carry Set: C==1
NC	0	0	0	1	1	Carry Not Set: C==0
EQ, Z	0	0	1	0	1	Equal or Zero: Z==1
NE, NZ	0	0	1	1	1	Not Equal or Not Zero: Z==0
N	0	1	0	0	1	Negative: N==1
PZ	0	0	1	1	1	Positive or Zero: N==0
V	0	1	1	0	1	Overflow: V==1
NV	0	1	1	1	1	No Overflow: V==0
ZN	1	0	0	0	1	Zero or Negative: (Z OR N) == 1
P	1	0	0	1	1	Positive: (Z OR N) == 0
GE	1	0	1	1	1	Signed Greater Than or Equal: (N XOR V) == 0
L	1	0	1	0	1	Signed Less Than (N XOR V) == 1
G	1	1	0	1	1	Signed Greater Than (Z OR (N XOR V)) == 0
LE	1	1	0	0	1	Signed Less Than (Z OR (N XOR V)) == 1
LO	1	1	1	1	1	Unsigned Less Than: (C OR Z) == 0
HS	1	1	1	0	1	Unsigned Higher or Same (C OR Z) == 1

Execution

```

If (Condition is true)
{
    If (Interrupt Enable == 1) HETFLG[n] = 1;      /* n depends on address */
    If ([C28:C27] == 01) Generate request on request line [P25:P23];
    If ([C28:C27] == 11) Generate quiet request on request line [P25:P23];
    Jump to Conditional Address;
}
else
{
    Jump to Next Program Address;
}

```

Prv bit = Current Lx value of selected pin; (Always Executed)

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).

23.6.3.8 CNT (Count)

Syntax

```

CNT {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    [angle_count={OFF | ON}]
    [reg={A | B | T | NONE}]
    [comp = {EQ | GE}]
    [irq={OFF | ON}]
    [control={OFF | ON}]
    max={25-bit unsigned integer}
    [data={25-bit unsigned integer}]
}

```

Figure 23-134. CNT Program Field (P31:P0)

31	26	25	23	22	21	13	12	9	8	7	6	5	4	1	0
0		Request Number		BRK		Next program address			0110		Angle count	Register	Comp. select	Res.	Int. ena
6		3		1		9			4		1	2	1	4	1

Figure 23-135. CNT Control Field (C31:C0)

31	29	28	27	26	25	24	0
Res.			Request type		Control	Res.	Max Count
3			2		1	1	25

Figure 23-136. CNT Data Field (D31:D0)

31	7	6	0
Data		Reserved	
25	7		

Cycles One or two
One cycle (time mode), two cycles (angle mode)

Register modified Selected register (A, B or T)

Description This instruction defines a virtual timer. The counter value stored in the data field [D31:7] is incremented unconditionally on each execution of the instruction when in time mode (angle count bit [P8] = 0). When the count reaches the maximum count specified in the control field, the counter is reset. It takes one cycle in this mode.

In angle mode (angle count bit [P8] = 1), CNT needs data from the software angle generator (SWAG). When in angle count mode the angle increment value will be 0 or 1. It takes two cycles in this mode.

angle_count	Specifies when the counter is incremented. A value of ON causes the counter value to be incremented only if the new angle flag is set (NAF_global = 1). A value of OFF increments the counter each time the CNT instruction is executed. Default value for this field is OFF.
comp	When set to EQ the counter is reset, when it is equal to the maximum count. When set to GE the counter is reset, when it is greater or equal to the maximum count. Default: GE.
irq	ON generates an interrupt when the counter overflows to zero. The interrupt is not generated until the data field is reset to zero. If irq is set to OFF, no interrupt is generated. Default: OFF.
max	Specifies the 25-bit integer value that defines the maximum count value allowed in the data field. When the count in the data field is equal to max, the data field is reset to 0 and the Z system flag is set to 1.
data	Specifies the 25-bit integer value serving as a counter. Default: 0.

Execution

```

Z = 0;

If (Angle Count (bit P8 == 1))
{
    If (NAF_global == 0)
    {
        Selected register = immediate data field;
        Jump to Next Program Address;
    }
    else
    {
        If ((Immediate Data Field + Angle Increment) >= Max count)
        {
            Z = 1;
            Selected register = ((Immediate Data Field + Angle Inc.) - Max count);
            Immediate Data Field = ((Immediate Data Field + Angle Inc.) - Max count);

            If (Interrupt Enable == 1) HETFLG[n] = 1;      /* n depends on address */
            If ([C28:C27] == 01) Generate request on request line [P25:P23];
            If ([C28:C27] == 11) Generate quiet request on request line [P25:P23];
        }
        else
        {
            Selected register = Immediate Data Field + Angle Increment;
            Immediate Data Field = Immediate Data Field + Angle Increment;
        }
    }
}

else if(Time mode (bit P8 == 0))
{
    If [(P5==0) AND (Immediate Data Field == Max count)]
    OR [(P5==1) AND (Immediate Data Field >= Max count)]
    {
        Z = 1;
        Selected register = 00000;
        Immediate Data Field = 00000;

        If (Interrupt Enable == 1) HETFLG[n] = 1;      /* n depends on address */
        If ([C28:C27] == 01) Generate request on request line [P25:P23];
        If ([C28:C27] == 11) Generate quiet request on request line [P25:P23];
    }
    else
    {
        Selected register = Immediate Data Field + 1;
        Immediate Data Field = Immediate Data Field + 1;
    }
}

Jump to Next Program Address;

```

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).

23.6.3.9 DADM64 (Data Add Move 64)

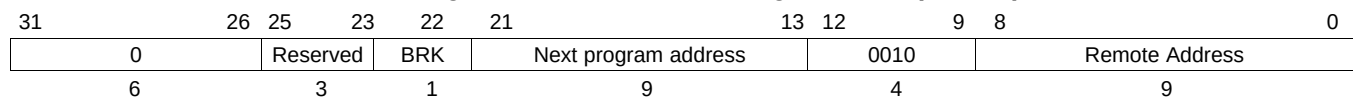
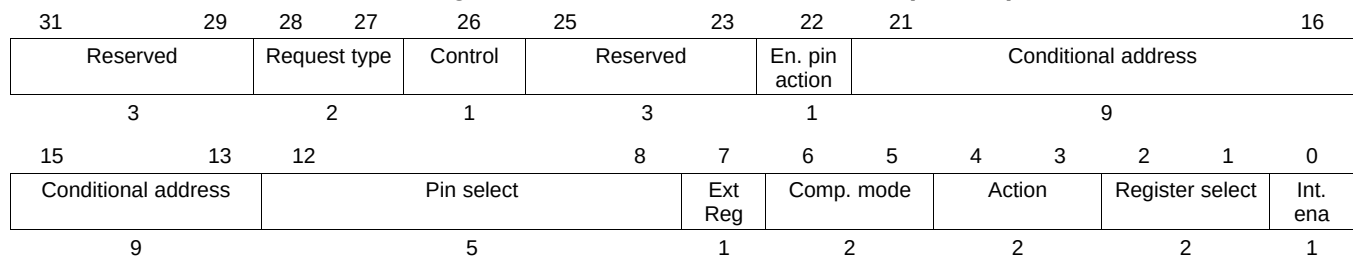
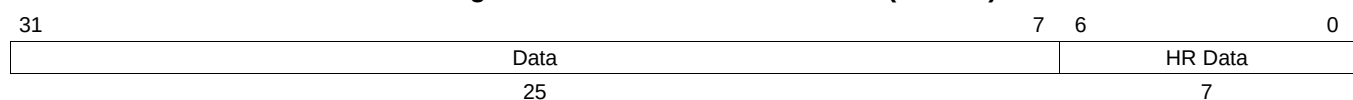
Syntax

```
DADM64 {
[brk={OFF | ON}]
[next={label | 9-bit unsigned integer}]
[remote={label | 9-bit unsigned integer}]
[request={NOREQ | GENREQ | QUIET}]
[control={OFF | ON}]
[en_pin_action={OFF | ON}]
[cond_addr={label | 9-bit unsigned integer}]
[pin={pin number}]
comp_mode={ECMP | SCMP | MCMP1 | MCMP2}
[action={CLEAR | SET | PULSELO | PULSEHI}]
[reg={A | B | R | S | T | NONE}]
[irq={OFF | ON}]
[data={25-bit unsigned integer}]
[hr_data= {7-bit unsigned integer}]
}
```

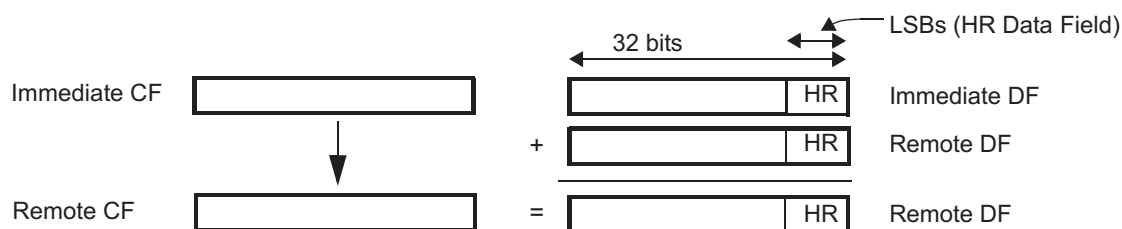
-or-

Syntax

```
DADM64 {
[brk={OFF | ON}]
[next={label | 9-bit unsigned integer}]
[remote={label | 9-bit unsigned integer}]
cntl_val={29-bit unsigned integer}
data={25-bit unsigned integer}
[hr_data= {7-bit unsigned integer}]
}
```

Figure 23-137. DADM64 Program Field (P31:P0)

Figure 23-138. DADM64 Control Field (C31:C0)

Figure 23-139. DADM64 Data Field (D31:D0)


Cycles	Two
Register modified	Register T (implicitly)
Description	This instruction modifies the data field and the control field at the remote address. The remote data field value is not just replaced, but is added with the DADM64 data field. DADM64 has two distinct syntaxes. In the first syntax, bit values may be set by assigning a value to each of the control fields. This syntax is convenient for modifying control fields that are arranged similarly to the format of the DADM64 control field. A second syntax, in which the entire 29-bit control field is specified by the cntl_val field, is convenient when the remote control field is dissimilar to the DADM64 control field. Either syntax may be used, but you must use one or the other but not a combination of syntaxes. Figure 23-140 shows the DADM64 add and move operation.

Figure 23-140. DADM64 Add and Move Operation

Table 23-84. DADM64 Control Field Description

request	maintains the control field for the remote instruction
control	maintains the control field for the remote instruction
en_pin_action	maintains the control field for the remote instruction
cond_addr	maintains the control field for the remote instruction

Table 23-84. DADM64 Control Field Description (continued)

pin	maintains the control field for the remote instruction
register	maintains the control field for the remote instruction
action	maintains the control field for the remote instruction
irq	maintains the control field for the remote instruction
data	Specifies the 25-bit initial value for the data field.
hr_data	Seven least significant bits of the 32 bit data field. Default: 0
cntl_val	Specifies the 29 least significant bits of the Control field.

Execution

```

Remote Data Field = Remote Data Field + Immediate Data Field;
Register T = Immediate Data Field;
Remote Control Field = Immediate Control Field;
Jump to Next Program Address;

```

23.6.3.10 DJZ (Decrement and Jump if Zero)

DJNZ is also a supported syntax. The functionality of the two instruction names is identical.

Syntax

```

DJZ {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    [control={OFF | ON}]
    [cond_addr={label | 9-bit unsigned integer}]
    [reg={A | B | T | NONE}]
    [irq={OFF | ON}]
    [data={25-bit unsigned integer}]
}
    
```

Figure 23-141. DJZ Program Field (P31:P0)

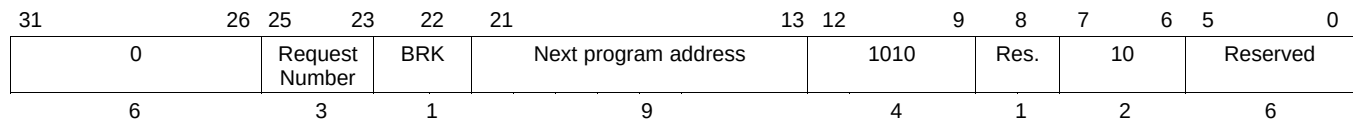


Figure 23-142. DJZ Control Field (C31:C0)

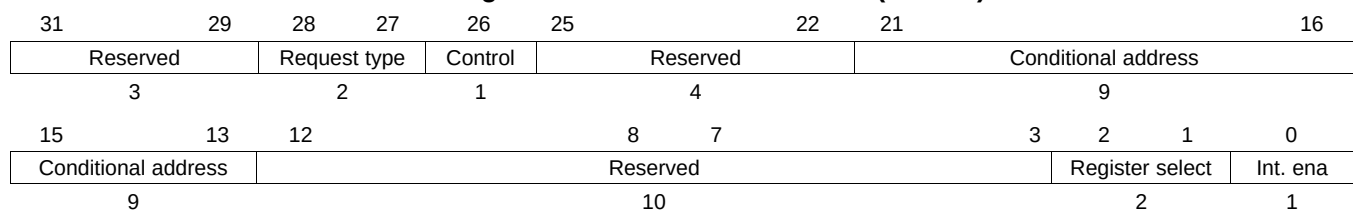
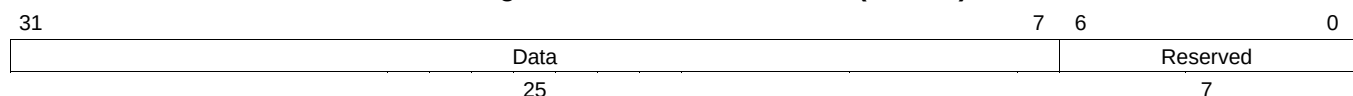


Figure 23-143. DJZ Data Field (D31:D0)



Cycles One

Register modified Selected register (A, B, or T)

Description

This instruction defines a virtual down counter used for delayed execution of certain instructions (to generate minimum on/off times). When DJZ is executed with counter value not zero, the counter value is decremented. If the counter value is zero, the counter remains zero until it is reloaded with a non-zero value. The program flow can be modified when down counter value is zero by using the conditional address.

cond_addr	This field is not optional for the DJZ instruction.
irq	ON generates an interrupt when the data field reaches zero. No interrupt is generated when the bit is OFF. Default: OFF.
data	Specifies the 25-bit integer value used as a counter. This counter is decremented each time the DJZ instruction is executed until the counter reaches 0. Default: 0.

Execution

```

If (Data != 0)
{
    Data = Selected register = Data - 1;
    Jump to Next Program Address;
}
else
{
    Selected register = 000000h;

    If (Interrupt Enable == 1) HETFLG[n] = 1;      /* n depends on address */
    If ([C28:C27] == 01) Generate request on request line [P25:P23];
    If ([C28:C27] == 11) Generate quiet request on request line [P25:P23];

    Jump to conditional Address;
}

```

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).

23.6.3.11 ECMP (Equality Compare)

Syntax

```

ECMP {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    [hr_lr={HIGH | LOW}]
    [angle_comp={OFF | ON}]
    [control={OFF | ON}]
    [en_pin_action={OFF | ON}]
    [cond_addr={label | 9-bit unsigned integer}]
    pin={pin number}
    [action={CLEAR | SET | PULSELO | PULSEHI}]
    [reg={A | B | R | S | T | NONE}]
    [irq={OFF | ON}]
    [data={25-bit unsigned integer}]
    [hr_data={7-bit unsigned integer}]
}

```

Figure 23-144. ECMP Program Field (P31:P0)

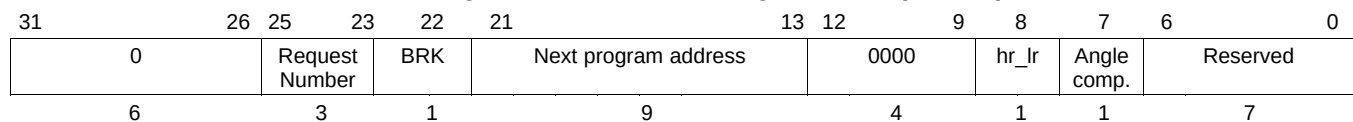


Figure 23-145. ECMP Control Field (C31:C0)

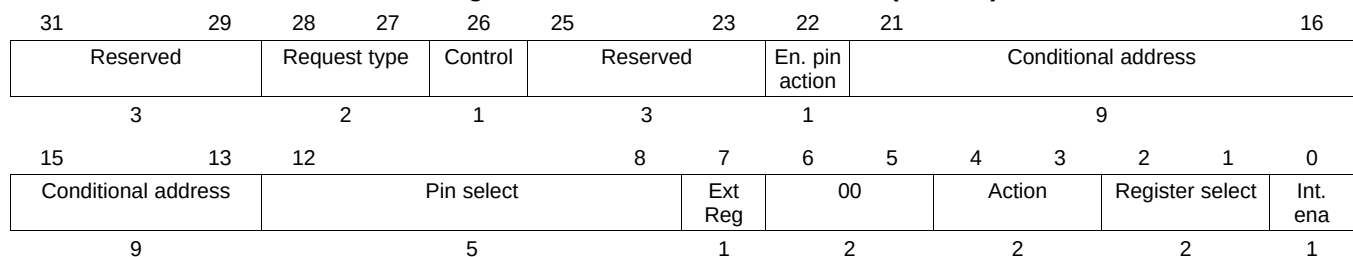
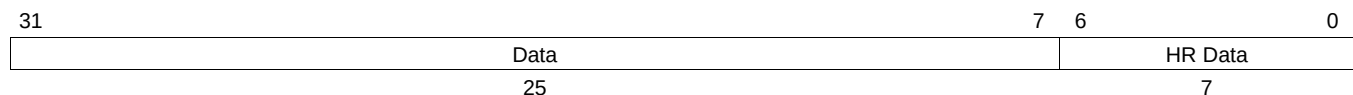


Figure 23-146. ECMP Data Field (D31:D0)



Cycles	One
Register modified	Register A, B, R, S or T if selected
Description	ECMP can use all pins. This instruction compares a 25-bit data value stored in the data field (D31–D7) to the value stored in the selected ALU register (A, B, R, S, or T). Register select encoding can be found in Section 23.6.2. If R, S, or T registers are selected, and if the 25-bit data field matches, ECMP updates the register with the 32-bit value (D31–D0). If the hr_lr bit is cleared, the pin action will occur after a high resolution delay from the next loop resolution clock. If the hr_lr bit is set, the delay is ignored. This delay is programmed in the data field (D6–D0). The behavior of the pins is governed by the four action options in bits C4:C3. ECMP uses the zero flag to generate opposite pin action (synchronized to the loop resolution clock).
angle_comp	Determines if an angle compare is performed. A value of ON causes the comparison to be performed only if the new angle flag is set (NAF = 1). If OFF is specified, the compare is then performed regardless of the state of the new angle flag. Default: OFF.
irq	Specifies whether or not an interrupt is generated. A value of ON sends an interrupt if register and data field values are equivalent. If OFF is selected, no interrupt is generated. Default: OFF.
data	Specifies the value for the data field. This value is compared with the selected register.
hr_data	Specifies the HR delay. Default: 0.

Execution

```

If (Angle Comp. bit == 0 OR (Angle Comp. bit == 1 AND NAF_global == 1))
{
    If (Selected register value == Immediate data field value)
    {
        If (hr_lr bit == 0)
        {
            If (Enable Pin action == 1)
            {
                Selected Pin = Pin Action AT next loop resolution clock + HR delay;
            }
        }
        else
        {
            If (Enable Pin action == 1)
            {
                Selected Pin = Pin Action AT next loop resolution clock;
            }
        }

        If (Z == 1 AND Opposite action == 1)
        {
            If (Enable Pin action == 1)
            {
                Selected Pin = opposite Pin Action AT next loop resolution clock;
            }

            If (Interrupt Enable == 1) HETFLG[n] = 1;          /* n depends on address */
            If ([C28:C27] == 01) Generate request on request line [P25:P23];
            If ([C28:C27] == 11) Generate quiet request on request line [P25:P23];

            If (register R is selected) R register = Compare value (32 bit);
            If (register S is selected) S register = Compare value (32 bit);
            If (register T is selected) T register = Compare value (32 bit);

            Jump to Conditional Address;
        }
    }
    elseif (Z == 1 AND Opposite action == 1)
    {
        If (Enable Pin action == 1)
        {
            Selected Pin = opposite Pin Action AT next loop resolution clock;
        }
        Jump to Next Program Address;
    }
    else // Angle Comp. bit == 1 AND NAF_global == 0
    {
        Jump to Next Program Address;
    }
}

```

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).

23.6.3.12 ECNT (Event Count)

Syntax

```
ECNT {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    [control={OFF | ON}]
    [prv={OFF | ON}]
    [cond_addr={label | 9-bit unsigned integer}]
    pin={pin number}
    event={NAF | FALL | RISE | BOTH | ACCUHIGH | ACCULOW}
    [reg={A | B | R | S | T | NONE}]
    [irq={OFF | ON}]
    [data={25-bit unsigned integer}]
}
```

Figure 23-147. ECNT Program Field (P31:P0)

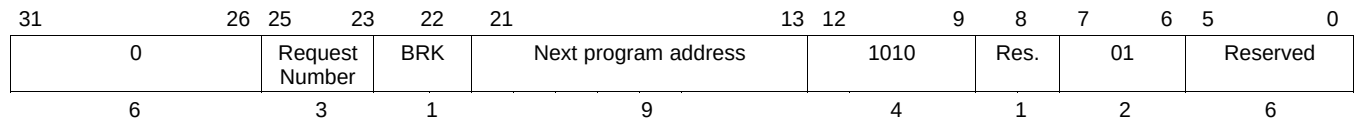


Figure 23-148. ECNT Control Field (C31:C0)

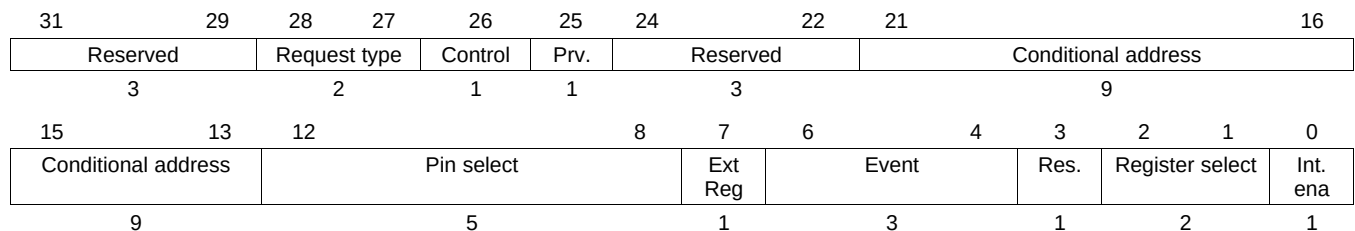
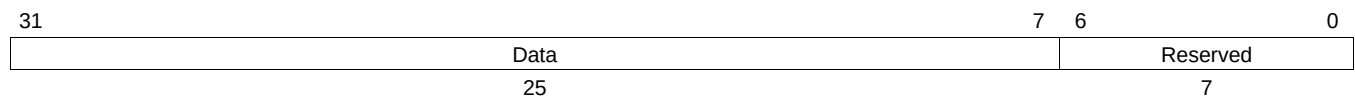


Figure 23-149. ECNT Data Field (D31:D0)



Cycles

One cycle

Register modified

Selected Register (A, B, R, S, T or none)

Description

This instruction defines a specialized 25-bit virtual counter used as an event counter or pulse accumulator (see [Table 23-85](#)). The counter value is stored in the data field [D31:D7] and the selected register. If one of the 32-bit registers (R,S,T) is selected, the 25 bit count value is stored left justified in the register with zeros in the seven least significant bits.

When an event count condition is specified, the counter value is incremented on a pin edge condition or on the NAF condition (NAF is defined in ACNT). This instruction can be used with all pins.

event The event that triggers the counter.

Table 23-85. Event Encoding Format for ECNT

Event	C6	C5	C4	Count Conditions	Mode	Int. Available
NAF	0	0	0	NAF flag is Set	Angle counter	Y
FALL	0	0	1	Falling edge on selected pin	Event counter	Y
RISE	0	1	0	Rising edge on selected pin	Event counter	Y
BOTH	0	1	1	Rising and Falling edge on selected pin	Event counter	Y
ACCUHIGH	1	0	-	while pin is high level	Pulse accumulation	N
ACCULOW	1	1	-	while pin is low level	Pulse accumulation	N

irq ON generates an interrupt when event in counter mode occurs. No interrupt is generated with OFF.
Default: OFF.

data 25-bit integer value serving as a counter.
Default: 0.

Execution

```

If (event occurs)
{
    If (Register A or B Selected) {
        Selected register = Immediate Data Field + 1;
    }

    If (Register R, S or T Selected)
    {
        Selected register[31:7] = Immediate Data Field + 1;
        Selected register[6:0] = 0;
    }

    Immediate Data Field = Immediate Data Field + 1;

    If (Interrupt Enable == 1) HETFLG[n] = 1; /* n depends on address */
    If ([C28:C27] == 01) Generate request on line [P25:P23];
    If ([C28:C27] == 11) Generate quiet request on line [P25:P23];

    Jump to Conditional Address;
}
else
{
    Jump to Next Program Address;
}

Prv bit = Current Logic (Lx) value of selected pin; (Always executed)

```

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).

23.6.3.13 MCMP (Magnitude Compare)

Syntax

```

MCMP {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    [hr_lr={LOW | HIGH}]
    [angle_comp={OFF | ON}]
    [savesub={OFF | ON}]
    [control={OFF | ON}]
    [en_pin_action={OFF | ON}]
    [cond_addr={label | 9-bit unsigned integer}]
    pin={pin number}
    order={REG_GE_DATA | DATA_GE_REG}
    [action={CLEAR | SET | PULSELO | PULSEHI}]
    reg={A | B | R | S | T | NONE}
    [irq={OFF | ON}]
    [data={25-bit unsigned integer}]
    [hr_data={7-bit unsigned integer}]
}

```

Figure 23-150. MCMP Program Field (P31:P0)

31	26	25	23	22	21	13	12	9	8	7	6	5	4	0
0	Request Number			BRK	Next program address				0000	hr_lr	Angle comp.	Res.	Save sub.	Res.
6	3			1	9				4	1	1	1	1	5

Figure 23-151. MCMP Control Field (C31:C0)

31	29	28	27	26	25	23	22	21	16					
Reserved			Request type		Control	Reserved			En. pin action	Conditional address				
3	2		1		3	1			9					
15	13	12	8				7	6	5	4	3	2	1	0
Conditional address			Pin select				Ext Reg	1	Order	Action		Register select		Int. ena
9	5				1	1	1	1	2	2		1		

Figure 23-152. MCMP Data Field (D31:D0)

31											7	6				0
Data											HR Data					
25											7					

Cycles One

Register modified T (if save sub bit P[5] is set)

Description	This instruction compares the magnitude of the 25-bit data value stored in the data field (D31-D7) and the 25-bit value stored in the selected ALU register (A, B, R, S, or T). If the hr_lr bit is reset, pin action will occur after a delay from the next loop resolution clock. If the hr_lr bit is set, the delay is ignored. This delay is programmed in the data field (D6-D0). When the data value matches, an output pin can be set or reset according to the pin action bit (C[4]). The pin will not change states if the enable pin action bit (C[22]) is reset. MCMP uses the zero flag set to generate opposite pin action (synchronized to the loop resolution clock). The save sub bit (P[5]) provides the option to save the result of a subtraction into register T.
--------------------	---

NOTE: The Difference Between Compare Values

The difference between the two data values must not exceed $(2^{24}) - 1$.

angle_comp	Determines whether or not an angle compare is performed. A value of ON causes the comparison to be performed only if the new angle flag is set (NAF = 1). If OFF is specified, the compare is then performed regardless of the state of the new angle flag. Default: OFF.
savesub	When set, the comparison result is saved into the T register (upper 25 bits). Default: OFF.
order	Specifies the order of the operands for the comparison.

Table 23-86. Magnitude Compare Order for MCMP

Order	C5	Description
REG_GE_DATA	0	Evaluates to true if the register value is greater than or equal to the data field value.
DATA_GE_REG	1	Evaluates to true if the data field value is greater than or equal to the register value.

irq	Specifies whether or not an interrupt is generated. A value of ON sends an interrupt if the compare match occurs according to the order selected. If OFF is selected, no interrupt is generated.
data	Specifies the value for the data field. This value is compared with the selected register.
hr_data	HR delay. The default value for an unspecified bit is 0.

Execution

```

If (Angle Compare P[7] == 0 OR (P[7] == 1 AND NAF_global == 1))
{
    If( (Order C[5] == 1) AND (Data[31:7]- Selected register[31:7]) >= 0))
    OR ( (Order C[5] == 0) AND Selected register[31:7] - Data[31:7]) >= 0))
    {
        If (Order C[5] == 1 AND Save subtract P[5] == 1)
        {
            Register T[31:7] = Data[31:7] - Selected register[31:7];
            Register T[6:0] = 0;
        }

        If (Order C[5] == 0 AND Save subtract P[5] == 1)
        {
            Register T[31:7] = Selected register[31:7] - Data[31:7];
            Register T[6:0] = 0;
        }

        If (Enable Pin Action C[22] == 1)
        {
            If (hr_lr P[8] = 0) {
                Schedule Action on Selected Pin C[12:8] at start of next loop
                + HR Delay D[6:0];
            }
            else
            {
                Schedule Pin Action on Selected Pin C[12:8] at start of next loop;
            }
        }

        If (Interrupt Enable == 1) HETFLG[n] = 1;          /* n depends on address */
        If ([C28:C27] == 01) Generate request on request line [P25:P23];
        If ([C28:C27] == 11) Generate quiet request on request line [P25:P23];

        Jump to Conditional Address;
    }
    else if (Z == 1 AND Opposite Action C[3] == 1 )
    {
        If (Enable Pin Action C[22] == 1)
        {
            Schedule Opposite Pin Action on Selected Pin C[12:8] at start of next loop;
        }

        Jump to Next Program Address;
    }
    else
        Jump to Next Program Address;
}
else // Angle Comp. bit == 1 AND NAF_global == 0
    Jump to Next Program Address;

```

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).

23.6.3.14 MOV32 (Data Move 32)

Syntax

```
MOV32 {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    remote={label | 9-bit unsigned integer}
    [control={OFF | ON}]
    [z_cond={OFF | ON}]
    [init={OFF | ON}] ON}}
    type={IMTOREG | IMTOREG&REM | REGTOREM | REMTOREG}
    [reg={A | B | R | S | T | NONE}]
    [data={25-bit unsigned integer}]
    [hr_data={7-bit unsigned integer}]
}
```

Figure 23-153. MOV32 Program Field (P31:P0)

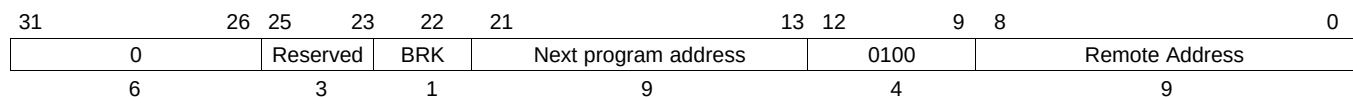


Figure 23-154. MOV32 Control Field (C31:C0)

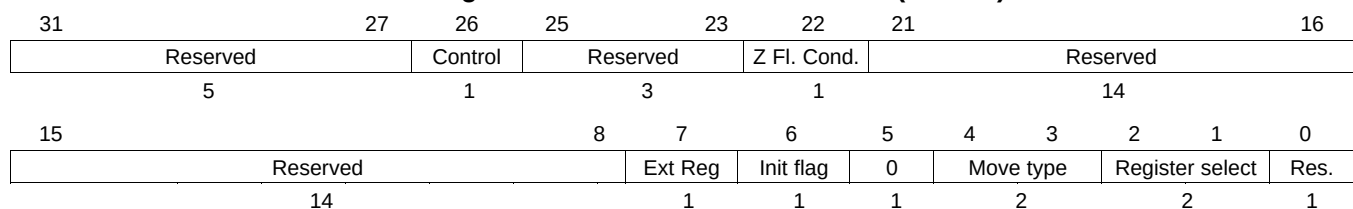
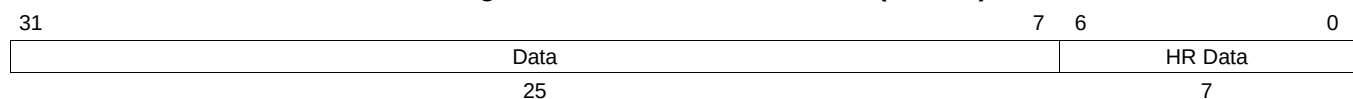


Figure 23-155. MOV32 Data Field (D31:D0)



Cycles One or two cycles

Register modified Selected register (A, B, R, S, or T)

Description MOV32 replaces the selected ALU register and/or the data field values at the remote address location depending on the move type.
[Figure 23-156](#) through [Figure 23-159](#) illustrate these operations. If *no register* is selected, the move is not executed, except for configuration C4:C3 = 01, where the remote data field is written with the immediate data field value.

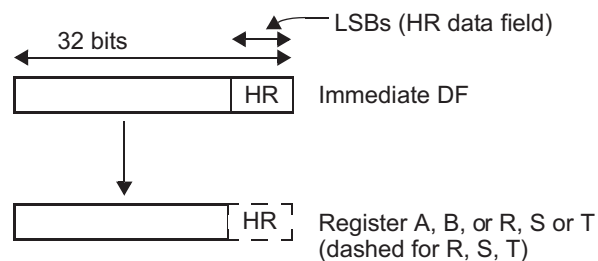
remote	Determines the location of the remote address. Default: Current instruction + 1.
z_cond	When set to OFF the MOV32 performs the move operation specified by the move type whenever it is executed (independent on the state of the Z-Flag). When set to ON the MOV32 performs the move operation specified by the move type only when the Z-Flag is set.
init	(Optional) Determines whether or not system flags are initialized. A value of ON reinitializes the following system flags to these states: Acceleration flag (ACF) = 0 Deceleration flag (DCF) = 1 Gap flag (GPF) = 0 New angle flag (NAF) = 0 A value of OFF results in no change to the system flags.
type	Specifies the move type to be executed.

Table 23-87. Move Type Encoding Selection

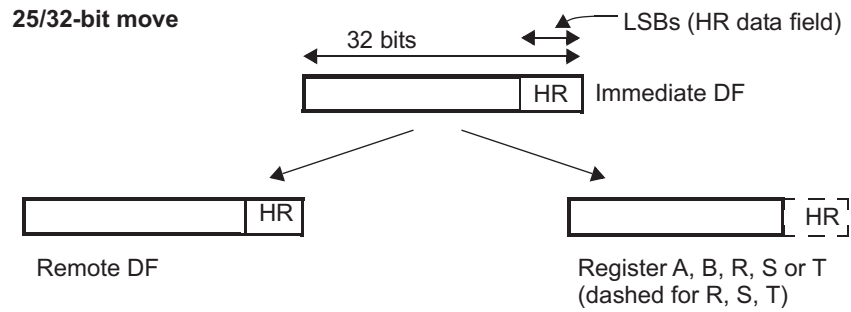
Move Type	C4	C3	Source	Destination(s)	Cycles
IMTOREG	0	0	Immediate data field	Register A, B, R, S, or T	1
IMTOREG&REM	0	1	Immediate data field	Remote data field and register A, B, R, S, or T	1
REGTOREM	1	0	Register A, B, R, S, or T	Remote data field	1
REMTOREG	1	1	Remote data field	Register A, B, R, S, or T	2

Figure 23-156. MOV32 Move Operation for IMTOREG (Case 00)

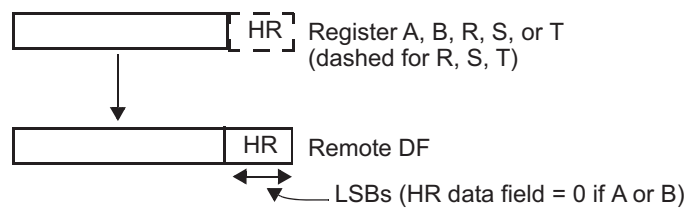
25/32-bit move



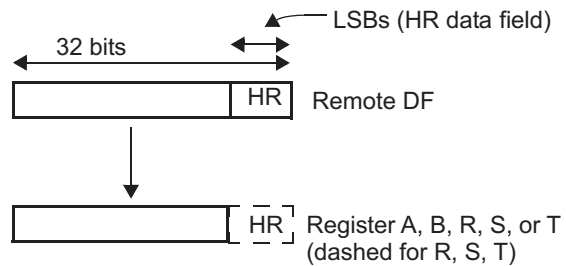
reg	Specifies which register (A, B, T, or NONE) is involved in the move. A register (A, B, or T) must be specified for every move type except IMTOREG&REM. If <i>NONE</i> is used with move type IMTOREG&REM, the MOV32 executes a move from the immediate data field to the remote data field. If <i>NONE</i> is used with any other move type, no move is executed.
data	Specifies a 25-bit integer value to be written to the remote data field or selected register.
hr_data	(Optional) HR delay. The default value for an unspecified bit is 0.

Figure 23-157. MOV32 Move Operation for IMTOREG&REM (Case 01)

Figure 23-158. MOV32 Move Operation for REGTOREM (Case 10)

25/32-bit move


Figure 23-159. MOV32 Move Operation for REMTOREG (Case 11)

25/32-bit move



Execution

```

If [(z_cond C[22] ==0) OR ((z_cond C[22] == 1) AND (Z Flag == 1))]
{
    switch (type C[4:3])
    {
        case 00: // IMTOREG
            Selected register = Immediate Data Field;
        case 01: // IMTOREG&REM
            Selected register = Immediate Data Field;
            Remote Data Field = Immediate Data Field;
        case 10: // REGTOREM
            Remote Data Field = Selected register;
        case 11: // REMTOREG
            Selected register = Remote Data Field;
    }
}

If (Init Flag == 1)
{
    ACF = 0;
    DCF = 1;
    GPF = 0;
    NAF = 0;
}
else
    All flags remain unchanged;

Jump to Next Program Address;

```

23.6.3.15 MOV64 (Data Move 64)

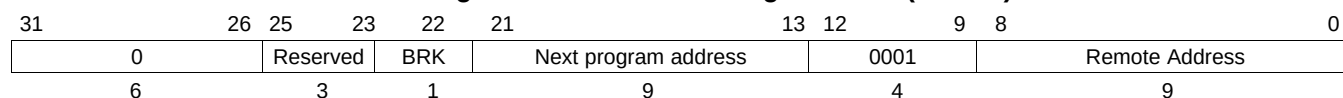
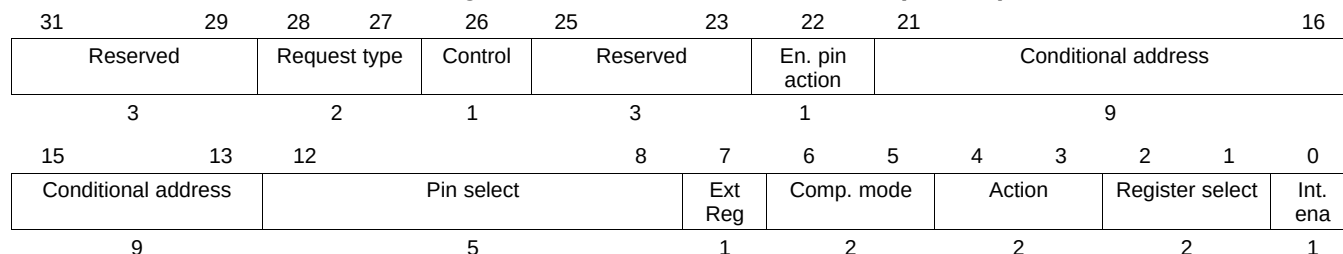
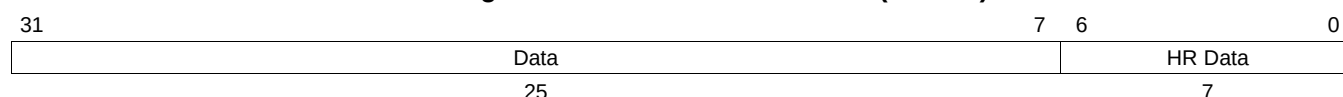
Syntax

```
MOV64 {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    remote={label | 9-bit unsigned integer}
    [request={NOREQ | GENREQ | QUIET}]
    [control={OFF | ON}]
    [en_pin_action={OFF | ON}]
    [cond_addr={label | 9-bit unsigned integer}]
    [pin={pin number}]
    comp_mode={ECMP | SCMP | MCMP1 | MCMP2}
    [action={CLEAR | SET | PULSELO | PULSEHI}]
    [reg={A | B | R | S | T | NONE}]
    [irq={OFF | ON}]
    [data={25-bit unsigned integer}]
    [hr_data= {7-bit unsigned integer}]
}
```

-or-

Syntax

```
MOV64 {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    remote={label | 9-bit unsigned integer}
    cntl_val={29-bit unsigned integer}
    [data={25-bit unsigned integer}]
    [hr_data= {7-bit unsigned integer}]
}
```

Figure 23-160. MOV64 Program Field (P31:P0)

Figure 23-161. MOV64 Control Field (C31:C0)

Figure 23-162. MOV64 Data Field (D31:D0)

Cycles

One

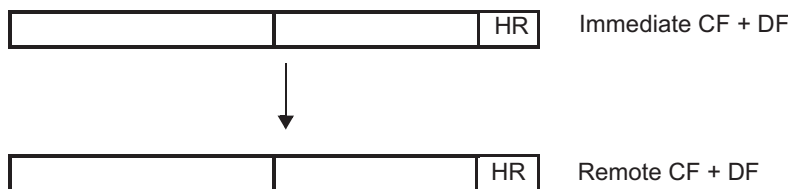
Register modified

None

Description

This instruction modifies the data field and the control field at the remote address.

MOV64 has two distinct syntaxes. In the first syntax, bit values may be set by assigning a value to each of the control fields. This syntax is convenient for modifying control fields that are arranged similarly to the format of the MOV64 control field. A second syntax, in which the entire 29-bit control field is specified by the cntl_val field, is convenient when the remote control field is dissimilar to the MOV64 control field. Either syntax may be used, but you must use one or the other but not a combination of syntaxes. See [Figure 23-163](#).

Figure 23-163. MOV64 Move Operation

Table 23-88. MOV64 Control Field Descriptions

request	Maintains the control field for the remote instruction.
control	Maintains the control field for the remote instruction.
en_pin_action	Maintains the control field for the remote instruction.
cond_addr	Maintains the control field for the remote instruction.
pin	Maintains the control field for the remote instruction.
register, ext reg	Maintains the control field for the remote instruction.
comp_mode	Selects the comparison mode type to be used by the remote instruction.

Table 23-88. MOV64 Control Field Descriptions (continued)

action	Maintains the control field for the remote instruction.
irq	Maintains the control field for the remote instruction.
data	Specifies the 25-bit initial count value for the data field. If omitted, the field defaults to 0.
hr_data	(Optional) HR delay. The default value for an unspecified bit is 0.

Table 23-89. Comparison Type Encoding Format

comp_mode	C[6]	C[5]	MCMP Order
ECMP	0	0	
SCMP	0	1	
MCMP1	1	0	REG_GE_DATA
MCMP2	1	1	DATA_GE_REG

Execution

Remote Data Field = Immediate Data Field;
 Remote Control Field = Immediate control Field;
 Jump to Next Program Address;

23.6.3.16 PCNT (Period/Pulse Count)

Syntax

```
PCNT {
    [hr_lr={HIGH | LOW}]
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    [irq={OFF | ON}]
    type={FALL2RISE | RISE2FALL | FALL2FALL | RISE2RISE}
    pin={pin number}
    [control={OFF | ON}]
    [prv={OFF | ON}]
    [period={25-bit unsigned integer}]
    [data={25-bit unsigned integer}]
    [hr_data= {7-bit unsigned integer}]
}
```

Figure 23-164. PCNT Program Field (P31:P0)

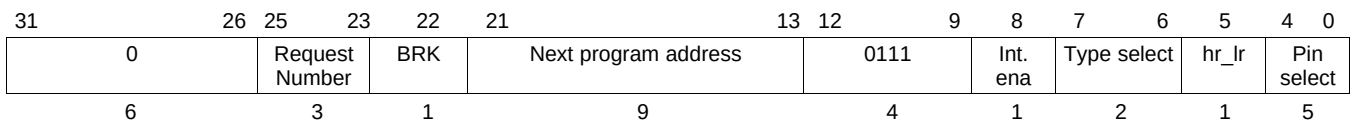


Figure 23-165. PCNT Control Field (C31:C0)

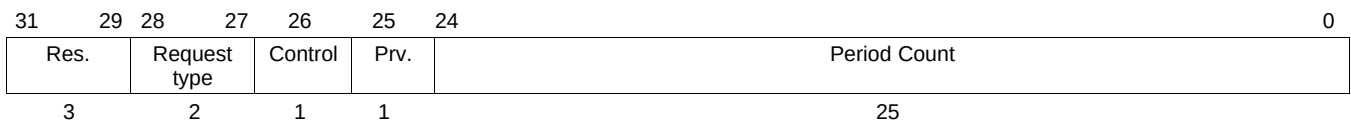
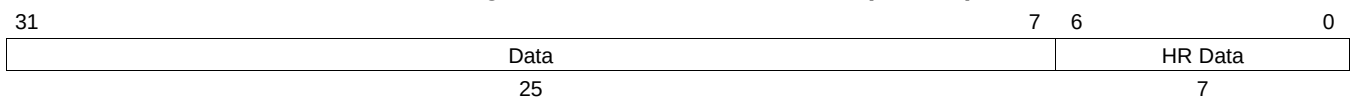


Figure 23-166. PCNT Data Field (D31:D0)



Cycles One

Register modified Register A

Description This instruction detects the edges of the external signal at loop start and measures its period or pulse duration. The counter value stored in the control field C[24:0] and in the register A is incremented each N2HET loop. PCNT uses the HR structure on the pin to measure an HR period/pulse count value.

hr_lr (Optional) Specifies whether the PCNT instruction captures the HR delay into the HR data field on the selected edge condition. If hr_lr is 0 (HIGH) then PCNT captures the HR delay. if hr_lr is 1 (LOW) then PCNT only captures at loop resolution.

irq	(Optional) Specifies whether or not an interrupt is generated. A value of ON sends an interrupt when a new value is captured. If OFF is selected, no interrupt is generated.
type	(Optional) Determines the type of counter that is implemented.

Table 23-90. Counter Type Encoding Format

	P7	P6	Period/Pulse Select	Reset On	Capture On
FALL2RISE	0	0	Count low-pulse duration on selected pin	Falling edge	Rising edge
RISE2FALL	0	1	Count high-pulse duration on selected pin	Rising edge	Falling edge
FALL2FALL	1	0	Count period between falling edges on selected pin	Falling edge	Falling edge
RISE2RISE	1	1	Count period between rising edges on selected pin	Rising edge	Rising edge

period	Specifies the 25-bit integer value that holds the counter value. The counter value is also stored in register A. Default: 0.
data	25-bit integer representing the last captured counter value. Default: 0.
hr_data	HR delay. Default: 0.

If *period-measure* is selected, PCNT captures the counter value into the period/pulse data field [D31:D7] on the selected edge. The HR structure provides HR capture field [D6:D0]. The counter value [C24:C0] is reset on the same edge. The captured period value is a 32-bit value.

If *pulse-measure* is selected, PCNT captures the counter value into the period/pulse count field [D31:D7] on the selected edge. The HR structure provides HR capture field [D6:D0]. The counter value [C24:C0] is reset on the next opposite edge. The captured pulse value is a 32-bit value.

When the overflow count (all 1's in the counter value) is reached, PCNT stops counting until the next reset edge is detected.

Note: For FALL2FALL/RISE2RISE, the user should always discard the first interrupt/HTU request if interrupt/request are enabled before HET_ON. For both the types, reset edge and capture edge are the same and the interrupt or HTU request is triggered on capture edge (which is nothing but the reset edge). Once the execution unit is enabled, the first edge generates an interrupt but the value of the counter is of no use as this is not the period between 2 edges. So first edge after turning on N2HET is used mainly for resetting the counter and start the period count.

Execution

```

Z = 0;

If (Period C[24:0] != 1FF_FFFFh) {
    Period C[24:0] = Period C[24:0] + 1;
}

Register A = Period C[24:0];

If (specified capture edge detected on selected pin)
{
    Z = 1;

    If (Period value != 1FF_FFFFh)
    {
        HR Capture Value = selected HR counter;
    }
    else
    {
        HR Capture Value = 7Fh;
    }

    If (Interrupt Enable == 1) HETFLG[n] = 1;      /* n depends on address */
    If ([C28:C27] == 01) Generate request on request line [P25:P23];
    If ([C28:C27] == 11) Generate quiet request on request line [P25:P23];
}

If (specified reset edge detected on selected pin)
{
    Period value = 0000000h;
}

Prv bit = Current Logic (Lx) value of selected pin;

Jump to Next Program Address;

```

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).

23.6.3.17 PWCNT (Pulse Width Count)

Syntax

```

PWCNT {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    [hr_lr={HIGH | LOW}]
    [control={OFF | ON}]
    [cond_addr={label | 9-bit unsigned integer}]
    [en_pin_action={OFF | ON}]
    pin ={pin number}
    [action={CLEAR | SET | PULSELO | PULSEHI}]
    [reg={A | B | T | NONE}]
    [irq={OFF | ON}]
    [data={25-bit unsigned integer}]
    [hr_data={7-bit unsigned integer}]
}
    
```

Figure 23-167. PWCNT Program Field (P31:P0)

31	26	25	23	22	21	13	12	9	8	7	6	5	0
0	Request Number		BRK	Next program address				1010	hr_lr	11	Reserved		
6	3		1	9				4	1	2	6		

Figure 23-168. PWCNT Control Field (C31:C0)

31	29	28	27	26	25	23	22	21	16				
Reserved		Request type		Control	Reserved		En. pin action	Conditional address					
3		2		1	3		1	9					
15	13	12	8		7	5		4	3	2	1	0	
Conditional address		Pin select				Reserved		Action		Register select		Int. ena	
9		5				3		2		2		1	

Figure 23-169. PWCNT Data Field (D31:D0)

31											7	6											0							
										Data										HR Data										
										25																				7

Cycles	One
Register modified	Selected register (A, B or T)
Description	This instruction defines a virtual timer used to generate variable length pulses. The counter value stored in the data field is decremented unconditionally on each timer resolution until it reaches zero, and it then stays at zero until it is reloaded with a non-zero value. The specified pin action is performed as long as the count after count value is decremented is greater than 0. The opposite pin action is performed when the count after decrement just reaches 0. If the hr_lr bit is reset, the opposite pin action will be taken after a HR delay from the next loop resolution clock. If the hr_lr bit is set, the delay is ignored. This delay is programmed in bits [D6:D0].
irq	ON generates an interrupt when the data field value reaches 0. No interrupt is generated for OFF. Default: OFF.
data	25-bit integer value serving as a counter.
hr_data	HR delay. Default: 0.

Execution

```

If (Data field value == 0)
{
    Selected register = 0;
    Jump to Next Program Address;
}

If (Data field value > 1)
{
    Selected register = Data field value - 1;
    Data field value = Counter value - 1;

    If (Enable Pin action == 1)
    {
        Selected Pin = Pin Action AT next loop resolution clock;
    }

    Jump to Next Program Address;
}

If (Data field value == 1)
{
    Selected register = 00000000h;
    Data field value = 00000000h;

    If (Opposite action == 1)
    {
        If (hr_lr bit == 0)
        {
            If (Enable Pin action == 1)
            {
                Selected Pin = Opposite level of Pin Action AT next loop resolution clock
                + HR delay;
            }
        }
        else
        {
            If (Enable Pin action == 1)
            {
                Selected Pin = Opposite level of Pin Action AT next loop
                resolution clock;
            }
        }

        If (Interrupt Enable == 1) HETFLG[n] = 1;          /* n depends on address */
        If ([C28:C27] == 01) Generate request on request line [P25:P23];
        If ([C28:C27] == 11) Generate quiet request on request line [P25:P23];
    }

    Jump to Conditional Address
}

```

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).

23.6.3.18 RADM64 (Register Add Move 64)

Syntax

```
RADM64 {
[brk={OFF | ON}]
[next={label | 9-bit unsigned integer}]
[remote={label | 9-bit unsigned integer}]
[request={NOREQ | GENREQ | QUIET}]
[control={OFF | ON}]
[en_pin_action={OFF | ON}]
[cond_addr={label | 9-bit unsigned integer}]
[pin={pin number}]
comp_mode={ECMP | SCMP | MCMP1 | MCMP2}
[action={CLEAR | SET | PULSELO | PULSEHI}]
[reg={A | B | R | S | T | NONE}]
[irq={OFF | ON}]
[data={25-bit unsigned integer}]
[hr_data= {7-bit unsigned integer}]
}
```

-or-

Syntax

```
RADM64 {
[brk={OFF | ON}]
[next={label | 9-bit unsigned integer}]
[remote={label | 9-bit unsigned integer}]
cntl_val={29-bit unsigned integer}
[data={25-bit unsigned integer}]
[hr_data= {7-bit unsigned integer}]
}
```

Figure 23-170. RADM64 Program Field (P31:P0)

31	26	25	23	22	21	13	12	9	8	0
0	Reserved	BRK	Next program address	0011	Remote Address					
6	3	1	9	4	9					

Figure 23-171. RADM64 Control Field (C31:C0)

31	29	28	27	26	25	23	22	21	16				
Reserved		Request type		Control	Reserved		En. pin action	Conditional address					
3		2		1	3		1	9					
15	13	12			8	7	6	5	4	3	2	1	0
Conditional address		Pin select				Ext Reg	Comp. mode		Action		Register select		Int. ena
9		5				1	2		2		2		1

Figure 23-172. RADM64 Data Field (D31:D0)

31	7	6	0
Data	HR Data		
25	7		

Cycles

Normally One Cycle. Two cycles if writing to remote address that is also the next address.

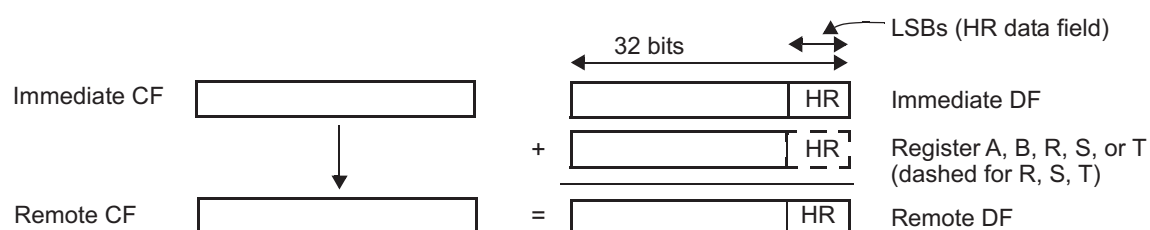
Register modified

None

Description

This instruction modifies the data field, the HR data field and the control field at the remote address. The advantage over DADM64 is that It executes one cycle faster. In case the R, S, or T register is selected, the addition is a 32-bit addition. The table description shows the bit encoding for determining which ALU register is selected.

RADM64 has two distinct syntaxes. In the first syntax, bit values may be set by assigning a value to each of the control fields. This syntax is convenient for modifying control fields that are arranged similar to the format of the RADM64 control field. A second syntax, in which the entire 29-bit control field is specified by the `cntl_val` field, is convenient when the remote control field is dissimilar from the RADM64 control field. Either syntax may be used, but you must use one or the other but not a combination of syntaxes. See [Figure 23-173](#).

Figure 23-173. RADM64 Add and Move Operation

comp_mode

Selects the comparison mode type to be used.

Table 23-91. Comparison Type Encoding Format

comp_mode	C[6]	C[5]	MCMP Order
ECMP	0	0	
SCMP	0	1	
MCMP1	1	0	REG_GE_DATA
MCMP2	1	1	DATA_GE_REG

Table 23-92. RADM64 Control Field Descriptions

request	Maintains the control field for the remote instruction.
Control	Maintains the control field for the remote instruction.
en_pin_action	Maintains the control field for the remote instruction.
cond_addr	Maintains the control field for the remote instruction.
pin	Maintains the control field for the remote instruction.
register	Maintains the control field for the remote instruction.
action	Maintains the control field for the remote instruction.
irq	Maintains the control field for the remote instruction.
data	Specifies the 25-bit initial value for the data field. If omitted, the field defaults to 0.
hr_data	Seven least significant bits of the 32-bit data field. Default: 0.
cntl_val	Specifies the 29 least significant bits of the Control field.

Execution

Remote Data Field = Selected register + Immediate Data Field (including HR field);
 Remote Control Field = Immediate Control Field;
 Jump to Next Program Address;

23.6.3.19 RCNT (Ratio Count)

Syntax

```
RCNT {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [control={OFF | ON}]
    divisor={25-bit unsigned integer}
    [data={25-bit unsigned integer}]
}
```

Figure 23-174. RCNT Program Field (P31:P0)

31	26	25	23	22	21	13	12	9	8	7	6	5	4	3	1	0
0		Reserved		BRK	Next program address				1010	Res.	00	Step width	Res.	1		
6		3		1	9				4	1	2	2		3	1	

Figure 23-175. RCNT Control Field (C31:C0)

31	27	26	25	24	0
Reserved			Control	Res.	Divisor
5	1		1	25	

Figure 23-176. RCNT Data Field (D31:D0)

31	7	6	0
Data			Reserved
25			7

Cycles Two Cycles (One Cycle if T=0)

Register modified None

Description RCNT is used with other instructions to convert an input period measurement T_{Input} to the form of (Equation 35) where the input period is expressed as a fraction of a reference period $T_{\text{Reference}}$.

$$T_{\text{Input}} = T_{\text{Reference}} \cdot \left(\frac{N}{M} \right) \quad (35)$$

RCNT computes the numerator N of (Equation 35). The denominator M of (Equation 35) is a constant that is of interest. For example, choosing $M = 100$ allows the input period to be expressed as a percentage (%) of the reference period. Note that if $T_{\text{Input}} > T_{\text{Reference}}$, then RCNT will return $N > M$; which would be correct if, for example, the input pulse period is 110% of the reference pulse period.

RCNT expects that register T is loaded with the value of $T_{\text{Reference}}$. The input period T_{Input} is determined by counting the number of loop resolution periods between edges on the input pin. This information is conveyed through the Z flag from a PCNT instruction that precedes the RCNT instruction.

The divisor field of the RCNT instruction should be chosen as:
 $\text{Divisor} = M \cdot l_r$, where M is the desired denominator from (Equation 35) and l_r is the loop resolution prescale value.

An example N2HET program that makes use of the RCNT instruction is:

```
L0: MOV32 { remote=dummy, type=IMTOREG, reg=T, data=0x8, hr_data=0};

L1: PCNT { hr_lr=HIGH, brk=OFF, type=FALL2FALL, pin=0};

L2: RCNT { divisor=320, data=0x4};

L3: BR {cond_addr=L5, event = Z}

L4: ADC { src1=ZERO, src2=IMM, dest=IMM, next=L0, data=0, hr_data=0};

L5: ADD { src1=REM, src2=ZERO, dest=IMM, remote=L4, data=0, hr_data=0};

L6: ADD { src1=ZERO, src2=ZERO, dest=NONE, rdest=REM,
        next=L0, remote=L4, data=0, hr_data=0};
```

dummy

In this small program an input signal on pin 0 is measured both in terms of absolute cycles by the PCNT instruction at L1 and as in 1/10ths of the reference period by the RCNT instruction at L2. In this example the reference period is a constant 0x400 cycles; this value is loaded into register T by the MOV32 instruction at L0. (0x400 is data=8, hr_data=0)

RCNT follows PCNT and is initialized to a working count of T/2 (0x200) whenever the PCNT instruction detects a falling edge on pin 0. Between falling edges on pin0, RCNT accumulates counts 10x faster than PCNT; so that the working data field of RCNT will reach the reference value of 0x400 in 1/10th the time that a PCNT instruction would. Each time the RCNT instruction passes the reference value, it sets the carry out flag and subtracts the reference value from the working count. By accumulating carry-outs from RCNT, the add with carry instruction at L4 effectively counts in increments of 1/10th of the reference period. Note that the divisor value 320 is 10 times 32; this assumes lr=32.

When the next falling edge is detected on pin 0, PCNT sets the Z flag and the RCNT instruction resets again to the initial data field of T/2. RCNT does not modify the Z flag, so that the branch instruction at L3 can execute instructions at L5, L6 instead of L4. The instructions at L5 and L6 capture the final result from L4 and reset the ADC instruction at L4 to zero for the start of the next period measurement.

Execution

```
If (register T[31:0] != 00000000h)
{
    C = 0;

    If (Z == 0)
    {
        Data Field[31:0] = Data Field[31:0] + Divisor[24:0];

        If (Data Field[31:0] >= Reg T[31:0])
        {
            Data Field[31:0]=Data Field[31:0] - Reg T[31:0];
            C = 1;
        }
    }
    else
    {
        Data Field[31:0] = T[31:0] >> 1; /* T/2 */
    }
}
Jump to Next Program Address;
```

23.6.3.20 SCMP (Sequence Compare)

Syntax

```

SCMP {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    [control={OFF | ON}]
    [en_pin_action={OFF | ON}]
    cond_addr={label | 9-bit unsigned integer}
    pin ={pin number}
    [action={CLEAR | SET}]
    [restart={OFF | ON}]
    [irq={OFF | ON}]
    [data={25-bit unsigned integer}]
}

```

Figure 23-177. SCMP Program Field (P31:P0)

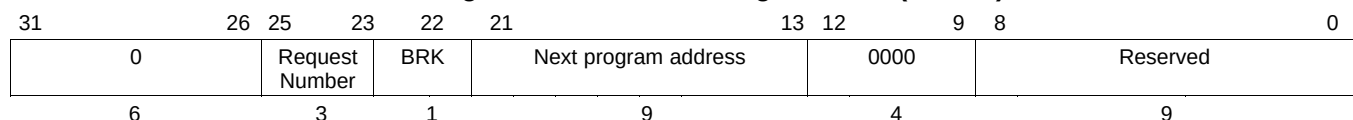


Figure 23-178. SCMP Control Field (C31:C0)

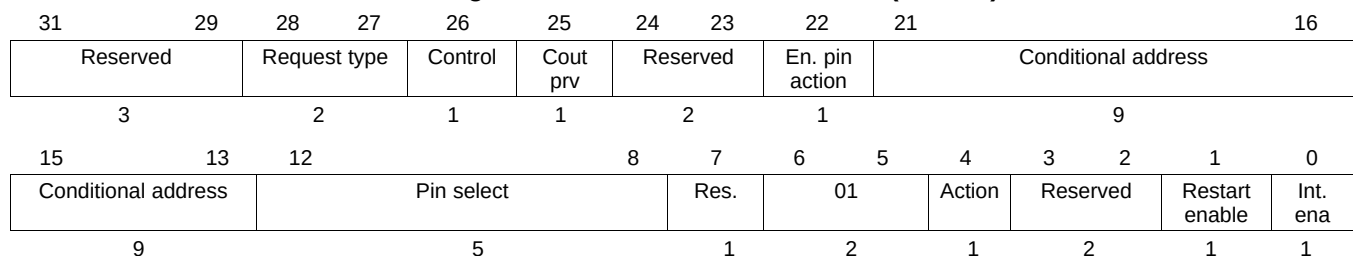
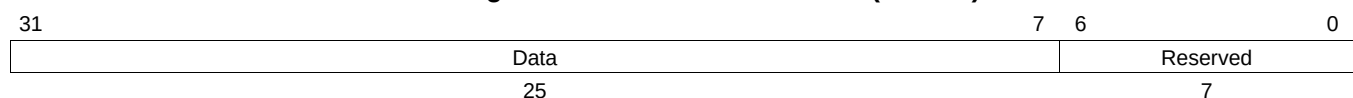


Figure 23-179. SCMP Data Field (D31:D0)



Cycles One

Register modified Register T (implicitly)

Description This instruction alternately performs angle- and time-based operations to generate pulse sequences, using the angle referenced time base. These pulse sequences last for a relative duration using a free running time base. Generally, register B holds the angle values and register A holds the time values. Bit 0 of the conditional address field (C13) specifies whether the instruction is operating in angle or time operation mode.

When the compared values match in angle mode, a pin can be set or reset according to the pin action bit (C4). The pin does not change states if the enable pin action bit (C22) is reset.

The restart enable bit (C1) provides the option to unconditionally restart a sequence using the X-flag bit of ACMP.

restart	If restart is set to ON and the X flag = 1, the assembler writes a value of 1 into the immediate index field, writes the value in register A into the immediate data field, and jumps to the next program address. The X flag is set or cleared by the ACMP instruction. If restart is set to OFF, the X flag is ignored; no special action is performed. Default: OFF.
irq	ON generates an interrupt if the compare match occurs in angle mode. No interrupt is generated when the field is OFF. Default: OFF.
data	Specifies the 25-bit compare value.
cond_addr	Since the LSB of the conditional address is used to select between time mode and angle mode, and since the conditional address is taken only in time mode, the destination for the conditional address must be odd.

Execution

```

If (Data field value <= Selected register value) Cout = 0; else Cout = 1;

If (Restart Enable == 1 AND X == 1)
{
    C13 = 1;
    Immediate Data Field = Register A;
    Cout = 0;
    Jump to Next Program Address;
}

If (Angle Mode (C13 == 0) AND ((Restart En. == 1 AND X == 0) OR Restart En. == 0))
{
    If (Z == 0 AND (Register B value - Angle Inc. < Data field value) AND Cout == 0) OR
        (Z == 1 AND (Cout_prv == 1 OR Cout == 0))
    {
        If (Enable Pin Action == 1) Selected Pin = Pin Action;
        If (Interrupt Enable == 1) HETFLG[n] = 1; /* n depends on address */
        If ([C28:C27] == 01) Generate request on request line [P25:P23];
        If ([C28:C27] == 11) Generate quiet request on request line [P25:P23];

        Immediate Data Field = Register A;
        C13 = 1; /* switch to Time Mode */
    }
    Jump to Next Program Address;
}
Else If (Time Mode (C13 == 1)) AND ((Restart En. == 1 AND X == 0) OR Restart En. == 0)
{
    /* Result of subtract must not exceed 2^24 - 1 */
    Register T = Register A - Immediate Data Field;
    Jump to Conditional Program Address;
}
Cout_prv = Cout; (always executed)

```

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).

23.6.3.21 SCNT (Step Count)

Syntax

```

SCNT {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    step={8 | 16 | 32 | 64}
    [control={OFF | ON}]
    gapstart={25-bit unsigned integer}
    [data={25-bit unsigned integer}]
}

```

Figure 23-180. SCNT Program Field (P31:P0)

31	26	25	23	22	21	13	12	9	8	7	6	5	4	3	1	0	
0		Reserved		BRK	Next program address					1010		Res.	00	Step width	Res.	1	
6		3		1	9					4		1	2	2	3		1

Figure 23-181. SCNT Control Field (C31:C0)

31	27	26	25	24	0
Reserved			Control	Res.	Gap start
5	1		1	25	

Figure 23-182. SCNT Data Field (D31:D0)

31	7	6	0
Data			Reserved
25			7

Cycles	One or two cycles (two cycles when DF is involved in the calculations)
Register modified	Register A
Description	This instruction can be used only once in a program and defines a specialized virtual timer used after APCNT and before ACNT to generate an angle-referenced time base synchronized to an external signal (that is, a toothed wheel signal) as defined in APCNT and ACNT. Step width selection bits are saved in two flags, SWF0, and SWF1, to be re-used in ACNT. SCNT multiplies the frequency of the external signal by a constant K defined in the step width field, [P5:P4]. The bit encoding for this field is defined in Table 23-93.
step	Specifies the step increment to be added to the counter value each program resolution. These two bits provide the values for the SWF0 and SWF1 flags. The valid values are listed in Table 23-93 .

Table 23-93. Step Width Encoding for SCNT

P5	P4	Step Width (K)
0	0	8
0	1	16
1	0	32
1	1	64

gapstart	Defines the gap start angle, which SCNT writes to register A. The gap start value has no effect on the SCNT instruction, but if the ACNT instruction is being used, register A must contain the correct gap start value. For a typical toothed wheel gear: $GAPSTART = (stepwidth \times (actual\ teeth\ on\ gear - 1)) + 1.$
data	Specifies the 25-bit integer value serving as a counter. Default: 0.

This instruction is incremented by the step value K on each timer resolution up to the previous period value P(n-1) measured by APCNT (stored in register T). The resulting period of SCNT is: $P(n - 1)/K$

Due to stepping, the final count of SCNT will not usually exactly match the target p(n-1). SCNT compensates for this error by starting each cycle with the remainder of the previous cycle.

When SCNT reaches the target p(n-1), the zero flag is set as an increment condition for ACNT. SCNT also specifies a gap start angle, defining the start of a range in ACNT where period measurements in APCNT are temporarily stopped to mask singularities in the external signal.

SCNT uses register A to store the gap start value. Gap start has no effect for SCNT.

Execution

```

SWF1 = P5;
SWF0 = P4;
Z = 0;

If (register T != 0000000h)
{
    If (DCF == 1 OR ACF == 1)
    {
        Data Field register = 0000000h;
        Counter value = 0000000h;
    }

    If (DCF == 0 AND ACF == 0)
    {
        Data Field register = Data field register + Step Width;
    }

    If ((Data Field register - register T) >= 0)
    {
        Data field register = Data Field register - register T;
        Z = 1;
    }

    Register A = Gap start value;
}

Jump to Next Program Address;

```

23.6.3.22 SHFT (Shift)

Syntax

```

SHFT {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    smode={OR0 | OL0 | OR1 | OL1 | ORZ | OLZ | IRM | ILL | IRZ | ILZ}
    [control={OFF | ON}]
    [prv={OFF | ON}]
    [cond_addr={label | 9-bit unsigned integer}]
    cond={UNC | FALL | RISE}
    pin ={pin number}
    [reg={A | B | R | S | T | NONE}]
    [irq={OFF | ON}]
    [data={25-bit unsigned integer}]
}

```

Figure 23-183. SHFT Program Field (P31:P0)

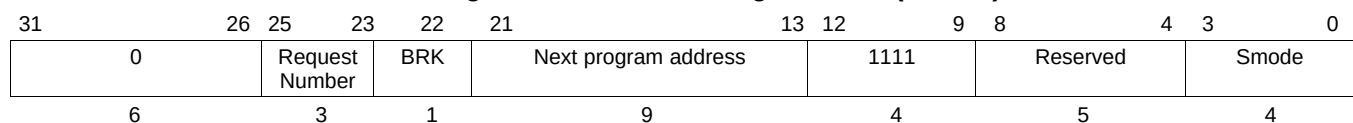


Figure 23-184. SHFT Control Field (C31:C0)

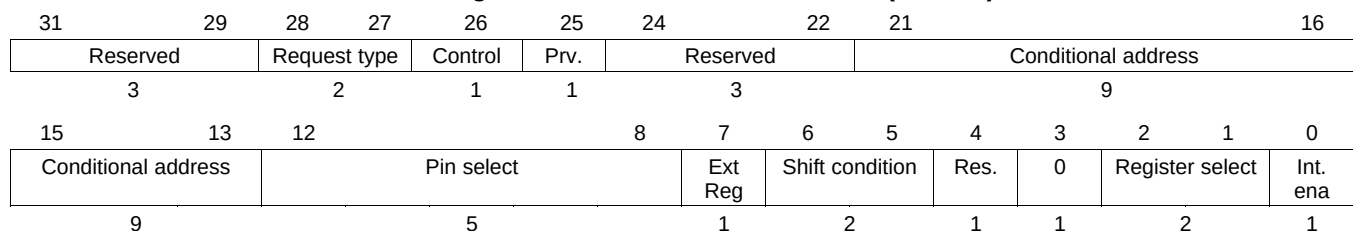
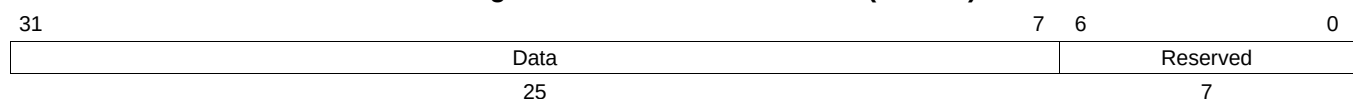


Figure 23-185. SHFT Data Field (D31:D0)



Cycles One

Register modified Selected register (A, B, R, S or T)

Description This instruction shifts the data field of the Instruction. N2HET pins can be used for data in or data out. SHFT includes parameters to select the shift direction (in, out, left, right), shift condition (shift on a defined clock edge on HET[0] or shift always), register for data storage (A, B, R, S or T), and the data pin.

smode Shift mode

Table 23-94. SHIFT MODE Encoding Format

smode	P3	P2	P1	P0	Operation	
OR0	0	0	0	0	Shift Out / Right	LSB 1st on HETx / 0 into MSB
OL0	0	0	0	1	Shift Out / Left	MSB 1st on HETx / 0 into LSB
OR1	0	0	1	0	Shift Out / Right	LSB 1st on HETx / 1 into MSB
OL1	0	0	1	1	Shift Out / Left	MSB 1st on HETx / 1 into LSB
ORZ	0	1	0	0	Shift Out / Right	LSB 1st on HETx / Z into MSB
OLZ	0	1	0	1	Shift Out / Left	MSB 1st on HETx / Z into LSB
IRM	1	0	0	0	Shift In / Right	HETx into MSB
ILL	1	0	0	1	Shift In / Left	HETx into LSB
IRZ	1	0	1	0	Shift In / Right	HETx in MSB / LSB into Z
ILZ	1	0	1	1	Shift In / Left	HETx in LSB / MSB into Z

cond Specifies the shift condition.

Table 23-95. SHIFT Condition Encoding

C6	C5	Shift Condition
0	X	Always
1	0	Rising edge of HET[0]
1	1	Falling edge of HET[0]

irq ON generates an interrupt if the Z flag is set. A value of OFF does not generate an interrupt.
Default: OFF.

data Specifies the 25-bit value for the data field.

Execution

```

If (SHIFT condition == 0X)
OR (SHIFT condition == 10 AND HET[0] rising edge)
OR (SHIFT condition == 11 AND HET[0] falling edge)
{
    If ([P3:P2] == 00)
    {
        If ((Immediate Data Field == all 0's AND [P3:P0] == 000X)
            OR (Immediate Data Field == all 1's AND [P3:P0] == 001X))
        {
            Z = 1;
        }
        else
        {
            Z = 0;
        }
    }
    else If ([P3:P0] == 1010)
    {
        Z = LSB of the Immediate Data Field;
    }
    else if ([P3:P0] == 1011)
    {
        Z = MSB of the Immediate Data Field;
    }
}

If( (Immediate Data Field == all 0's) OR
    (Immediate Data Field == all 1's))
{
    if (Interrupt Enable == 1)
    {
        HETFLG[n] = 1;      /* n depends on address */
    }
    Jump to Conditional Address;
}
else
{
    Jump to Next Program Address;
}

Prv. bit = HET[0] Pin level; (Always executed)

Shift Immediate Data Field once according to P[3:0];

Immediate Data Field = Result of the shift;

Selected register = Result of the shift;

Jump to Next Program Address;

```

NOTE: The immediate data field evaluates all 0s or all 1s and is performed before the shift operation.

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).

23.6.3.23 WCAP (Software Capture Word)

Syntax

```

WCAP {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    [hr_lr={HIGH | LOW}]
    [control={OFF | ON}]
    [prv={OFF | ON}]
    [cond_addr={label | 9-bit unsigned integer}]
    pin ={pin number}
    event={NOCOND | FALL | RISE | BOTH}
    reg={A | B | R | S | T | NONE}
    [irq={OFF | ON}]
    [data={25-bit unsigned integer}]
    [hr_data={7-bit unsigned integer}]
}
    
```

Figure 23-186. WCAP Program Field (P31:P0)

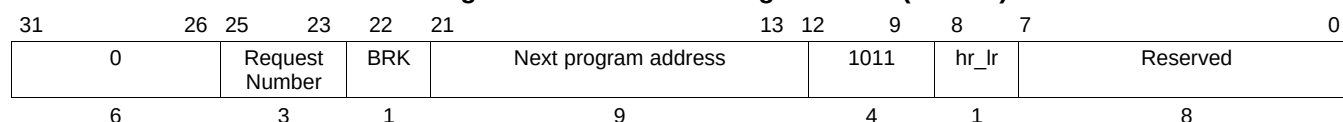


Figure 23-187. WCAP Control Field (C31:C0)

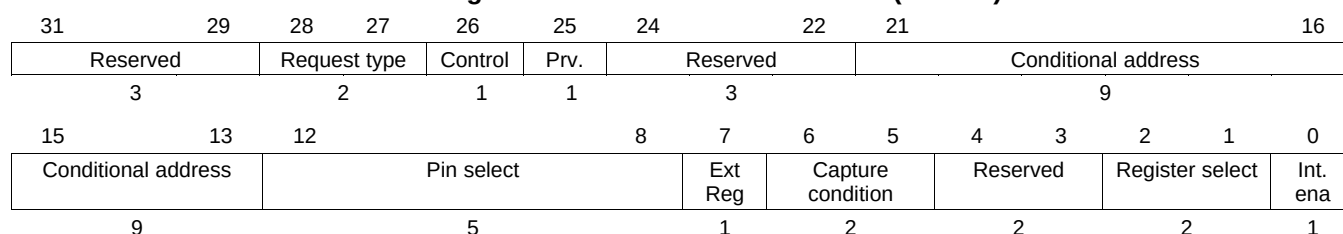
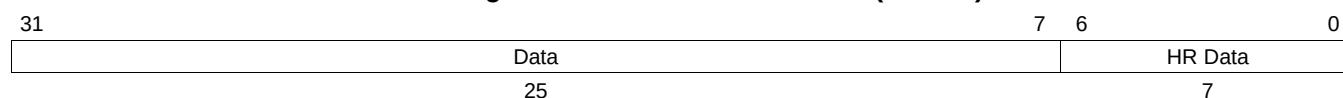


Figure 23-188. WCAP Data Field (D31:D0)



Cycles One

Register modified None

Description

This instruction captures the selected register into the data field if the specified capture condition is true on the selected pin. This instruction can be used with all pins.

If the hr_lr bit is reset, the WCAP instruction will capture an HR time stamp into the data field on the selected edge condition. If the hr_lr bit is set, the HR capture is ignored.

event Specifies the event that triggers the capture.

Table 23-96. Event Encoding Format for WCAP

C6	C5	Capture Condition
0	0	Always
0	1	Capture on falling edge
1	0	Capture on rising edge
1	1	Capture on rising and falling edge

irq ON generates an interrupt when the capture condition is met. No interrupt is generated for OFF.
Default: OFF.

data Specifies the 25-bit integer value to be written to the data field or selected register.

hr_data HR capture value.
Default: 0.

NOTE: WCAP in HR Mode: The HR Counter starts on a WCAP instruction execution (in the first loop clock) and will synchronize to the next loop clock. When N2HET is turned on and a capture edge occurs in the first loop clock (where the HR counter hasn't been synchronized to the loop clock), then the captured HR counter value is wrong and is of no use. So the captured HR data in the first loop clock should be ignored.

Execution

```

If (Specified Capture Condition is true on Selected Pin
OR Unconditional capture is selected)
{
    Immediate Data Field = Selected register value;

    If (hr_lr bit == 0) Capture the HR value in Immediate HR Data Field;

    If (Interrupt Enable == 1) HETFLG[n] = 1;      /* n depends on address */
    If ([C28:C27] == 01) Generate request on request line [P25:P23];
    If ([C28:C27] == 11) Generate quiet request on request line [P25:P23];

    Jump to Conditional Address;
}

Jump to Next Program Address;

Prv bit = Current Logic (Lx) value of selected pin; (always executed)

```

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).

23.6.3.24 WCAPE (Software Capture Word and Event Count)

Syntax

```

WCAPE {
    [brk={OFF | ON}]
    [next={label | 9-bit unsigned integer}]
    [reqnum={3-bit unsigned integer}]
    [request={NOREQ | GENREQ | QUIET}]
    [control={OFF | ON}]
    [prv={OFF | ON}]
    [cond_addr={label | 9-bit unsigned integer}]
    pin = {pin number}
    event={NOCOND | FALL | RISE | BOTH}
    [reg={A | B | R | S | T | NONE}]
    [irq={OFF | ON}]
    [ts_data={25-bit unsigned integer}]
    [ec_data={7-bit unsigned integer}]
}
    
```

Figure 23-189. WCAPE Program Field (P31:P0)

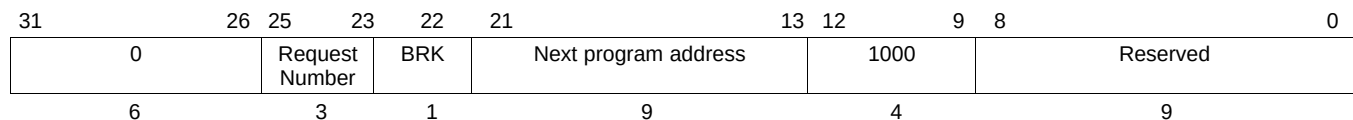


Figure 23-190. WCAPE Control Field (C31:C0)

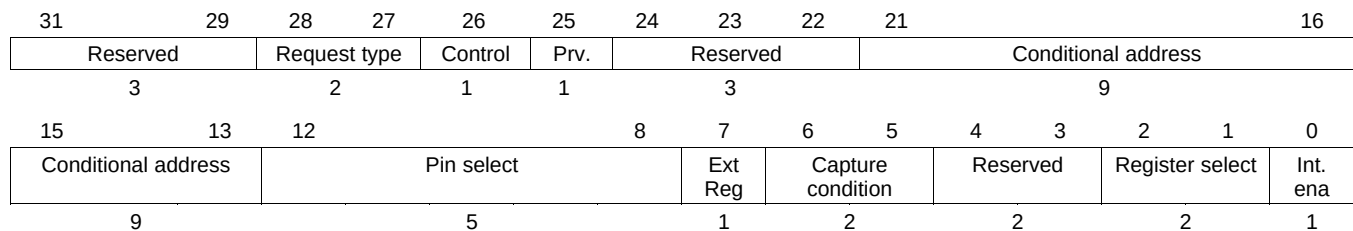
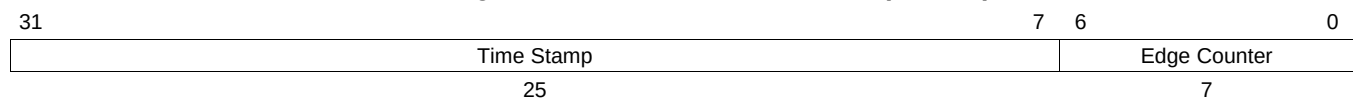


Figure 23-191. WCAPE Data Field (D31:D0)



Cycles One

Register modified None

Description This instruction captures the selected register into the data field [D31:D7] and increments an event counter [D6:D0] if the specified capture condition is true on the selected pin. This instruction can be used with all pins, but the time stamp [D31:D7] has loop resolution only.

event Specifies the event that triggers the capture.

Table 23-97. Event Encoding Format for WCAPE

C6	C5	Capture Condition
0	0	Always
0	1	Capture on falling edge
1	0	Capture on rising edge
1	1	Capture on rising and falling edge

irq ON generates an interrupt when the capture condition is met. No interrupt is generated for OFF.
Default: OFF.

ts_data Specifies the 25-bit integer value for [D31:D7]
Default: 0.

ec_data Specifies the initial 7-bit integer value for [D6:D0].
Default: 0.

Execution

```

If (Specified Capture Condition is true on Selected Pin
OR Unconditional capture is selected)
{
    Immediate Data Field[31:7] = Selected register value;
    Immediate Data Field [6:0] = Immediate Data Field [6:0] + 1;

    If (Interrupt Enable == 1) HETFLG[n] = 1;      /* n depends on address */
    If ([C28:C27] == 01) Generate request on request line [P25:P23];
    If ([C28:C27] == 11) Generate quiet request on request line [P25:P23];

    Jump to Conditional Address;
}

Jump to Next Program Address;

Prv bit = Current Logic (Lx) value of selected pin; (always executed)

```

The specific interrupt flag that is triggered depends on the address from which the instruction is executed, see [Section 23.2.7](#).